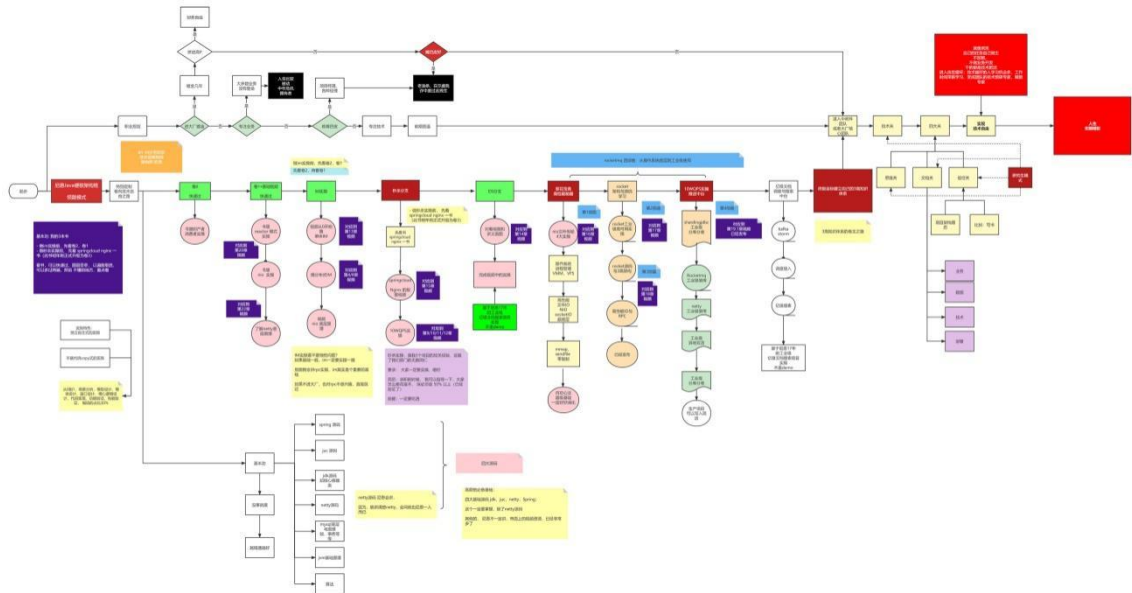


牛逼的职业发展之路

40 岁老架构尼恩用一张图揭秘：Java 工程师的高端职业发展路径，走向食物链顶端的之路

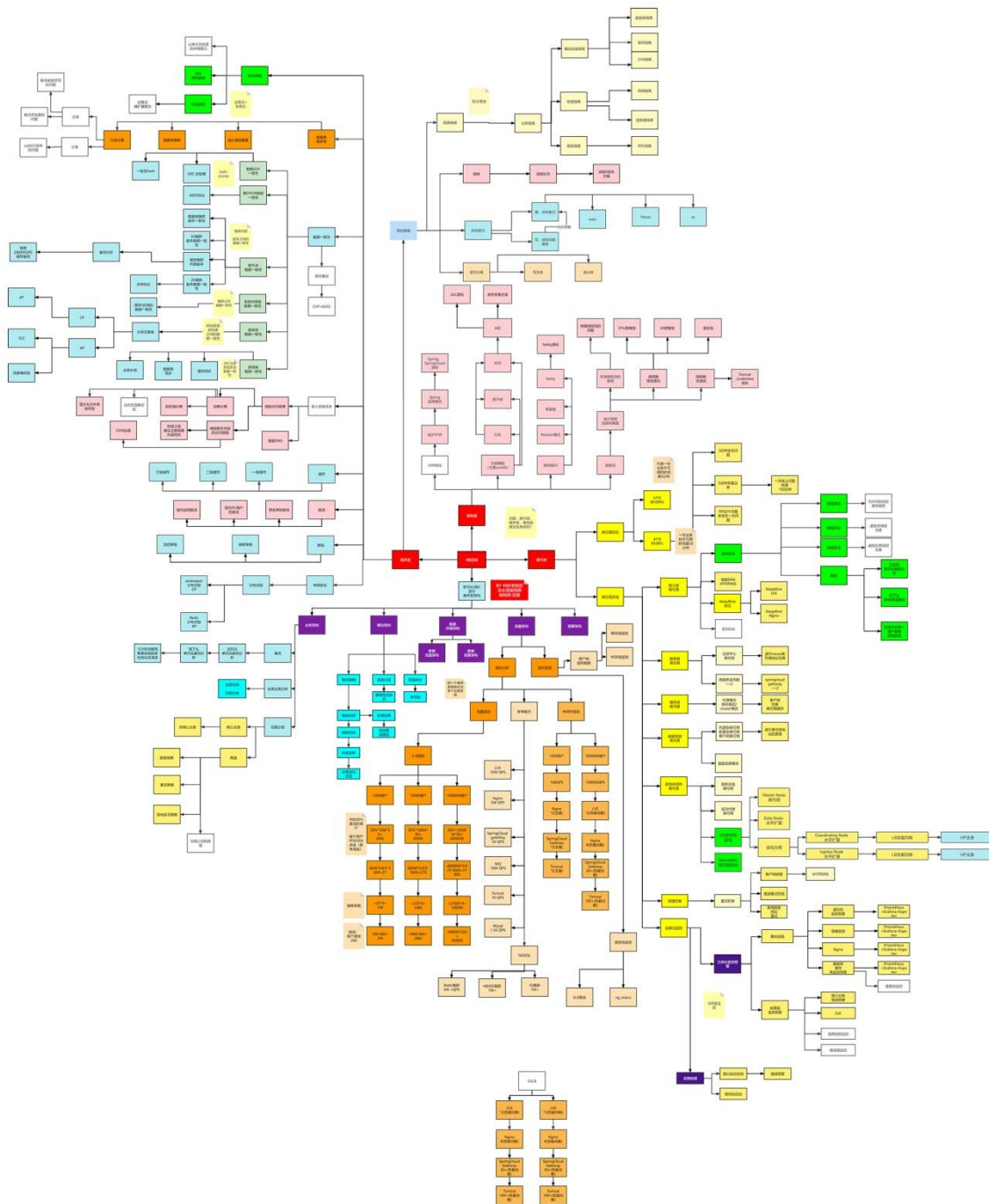
链接：<https://www.processon.com/view/link/618a2b62e0b34d73f7eb3cd7>



史上最全：价值10W的架构师知识图谱

此图梳理于尼恩的多个 3 高生产项目：多个亿级人民币的大型 SAAS 平台和智慧城市项目

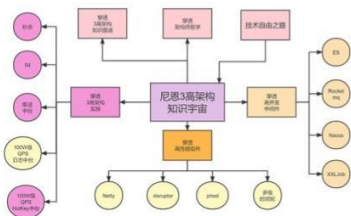
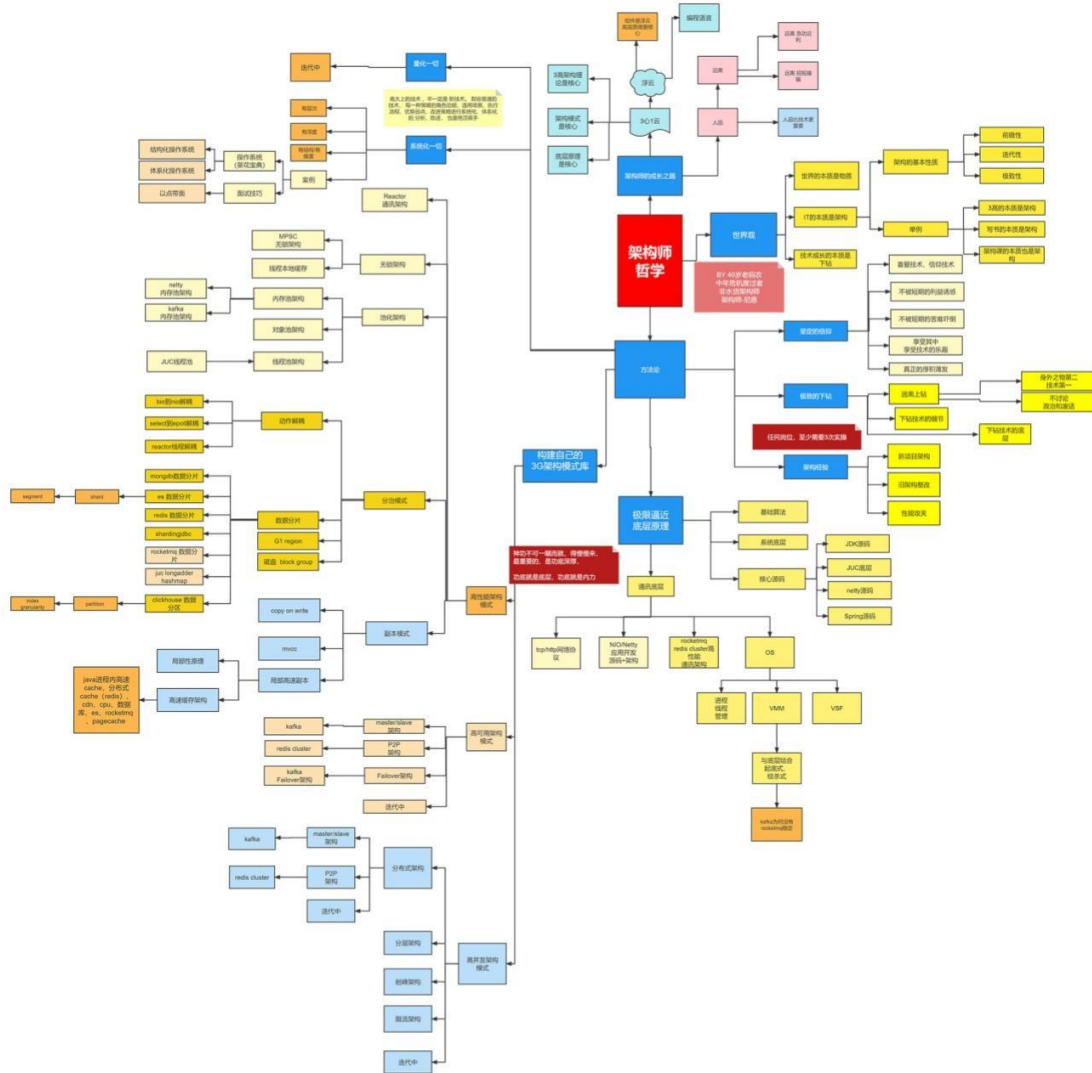
链接：<https://www.processon.com/view/link/60fb9421637689719d246739>



牛逼的架构师哲学

40 岁老架构师尼恩对自己的 20 年的开发、架构经验总结

链接: <https://www.processon.com/view/link/616f801963768961e9d9aec8>



牛逼的3高架构知识宇宙

尼恩 3 高架构知识宇宙，帮助大家穿透 3 高架构，走向技术自由，远离中年危机

链接: <https://www.processon.com/view/link/635097d2e0b34d40be778ab4>



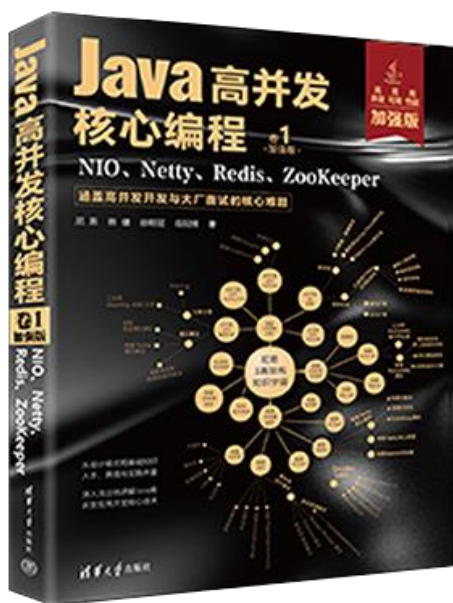
尼恩Java高并发三部曲（卷1加强版）

老版本：《Java 高并发核心编程 卷1：NIO、Netty、Redis、ZooKeeper》（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷1 **加强版**：NIO、Netty、Redis、ZooKeeper》

- 由浅入深地剖析了高并发 IO 的底层原理。
- 图文并茂地介绍了 TCP、HTTP、WebSocket 协议的核心原理。
- 细致深入地揭秘了 Reactor 高性能模式。
- 全面介绍了 Netty 框架，并完成单体 IM、分布式 IM 的实战设计。
- 详尽地介绍了 ZooKeeper、Redis 的使用，以帮助提升高并发、可扩展能力

详情：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



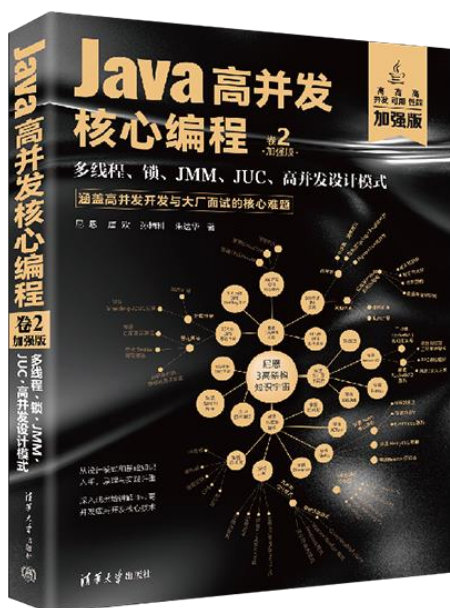
尼恩Java高并发三部曲（卷2加强版）

老版本：《Java 高并发核心编程 卷2：多线程、锁、JMM、JUC、高并发设计模式》
（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷2 **加强版**：多线程、锁、JMM、JUC、高并发设计模式》

- 由浅入深地剖析了 Java 多线程、线程池的底层原理。
- 总结了 IO 密集型、CPU 密集型线程池的线程数预估算法。
- 图文并茂的介绍了 Java 内置锁、JUC 显式锁的核心原理。
- 细致深入地揭秘了 JMM 内存模型。
- 全面介绍了 JUC 框架的设计模式与核心原理，并完成其高核心组件的实战介绍。
- 详尽地介绍了高并发设计模式的使用，以帮助提升高并发、可扩展能力

详情参阅：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



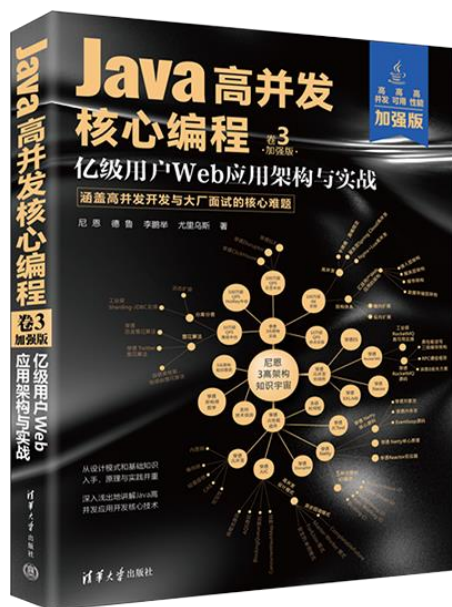
尼恩Java高并发三部曲（卷3加强版）

老版本：《SpringCloud Nginx 高并发核心编程》（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷3 **加强版**：亿级用户 Web 应用架构与实战》

- 在当今的面试场景中，3 高知识是大家面试必备的核心知识，本书基于亿级用户 3 高 Web 应用的架构分析理论，为大家对 3 高架构系统做一个系统化和清晰化的介绍。
- 从 Java 静态代理、动态代理模式入手，抽丝剥茧地解读了 SpringCloud 全家桶中 RPC 核心原理和执行过程，这是高级 Java 工程师面试必备的基础知识。
- 从Reactor 反应器模式入手，抽丝剥茧地解读了Nginx 核心思想和各配置项的底层知识和原理，这是高级 Java 工程师、架构师面试必备的基础知识。
- 从观察者模式入手，抽丝剥茧地解读了 RxJava、Hystrix 的核心思想和使用方法，这也是高级 Java 工程师、架构师面试必备的基础知识。

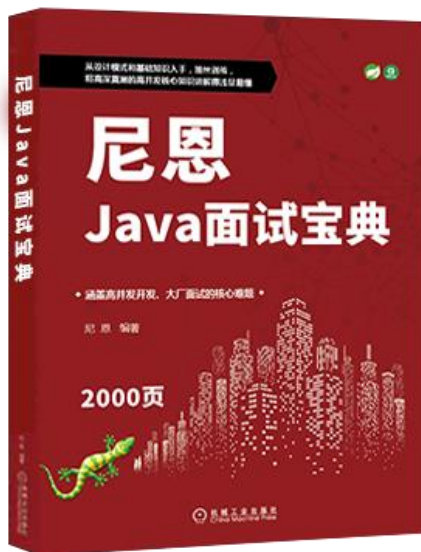
详情：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



尼恩Java面试宝典

35 个专题（卷王专供+ 史上最全 + 2023 面试必备）

详情：<https://www.cnblogs.com/crazymakercircle/p/13917138.html>



名称	
专题01: JVM面试题 (卷王专供 + 史上最全 + 2023面试必备) -V10-from-尼恩Java面试宝典-release.pdf	
专题02: Java算法面试题 (卷王专供 + 史上最全 + 2023面试必备) -V2 -from-Java面试红宝书-release.pdf	
专题03: Java基础面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书 -release.pdf	
专题04: 架构设计面试题 (卷王专供+ 史上最全 + 2023面试必备) -V10-from-Java面试红宝书-release.pdf	
专题05: Spring面试题_专题06: SpringMVC_专题07: Tomcat面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-尼恩面试宝典-release.pdf	
专题08: SpringBoot面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题09: 网络协议面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题10: TCP/IP协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题11: JUC并发包与容器类 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题12: 设计模式面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题13: 死锁面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题14: Redis 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V5-from-Java面试红宝书-release.pdf	
专题15: 分布式锁 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题16: Zookeeper 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题17: 分布式事务面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题18: 一致性协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题19: Zab协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题20: Paxos 协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题21: raft 协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题22: Linux面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题23: Mysql 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V11-from-Java面试红宝书-release.pdf	
专题24: SpringCloud 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V12-from-Java面试红宝书-release.pdf	
专题25: Netty 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题26: 消息队列面试题: RabbitMQ、Kafka、RocketMQ (卷王专供+ 史上最全 + 2023面试必备) -V10-from-Java面试红宝书-release.pdf	
专题27: 内存泄漏 内存溢出 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题28: JVM 内存溢出 实战 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题29: 多线程面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题30: HR面试题: 过五关斩六将后, 小心阴沟翻船! (史上最全、避坑宝典) -V2-from-Java面试红宝书-release.pdf	
专题31: Hash链表面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题32: 大厂面试的基本流程和面试准备 (阿里、腾讯、网易、京东、头条.....) -V2-from-Java面试红宝书-release.pdf	
专题33: BST、AVL、RBT红黑树、三大核心数据结构 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题34: Elasticsearch面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-Java面试红宝书-release.pdf	
专题35: Mybatis面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-尼恩Java面试宝典-release.pdf	

高性能核心组件穿透的意义

要想成为高手，就要穿透式、起底式、绞杀式的掌握 顶级组件、王者组件。

唯有如此，才能成为技术王者。

从中吸取思想和精华，为大家自己的业务CRUD所有。

接下来，和尼恩一起，开始穿透之旅吧：

- 高性能核心组件之1：

穿透**“IO之王 Netty”** 架构和源码

- 穿透高性能核心组件之2：

“队列之王 Disruptor” 架构和源码（Disruptor 红宝书）

- 穿透高性能核心组件之3：

穿透**“缓存之王 Caffeine”** 架构和源码（Caffeine 红宝书）

- 穿透高性能核心组件之4：

穿透**“链路之王 Skywalking”** 架构和源码

此文，带大家 穿透高性能核心组件之3：

穿透“缓存之王 Caffeine” 架构和源码

所以，此文又名：

Caffeine 红宝书

文章会不断升级，最新版本，请关注尼恩的朋友圈。

本地缓存的使用场景

场景1：突发性hotkey场景

突发性hotkey导致的分布式缓存性能变差、缓存击穿的场景

什么是热Key

在某段时间内某个key收到的访问次数，显著高于其他key时，我们可以将其称之为热key。

例如，某redis的每秒访问总量为10000，而其中某个key的每秒访问量达到了7000，这种情况下，我们称该key为热key。

热key带来的问题

- 1.热Key占用大量的Redis CPU时间使其性能变差并影响其它请求；
- 2.Redis Cluster中各node流量不均衡，造成Redis Cluster的分布式优势无法被Client利用，一个分片负载很高，而其它分片十分空闲从而产生读/写热点问题；
- 3.热Key的请求压力数量超出Redis的承受能力造成**缓存击穿**，此时大量强求将直接指向后端存储将其打挂并影响到其它业务；

热key出现的典型业务

预期外的访问量激增，如突然出现的爆款商品，访问量暴涨的热点新闻，直播间某大主播搞活动大量的刷屏点赞。

解决方案

通过分布式计算来探测热点key

分布式计算组件，计算出来之后，并通知集群内其他机器。

其他机器，本地缓存HotKey，

场景2：常规性hotkey场景

部门组织机构数据

人员类型数据

解决方案

本地缓存HotKey，

通过发布订阅解决数据一致性问题

本地缓存的主要技术

Java缓存技术可分为分布式缓存和本地缓存，分布式缓存在后面的 100Wqps 三级缓存组件中，再细致介绍。

先看本地缓存。

本地缓存的代表技术主要有HashMap，Guava Cache，Caffeine和Encache。

HashMap

通过Map的底层方式，直接将需要缓存的对象放在内存中。

优点：简单粗暴，不需要引入第三方包，比较适合一些比较简单的场景。

缺点：没有缓存淘汰策略，定制化开发成本高。

Guava Cache

Guava Cache是由Google开源的基于LRU替换算法的缓存技术。

但Guava Cache由于被下面即将介绍的Caffeine全面超越而被取代。

优点：支持最大容量限制，两种过期删除策略（插入时间和访问时间），支持简单的统计功能。

缺点：springboot2和spring5都放弃了对Guava Cache的支持。

Caffeine

Caffeine采用了W-TinyLFU（LUR和LFU的优点结合）开源的缓存技术。

缓存性能接近理论最优，属于是Guava Cache的增强版。

Encache

Ehcache是一个纯java的进程内缓存框架，具有快速、精干的特点。

是hibernate默认的cacheprovider。

优点：

支持多种缓存淘汰算法，包括LFU，LRU和FIFO；

缓存支持堆内缓存，堆外缓存和磁盘缓存；

支持多种集群方案，解决数据共享问题。

说明：本文会分析 Caffeine的源码，后面可以对照分析一下 Guava Cache的源码。

本地缓存的优缺点

1. 快但是量少：访问速度快，但无法进行大数据存储

本地缓存相对于分布式缓存的好处是，由于数据不需要跨网络传输，故性能更好，

但是由于占用了应用进程的内存空间，如 Java 进程的 JVM 内存空间，故不能进行大数据量的数据存储。

2. 需要解决数据一致性问题：集群的数据更新问题

与此同时，本地缓存只支持被该应用进程访问，一般无法被其他应用进程访问，故在应用进程的集群部署当中，

如果对应的数据库数据，存在数据更新，则需要同步更新不同部署节点的本地缓存的数据来保证数据一致性，

复杂度较高并且容易出错，如基于 rocketmq 的发布订阅机制来同步更新各个部署节点。

3.更新低可靠，容易丢失：数据随应用进程的重启而丢失

由于本地缓存的数据是存储在应用进程的内存空间的，所以当应用进程重启时，本地缓存的数据会丢失。

所以对于需要更改然后持久化的数据，需要注意及时保存，否则可能会造成数据丢失。

和缓存相关的几个核心概念

缓存污染

缓存污染，指留存在缓存中的数据，实际不会被再次访问了，但又占据了缓存空间。

换句话说，由于缓存空间有限，热点数据被置换或者驱逐出去了，而一些后面不用到的数据却反而被留下来，从而**缓存数据命中率**急剧下降

要解决缓存污染的关键点是**能识别出热点数据，或者未来更有可能被访问到的数据。**

换句话说: 是要提升 缓存数据命中率

缓存命中率

缓存命中率是一个缓存组件是否好用的 核心指标之一，而命中率又和缓存组件本身的缓存数据淘汰算法息息相关。

命中：可以直接通过缓存获取到需要的数据。

不命中：无法直接通过缓存获取到想要的数据，需要再次查询数据库或者执行其它的操作。原因可能是由于缓存中根本不存在，或者缓存已经过期。

通常来讲，**缓存的命中率越高则表示使用缓存的收益越高，应用的性能越好**（响应时间越短、吞吐量越高），抗并发的能力越强。

由此可见，在高并发的互联网系统中，缓存的命中率是至关重要的指标。

而 缓存的命中率 的提升，和 缓存数据淘汰算法，密切相关。

常见的缓存数据淘汰算法

主要的缓存数据淘汰算法（也叫做缓存数据驱逐算法），有三种：

- FIFO (First in first out) 先进先出算法

如果一个数据最先进入缓存中，则应该最早淘汰掉。

- LRU (Least recently used) 最近最少使用算法

如果数据**最近**被访问过，那么将来被访问的几率也更高。

- LFU (Least frequently used) 最近很少使用算法

如果一个数据在最近一段时间内**使用次数很少，使用频率最低**，那么在将来一段时间内被使用的可能性也很小。

参考一下：Redis的8种缓存淘汰策略

从能否解决缓存污染这一维度来分析Redis的8种缓存淘汰策略：

- noeviction策略：不会淘汰数据，解决不了。
- volatile-ttl策略：给数据设置合理的过期时间。当缓存写满时，会淘汰剩余存活时间最短的数据，避免滞留在缓存中，造成污染。
- volatile-random策略：随机选择数据，无法把不再访问的数据筛选出来，会造成缓存污染。
- volatile-lru策略：LRU策略只考虑数据的访问时效，对只访问一次的数据，不能很快筛选出来。
- volatile-lfu策略：LFU策略在LRU策略基础上进行了优化，筛选数据时优先筛选并淘汰访问次数少的数据。
- allkeys-random策略：随机选择数据，无法把不再访问的数据筛选出来，会造成缓存污染。
- allkeys-lru策略：LRU策略只考虑数据的访问时效，对只访问一次的数据，不能很快筛选出来。
- allkeys-lfu策略：LFU策略在LRU策略基础上进行了优化，筛选数据时优先筛选并淘汰访问次数少的数据。

缓存淘汰策略	解决缓存污染
noeviction策略	不能
volatile-ttl策略	能
volatile-random策略	不能
volatile-lru策略	不能
volatile-lfu策略	能
allkeys-random策略	不能
allkeys-lru策略	不能
allkeys-lfu策略	能

1 FIFO 先进先出算法

FIFO（First in First out）先进先出。可以理解为是一种类似队列的算法实现

算法：

如果一个数据最先进入缓存中，则应该最早淘汰掉。

换句话说：最先进来的数据，被认为**在未来被访问的概率也是最低的**，

因此，当规定空间用尽且需要放入新数据的时候，会优先淘汰最早进来的数据

优点:

最简单、最公平的一种数据淘汰算法，逻辑简单清晰，易于实现

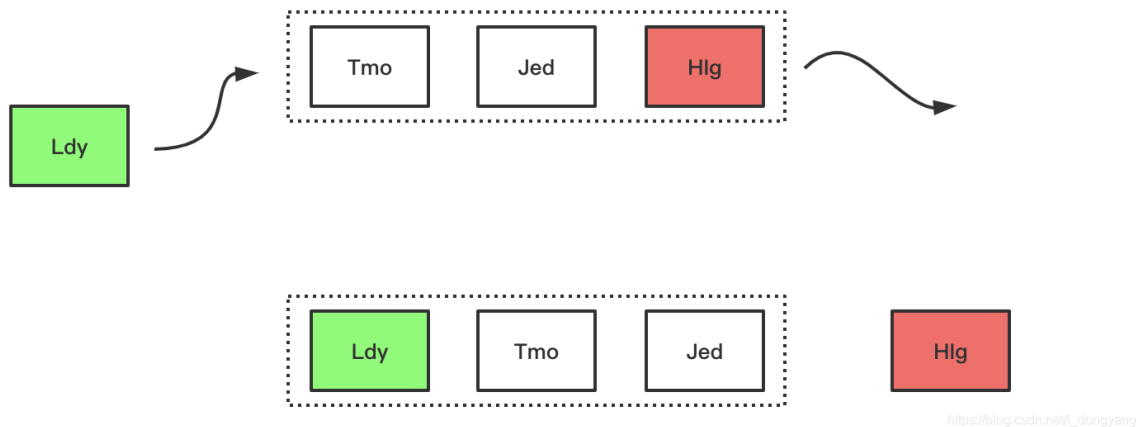
缺点:

这种算法逻辑设计所实现的缓存的命中率是比较低的，因为没有任何额外逻辑能够尽可能的保证常用数据不被淘汰掉

演示:

下面简单演示了FIFO的工作过程，假设存放元素尺寸是3，且队列已满，放置元素顺序如下图所示，当来了一个新的数据“Ldy”后，因为元素数量到达了阈值，则首先要进行淘汰置换操作，然后加入新元素，

操作如图展示：



2 LRU —— 适用于 局部突发流量场景

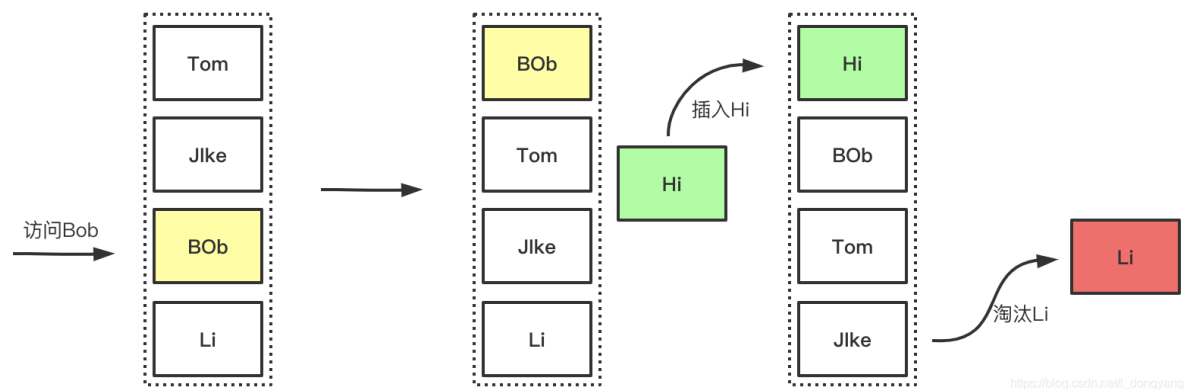
LRU (The Least Recently Used) 最近最久未使用算法。

算法:

如果一个数据最近很少被访问到，那么被认为在未来被访问的概率也是最低的，当规定空间用尽且需要放入新数据的时候，会优先淘汰最久未被访问的数据

演示:

下图展示了LRU简单的工作过程，访问时对数据的提前操作，以及数据满且添加新数据的时候淘汰的过程的展示如下：



此处介绍的LRU是有明显的缺点，

如上所述，对于偶发性、周期性的数据没有良好的抵抗力，很容易就造成缓存的污染，影响命中率，

因此衍生出了很多的LRU算法的变种，用以处理这种偶发冷数据突增的场景，

比如：LRU-K、Two Queues等，目的就是当判别数据为偶发或周期的冷数据时，不会存入空间内，从而降低热数据的淘汰率。

优点：

LRU 实现简单，在一般情况下能够表现出很好的命中率，是一个“性价比”很高的算法。

LRU可以有效的对访问比较频繁的数据进行保护，也就是针对热点数据的命中率提高有明显的效果。

LRU局部突发流量场景，对突发性的稀疏流量（sparse bursts）表现很好。

缺点：

在存在 周期性的局部热点 数据场景，有大概率可能造成**缓存污染**。

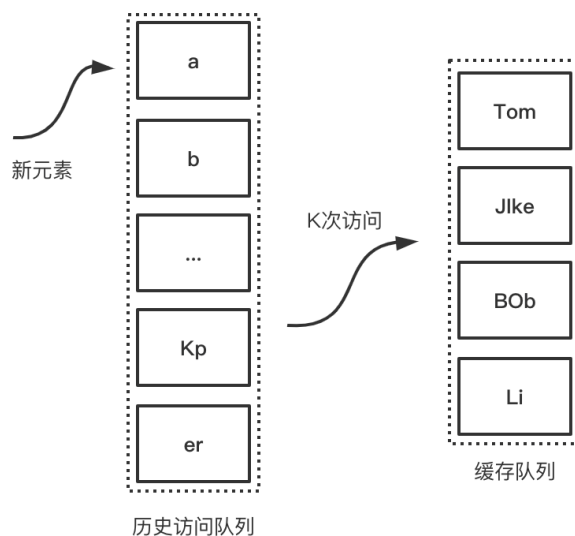
最近访问的数据，并不一定是周期性数据，比如把全量的数据做一次迭代，那么LRU 会产生较大的缓存污染，因为周期性的局部热点数据，可能会被淘汰。

演进一：LRU-K

下图展示了LRU-K的简单工作过程，

简单理解，LRU中的K是指数据被访问K次，传统LRU与此对比则可以认为传统LRU是LRU-1。

可以看到LRU-K有两个队列，新来的元素先进入到历史访问队列中，该队列用于记录元素的访问次数，采用的淘汰策略是LRU或者FIFO，当历史队列中的元素访问次数达到K的时候，才会进入缓存队列。



<https://blog.csdn.net/dongyang>

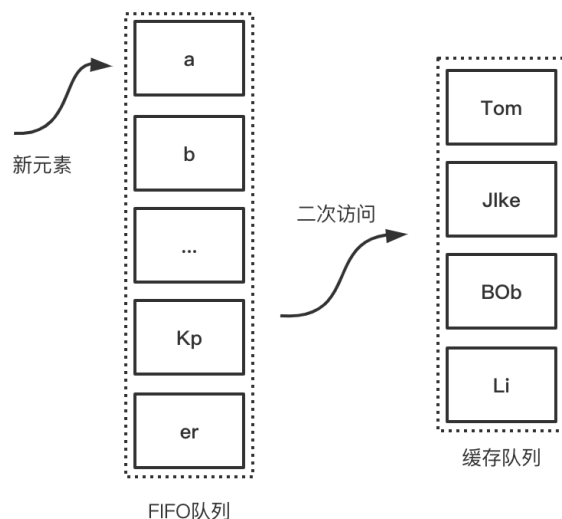
演进二：Two Queues

下图展示了Two Queues的工作过程，

Two Queues与LRU-K相比，他也同样是两个队列，不同之处在于，他的队列一个是缓存队列，一个是FIFO队列，

当新元素进来的时候，首先进入FIFO队列，当该队列中的元素被访问的时候，会进入LRU队列，

过程如下：



实际案例

Guava Cache是Google Guava工具包中的一个非常方便易用的本地化缓存实现，基于LRU算法实现，支持多种缓存过期策略。由于Guava的大量使用，Guava Cache也得到了大量的应用。

Guava的loading cache是使用LRU 的淘汰策略，但是很多场景，最近的数据不一定热，存在周期性的热点数据，而LRU反而容易把稍旧的周期热数据挤出去，

3 LFU —— 适用于 局部周期性流量场景

LFU (The Least Frequently Used) 最近很少使用算法，

如果一个数据在最近一段时间内**使用次数很少，使用频率最低**，那么在将来一段时间内被使用的可能性也很小。

与LRU的区别在于LRU是以时间先后来衡量，LFU是以时间段内的使用次数衡量

算法：

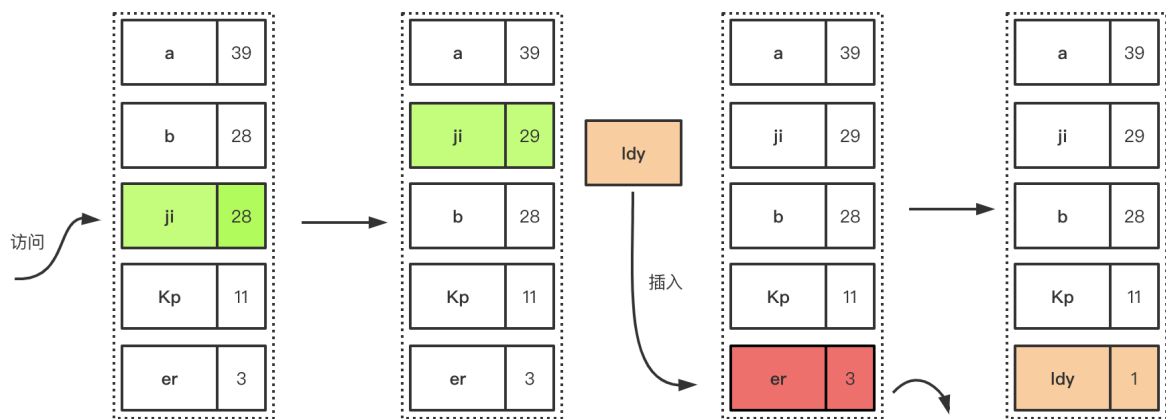
如果一个数据在一定时间内被访问的次数很低，那么被认为在未来被访问的概率也是最低的，

当规定空间用尽且需要放入新数据的时候，会优先淘汰时间段内访问次数最低的数据

演示：

下面描述了LFU的简单工作过程，首先是访问元素增加元素的访问次数，从而提高元素在队列中的位置，降低淘汰优先级，

后面是插入新元素的时候，因为队列已经满了，所以优先淘汰在一定时间间隔内访问频率最低的元素



优点:

LFU适用于 局部周期性流量场景，在这个场景下，比LRU有更好的缓存命中率。

在 局部周期性流量场景下， LFU是以次数为基准，所以更加准确，自然能有效的保证和提高命中率

缺点:

LFU 有几个的缺点:

第一，因为LFU需要记录数据的访问频率，因此需要额外的空间；

第二，它需要给每个记录项维护频率信息，每次访问都需要更新，这是个巨大的开销；

第三，在存在 局部突发流量场景下，有大概率可能造成**缓存污染**，算法命中率会急剧下降，这也是他最大弊端。所以，LFU 对突发性的稀疏流量 (sparse bursts) 是无效的。

why: LFU 对突发性的稀疏流量无效呢?

总体来说，LFU 按照访问次数或者访问频率取胜，这个次数有一个累计的长周期，导致前期经常访问的数据，访问次数很大，或者说权重很高，

新来的缓存数据，哪怕他是突发热点，但是，新数据的访问次数累计的时间太短，在老人面试，是个矮个子

LFU 就想一个企业，有点论资排辈，排斥性新人，新人进来，都需要吃苦头，哪怕他是明日之星

所以，LFU 算法中，老的记录已经占用了缓存，**过去的一些大量被访问的记录**，在将来不一定会继续是热点数据，但是就一直把“坑”占着了，而那些偶然的突破热点数据，不太可能会被保留下来，而是被淘汰。

所以，在存在突发性的稀疏流量下，LFU中的**偶然的、稀疏的突发流量**在访问频率上，不占优势，很容易被淘汰，造成缓存污染和未来缓存命中率下降。

LRU 和 LFU 的对比

LRU 实现简单，在一般情况下能够表现出很好的命中率，是一个“性价比”很高的算法，平时也很常用。

LRU 的优点之一对突发性的稀疏流量 (sparse bursts) 表现很好。

但是，LRU 这个优点也带来一个缺点:

对于周期性的局部热点数据，有大概率可能造成缓存污染。

最近访问的数据，并不一定是周期性数据，比如把全量的数据做一次迭代，那么LRU会产生较大的缓存污染，因为周期性的数据，可能会被淘汰。

如果是周期性局部热点数据，那么LFU可以达到最高的命中率。

但是 LFU 有三个大的缺点：

第一，因为LFU需要记录数据的访问频率，因此需要额外的空间；

第二，它需要给每个记录项维护频率信息，每次访问都需要更新，这是个巨大的时间开销；

第三，对突发性的局部热点数据/稀疏流量（sparse bursts），算法命中率会急剧下降，这也是他最大弊端。

无论 LRU 还是 LFU 都有其各自的缺点，不过，现在已经有很多针对其缺点而改良、优化出来的变种算法。

说明：本文会持续更新，最新PDF电子版本，请从下面的连接获取：[语雀](#) 或者 [码云](#)

4、TinyLFU

TinyLFU 就是其中一个优化算法，它是专门为了解决 LFU 上述提到的三个问题而被设计出来的。

第1：如何减少访问频率的保存，所带来的空间开销

第2：如何减少访问记录的更新，所带来的时间开销

第3：如果提升对局部热点数据的 算法命中率

解决第1个问题/第2个问题是采用了 Count-Min Sketch 算法。

解决第二个问题是让老的访问记录，尽量保证“新鲜度”（Freshness Mechanism）

首先：如何解决 访问频率 维护的时间开销和空间开销

解决措施：使用Count-Min Sketch算法存储访问频率，极大的节省空间；并且减少hash碰撞。

关于Count-Min Sketch算法，可以看作是布隆过滤器的同源的算法，

假如我们用一个hashmap来存储每个元素的访问次数，那这个量级是比较大的，并且hash冲突的时候需要做一定处理，否则数据会产生很大的误差，

如果用hashmap的方式，相同的下标变成链表，这种方式会占用很大的内存，而且速度也不是很快。

其实一个hash函数会冲突是比较低的，布隆过滤器 的优化之一，设置多个hash函数，多个hash函数，个个都冲突的概率就微乎其微了。

Count-Min Sketch算法将一个hash操作，扩增为多个hash，这样原来hash冲突的概率就降低了几个等级，且当多个hash取得数据的时候，取最低值，也就是Count Min的含义所在。

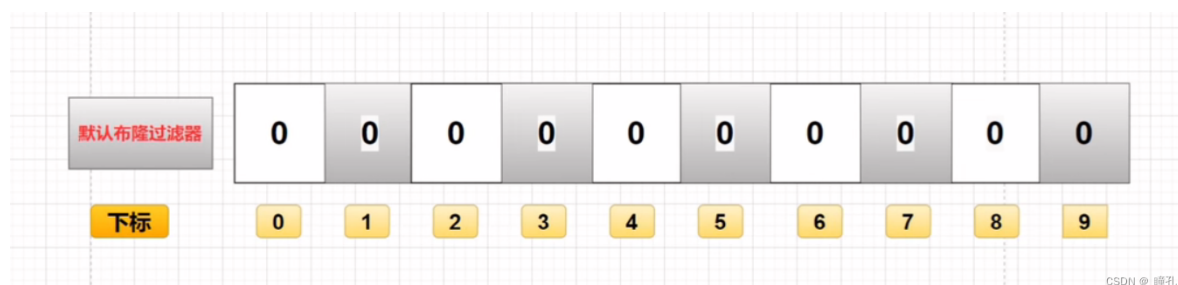
Sketch 是草图、速写的意思。

将要介绍的 Count-Min Sketch 的原理跟 Bloom Filter 一样，只不过 Bloom Filter 只有 0 和 1 的值，那么你可以把 Count-Min Sketch 看作是“数值”版的 Bloom Filter。

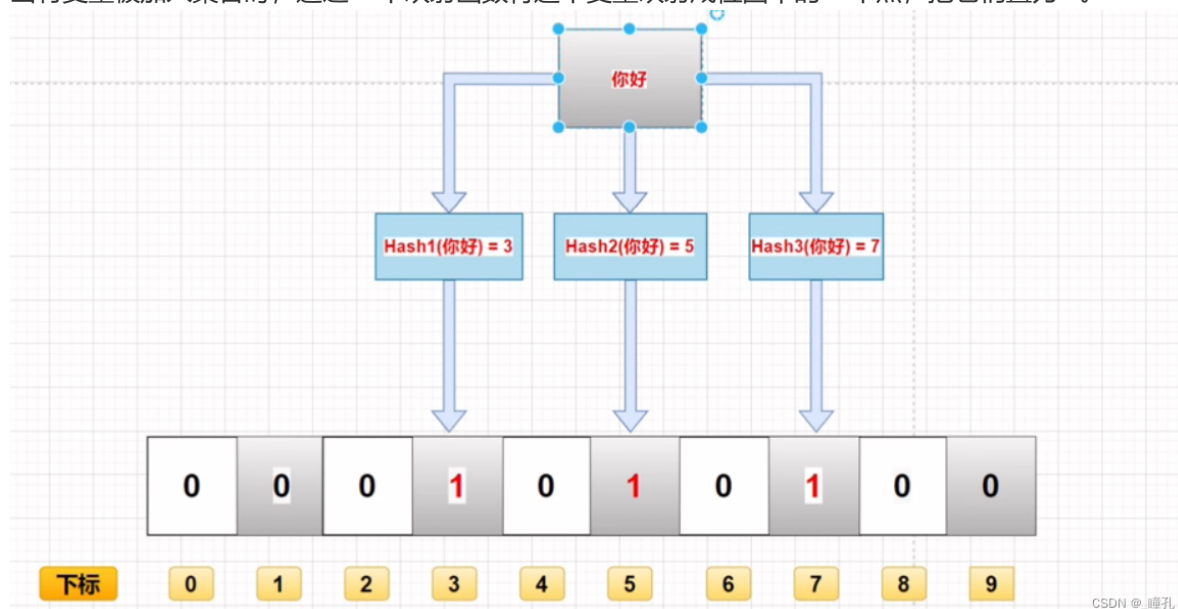
布隆过滤器原理

布隆过滤器是由一个固定大小的二进制向量或者位图（bitmap）和一系列映射函数组成的。

在初始状态时，对于长度为 m 的位数组，它的所有位都被置为0，如下图所示：



当有变量被加入集合时，通过 K 个映射函数将这个变量映射成位图中的 K 个点，把它们置为 1。



查询某个变量的时候我们只要看看这些点是不是都是 1 就可以大概率知道集合中有没有它了

- 如果这些点有任何一个 0，则被查询变量一定不在；
- 如果都是 1，则被查询变量很可能存在。为什么说是可能存在，而不是一定存在呢？那是因为映射函数本身就是散列函数，散列函数是会有碰撞的。

误判率：布隆过滤器的误判是指多个输入经过哈希之后在相同的bit位置1了，这样就无法判断究竟是哪个输入产生的，因此误判的根源在于相同的 bit 位被多次映射且置 1。这种情况也造成了布隆过滤器的删除问题，因为布隆过滤器的每一个 bit 并不是独占的，很有可能多个元素共享了某一位。如果我们直接删除这一位的话，会影响其他的元素。

特性：

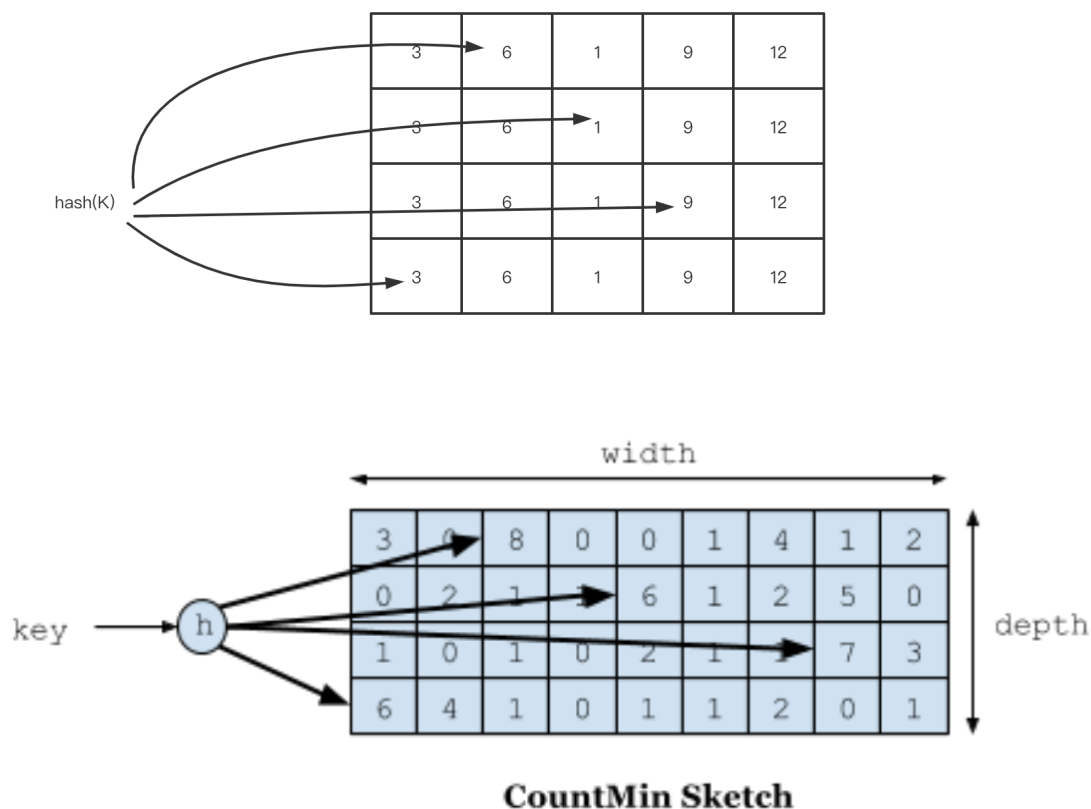
- 一个元素如果判断结果为存在的时候元素不一定存在，但是判断结果为不存在的时候则一定不存在。
- 布隆过滤器可以添加元素，但是不能删除元素。因为删掉元素会导致误判率增加。

说明：本文会持续更新，最新PDF电子版本，请从下面的连接获取：[语雀](#) 或者 [码云](#)

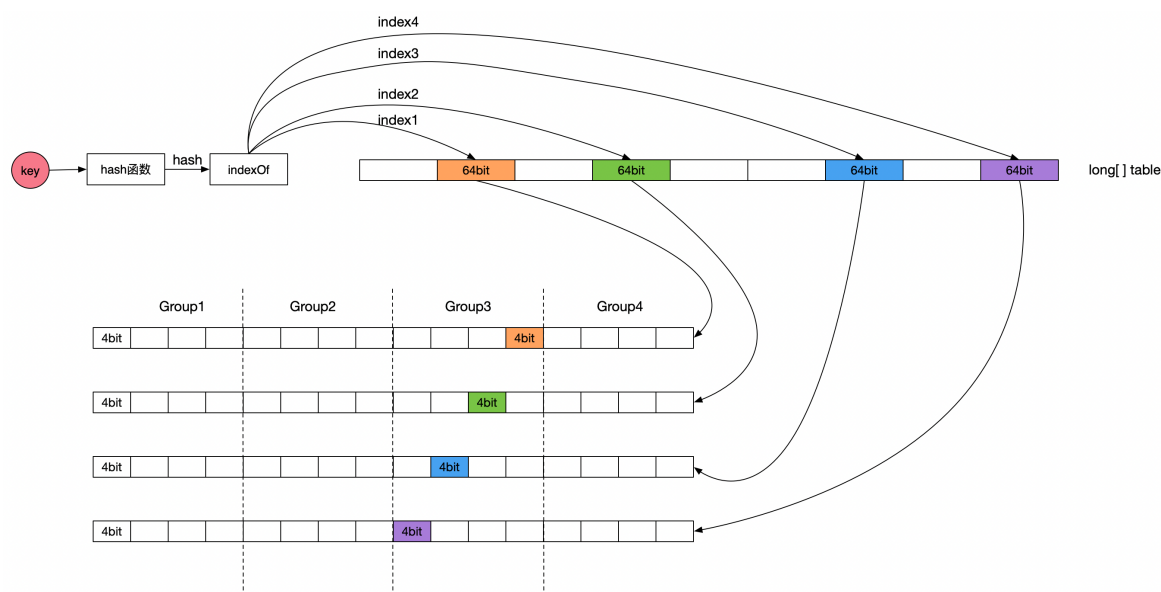
Count-Min Sketch算法原理

下图展示了Count-Min Sketch算法简单的工作原理：

1. 假设有四个hash函数，每当元素被访问时，将进行次数加1；
2. 此时会按照约定好的四个hash函数进行hash计算找到对应的位置，相应的位置进行+1操作；
3. 当获取元素的频率时，同样根据hash计算找到4个索引位置；
4. 取得四个位置的频率信息，然后根据Count Min取得最低值作为本次元素的频率值返回，即 $\text{Min}(\text{Count})$;



Count-Min Sketch算法详细实现方案如下：



如何进行Count-Min Sketch访问次数的空间开销？

用4个hash函数会存访问次数，那空间就是4倍了。怎么优化呢

解决办法是：

访问次数超过15次其实是很热的数据了，没必要存太大的数字。所以我们用4位就可以存到15了。

一个long有64位，可以存16个4位。

一个访问次数占4个位，一个long有64位，可以存 16个访问次数，4个访问一次一组的话，一个long可以分为4组。

一个 key 对应到 4个hash 值，也就是 4个 访问次数，那么，一个long 可以分为存储 4个Key的 访问次数。

最终，一个long对应的数组大小其实是容量的4倍了。

其次，如果提升对局部热点数据的 算法命中率

答案是，保鲜机制

为了让缓存保证“新鲜度”，

可以简单理解为：访问次数太大，越不新鲜，适当降低，保证新鲜度。

所以，Caffeine 有一个 Freshness Mechanism,

Caffeine 会剔除掉过往频率很高，但之后不经常的缓存，。

做法很简答，

就是当整体的统计计数（当前所有记录的频率统计之和，这个数值内部维护）达到某一个值时，那么所有记录的频率统计除以 2。

说明：本文会持续更新，最新PDF电子版本，请从下面的连接获取：[语雀](#) 或者 [码云](#)

TinyLFU 的算法流程

TinyLFU's architecture is illustrated in Figure 1.

Here, the cache eviction policy picks a cache victim, while TinyLFU decides if replacing the cache victim with the new item is expected to increase the hit-ratio.

当缓存空间不够的时候，TinyLFU 找到 要淘汰的元素（the cache victim），也就是使用频率最小的元素，

然后 TinyLFU 决定 将新元素放入缓存，替代 将 要淘汰的元素（the cache victim）

具体的流程如下：

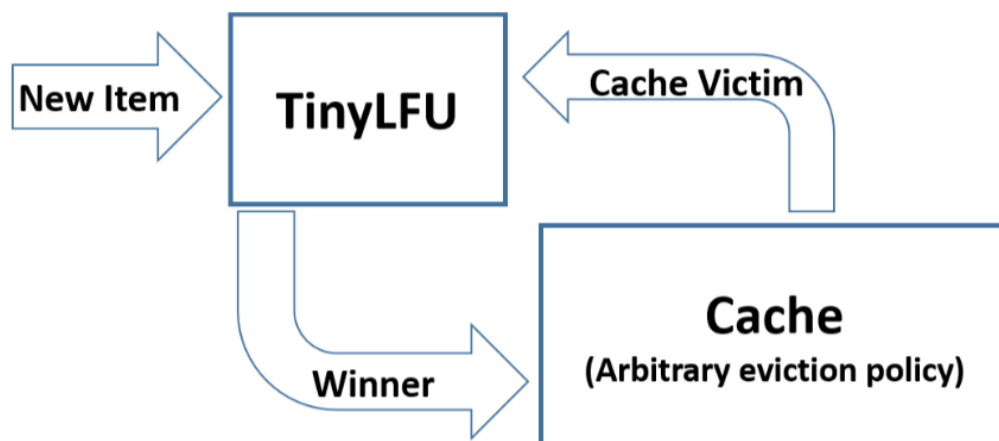


Figure 1: A general cache augmented with TinyLFU

5 W-TinyLFU

Caffeine 通过测试发现 TinyLFU 在面对突发性的稀疏流量（sparse bursts）时表现很差，
why?
因为新的记录（new items）还没来得及建立足够的频率就被剔除出去了，这就使得命中率下降。

W-TinyLFU是如何演进过来的呢？

首先 W-TinyLFU 看名字就能大概猜出来，它是 LFU 的变种，也是TinyLFU的变种，当然，也是一种缓存淘汰算法。

W-TinyLFU是如何演进过来的呢？

前面讲到：

- LRU能很好的 处理 局部突发流量
- LFU能很好的 处理 局部周期流量

so，取其精华去其糟粕，结合二者的优点

W-TinyLFU = LRU + LFU

当然，总是有个是**大股东**，这就是 LFU，或者说是 TinyLFU

so： W-TinyLFU（Window Tiny Least Frequently Used）是对TinyLFU的的优化和加强，加入 LRU 以应对局部突发流量，从而实现缓存命中率的最优。

W-TinyLFU的数据架构

W-TinyLFU 是怎么引入 LRU 的呢？他增加了一个 **W-LRU窗口队列** 的组件。

当一个数据进来的时候，会进行筛选比较，进入**W-LRU窗口队列**，经过淘汰后进入Count-Min Sketch 算法过滤器，通过访问访问频率判决，是否进入缓存。

W-TinyLFU 的设计如下所示：

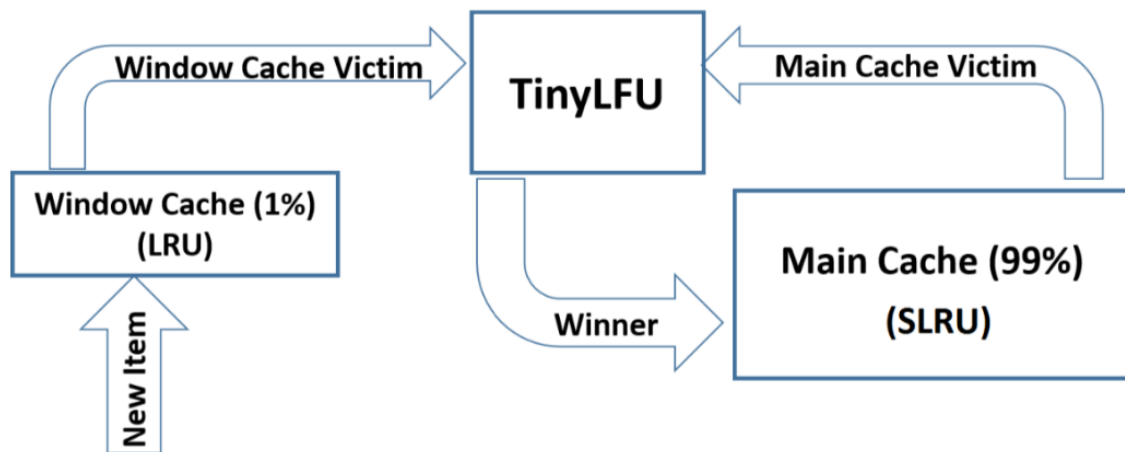


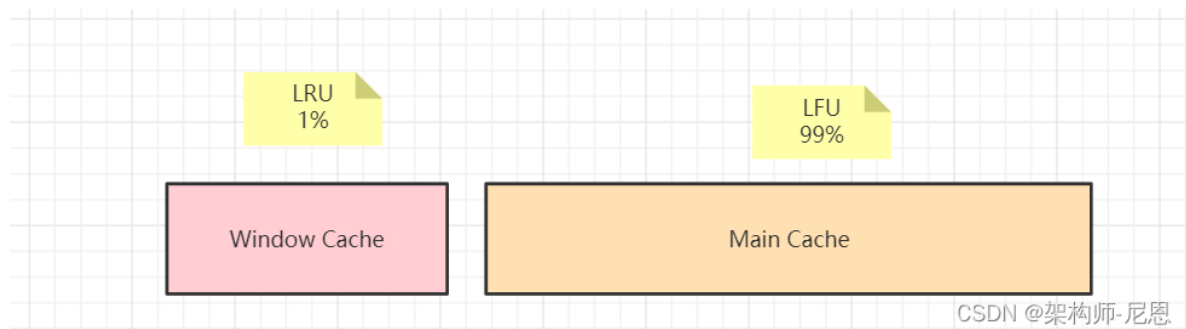
Figure 5: Window TinyLFU scheme

- W-LRU窗口队列 用于应 对 局部突发流量
- TinyLFU 用于 应对 局部周期流量

如果一个数据最近被访问的次数很低，那么被认为在未来被访问的概率也是最低的，当规定空间用尽的时候，会优先淘汰最近访问次数很低的数据；

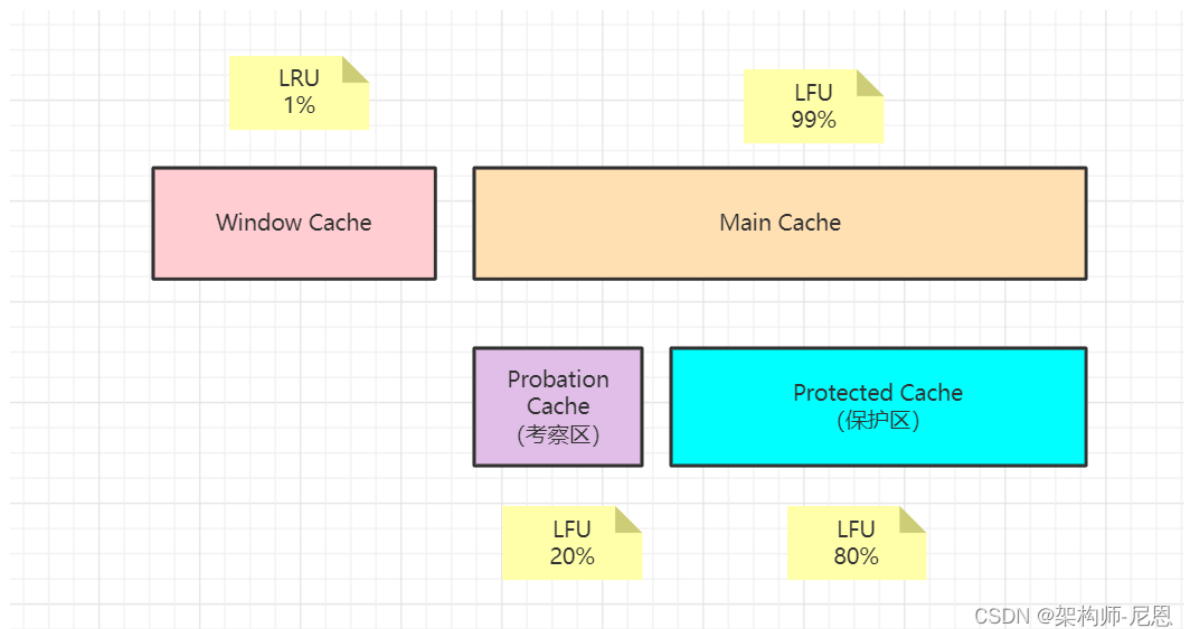
进一步的分治和解耦

W-TinyLFU将缓存存储空间分为两个大的区域：Window Cache和Main Cache，



Window Cache是一个标准的LRU Cache，Main Cache则是一个SLRU（Segmented LRU）cache，

Main Cache进一步划分为Protected Cache（保护区）和Probation Cache（考察区）两个区域，这两个区域都是基于LRU的Cache。



Protected 是一个受保护的区域，该区域中的缓存项不会被淘汰。

而且经过实验发现当 window 区配置为总容量的 1%，剩余的 99% 当中的 80% 分给 protected 区，20% 分给 probation 区时，这时整体性能和命中率表现得最好，所以 Caffeine 默认的比例设置就是这个。

不过这个比例 Caffeine 会在运行时根据统计数据 (statistics) 去动态调整，如果你的应用程序的缓存随着时间变化比较快的话，或者说具备的突发特点数据多，那么增加 window 区的比例可以提高命中率，

如果周期性热地数据多，缓存都是比较固定不变的话，增加 Main Cache 区 (protected 区 + probation 区) 的比例会有较好的效果。

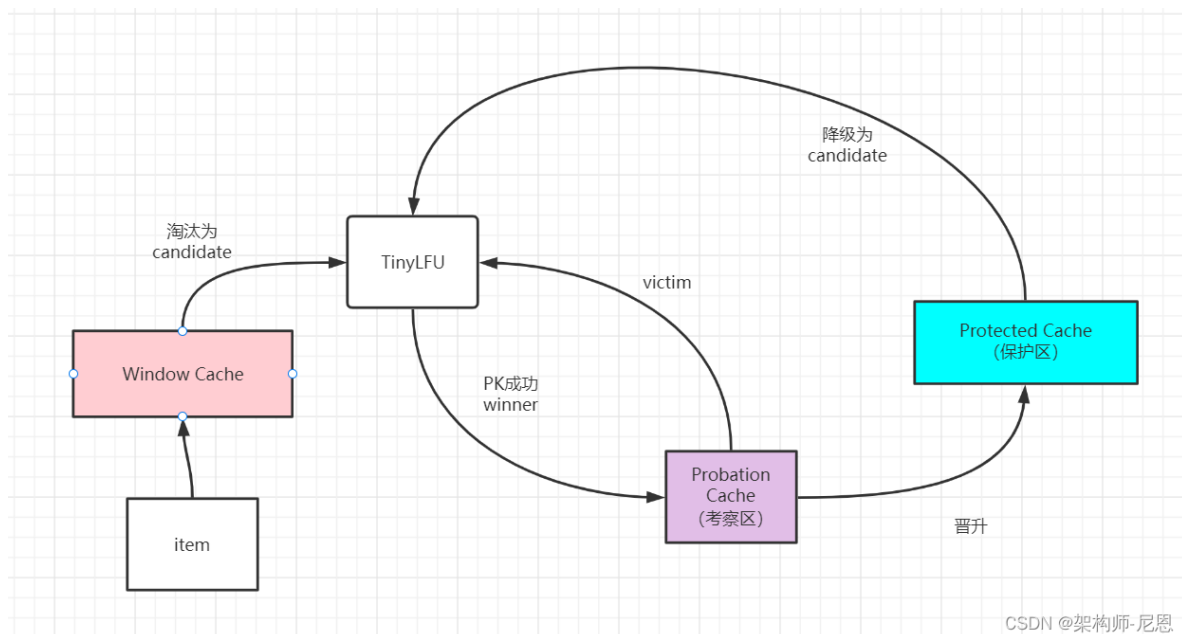
说明：本文会持续更新，最新PDF电子版本，请从下面的连接获取：[语雀](#) 或者 [码云](#)

W-TinyLFU的算法流程

当 window 区满了，就会根据 LRU 把 candidate (即淘汰出来的元素) 放到 Probation区域，

Probation区域则是一个观察区，当有新的缓存项需要进入Probation区时，

如果Probation区空间已满，则会将新进入的缓存项与Probation区中根据LRU规则需要被淘汰 (victim) 的缓存项进行比较，两个进行“PK”，胜者留在 probation，输者就要被淘汰了。



TinyLFU写入机制为：

当有新的缓存项写入缓存时，会先写入Window Cache区域，当Window Cache空间满时，最旧的缓存项会被移出Window Cache。

如果Probation Cache未满，从Window Cache移出的缓存项会直接写入Probation Cache；

如果Probation Cache已满，则会根据TinyLFU算法确定从Window Cache移出的缓存项是丢弃（淘汰）还是写入Probation Cache。

Probation Cache中的缓存项如果访问频率达到一定次数，会提升到Protected Cache；

如果Protected Cache也满了，最旧的缓存项也会移出Protected Cache，然后根据TinyLFU算法确定是丢弃（淘汰）还是写入Probation Cache。

TinyLFU淘汰机制为：

从Window Cache或Protected Cache移出的缓存项称为Candidate，Probation Cache中最旧的缓存项称为Victim。

如果Candidate缓存项的访问频率大于Victim缓存项的访问频率，则淘汰掉Victim。

如果Candidate小于或等于Victim的频率，那么如果Candidate的频率小于5，则淘汰掉Candidate；否则，则在Candidate和Victim两者之中随机地淘汰一个。

从上面对W-TinyLFU的原理描述可知，

caffeine综合了LFU和LRU的优势，将不同特性的缓存项存入不同的缓存区域，最近刚产生的缓存项进入Window区，不会被淘汰；

访问频率高的缓存项进入Protected区，也不会淘汰；

介于这两者之间的缓存项存在Probation区，当缓存空间满了时，Probation区的缓存项会根据访问频率判断是保留还是淘汰；

通过这种机制，很好的平衡了访问频率和访问时间新鲜程度两个维度因素，尽量将新鲜的访问频率高的缓存项保留在缓存中。

同时在维护缓存项访问频率时，引入计数器饱和和衰减机制，即节省了存储资源，也能较好的处理稀疏流量、短时超热点流量等传统LRU和LFU无法很好处理的场景。

W-TinyLFU的优点：

使用Count-Min Sketch算法存储访问频率，极大的节省空间；

TinyLFU会 定期进行新鲜度提升、 访问次数的 衰减操作，应对访问模式变化；

并且使用W-LRU机制能够尽可能避免缓存污染的发生，在过滤器内部会进行筛选处理，避免低频数据置换高频数据。

W-TinyLFU的缺点：

目前已知应用于Caffeine Cache组件里，应用不是很多。

W-TinyLFU与JVM分代内存关联的想通之处

在caffeine所有的数据都在ConcurrentHashMap中，这个和guava cache不同，guava cache是自己实现了个类似ConcurrentHashMap的结构。

与JVM分代内存类似，在caffeine中有三个记录引用的LRU队列：

Eden队列=window：

在caffeine中规定只能为缓存容量的%1，

假如：size=100，那Eden队列的有效大小就等于1。

Eden的作用：记录的是新到的数据，防止突发流量由于之前没有访问频率，而导致被淘汰。

比如有一部新剧上线，在最开始其实是没有访问频率的，

防止上线之后被其他缓存淘汰出去，而加入这个区域。

可以理解为，伊甸园区，保留的刚刚诞生的未年轻人，防止没有长大，直接被大人干死了。

Eden队列满了之后，淘汰的叫做 candidate候选人，进入Probation队列

Probation队列：

可以叫做考察队列，类似于survivor区

在这个队列就代表寿命开始延长，如果Eden队列已经满了，那种最近没有被访问过的元素，数据相对比较冷，第一轮淘汰，进行Probation队列做临时的考察。

如果经常了一段时间的积累，访问的频率还是不高，将被进入到 Protected队列。

Probation队列满了之后，淘汰的叫做 victim 牺牲品，进入Protected 队列

Protected队列：

可以叫做保护队列，类似于老年代

到了这个队列中，意味着已经取，你暂时不会被淘汰，

但是别急，如果Probation队列没有数据，或者Protected数据满了，你也将会被面临淘汰的尴尬局面。

当然想要变成这个队列，需要把Probation访问一次之后，就会提升为Protected队列。

这个有效大小为(size减去eden) X 80% 如果size =100，就会是79。

这三个队列关系如下:

所有的新数据都会进入Eden。

Eden满了，淘汰进入Probation。

如果在Probation中访问了其中某个数据，如果考察区空间不够，则这个数据升级为Protected。

如果Protected满了，又会继续降级为Probation。

对于发生数据淘汰的时候，会从Eden中选择候选人，和Probation中victim进行淘汰pk。

会把Probation队列中的数据队头称为受害者，这个队头肯定是最早进入Probation的，按照LRU队列的算法的话那他其实他就应该被淘汰，但是在这里只能叫他受害者，Probation 是考察队列，代表马上要给他行刑了。

Eden中选择候选人，也是Probation中队尾元素，也叫攻击者。这里受害者会和攻击者皇城PK决出我们应该被淘汰的。

通过我们的Count-Min Sketch中的记录的频率数据有以下几个判断:

如果 candidateFreq 大于victimFreq，那么受害者就直接被淘汰。

如果 candidateFreq 大于 >=6，那么随机淘汰。

如果 candidateFreq 大于 < 6，淘汰 candidate，留下victimKey

Java本地缓存使用实操

基于HashMap实现LRU

通过Map的底层方式，直接将需要缓存的对象放在内存中。

- 优点：简单粗暴，不需要引入第三方包，比较适合一些比较简单的场景。
- 缺点：没有缓存淘汰策略，定制化开发成本高。

```
package com.crazymakercircle.cache;

import com.crazymakercircle.util.Logger;
import org.junit.Test;
```

```

import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.Map;

public class LruDemo {

    @Test
    public void testSimpleLRUCache() {

        SimpleLRUCache cache = new SimpleLRUCache( 2 /* 缓存容量 */ );
        cache.put(1, 1);
        cache.put(2, 2);
        Logger.cfo(cache.get(1));          // 返回 1
        cache.put(3, 3);    // 该操作会使得 2 淘汰
        Logger.cfo(cache.get(2));          // 返回 -1 (未找到)
        cache.put(4, 4);    // 该操作会使得 1 淘汰
        Logger.cfo(cache.get(1));          // 返回 -1 (未找到)
        Logger.cfo(cache.get(3));          // 返回 3
        Logger.cfo(cache.get(4));          // 返回 4
    }

    @Test
    public void testLRUCache() {

        LRUCache cache = new LRUCache( 2 /* 缓存容量 */ );
        cache.put(1, 1);
        cache.put(2, 2);
        Logger.cfo(cache.get(1));          // 返回 1
        cache.put(3, 3);    // 该操作会使得 2 淘汰
        Logger.cfo(cache.get(2));          // 返回 -1 (未找到)
        cache.put(4, 4);    // 该操作会使得 1 淘汰
        Logger.cfo(cache.get(1));          // 返回 -1 (未找到)
        Logger.cfo(cache.get(3));          // 返回 3
        Logger.cfo(cache.get(4));          // 返回 4
    }

    static class SimpleLRUCache extends LinkedHashMap<Integer, Integer> {
        private int capacity;

        public SimpleLRUCache(int capacity) {
            super(capacity, 0.75F, true);
            this.capacity = capacity;
        }

        public int get(int key) {
            return super.getOrDefault(key, -1);
        }

        public void put(int key, int value) {
            super.put(key, value);
        }

        @Override
        protected boolean removeEldestEntry(Map.Entry<Integer, Integer> eldest)
        {

```

```

        return size() > capacity;
    }
}

static private class Entry {
    private int key;
    private int value;
    private Entry before;
    private Entry after;

    public Entry() {
    }

    public Entry(int key, int value) {
        this.key = key;
        this.value = value;
    }
}

static class LRUCache {
    //map容器，空间换时间，保存key对应的CacheNode，保证用O(1)的时间获取到value
    private Map<Integer, Entry> cacheMap = new HashMap<Integer, Entry>();
    // 最大容量
    private int capacity;
    /**
     * 通过双向指针来保证数据的插入更新顺序，以及队尾淘汰机制
     */
    //头指针
    private Entry head;
    //尾指针
    private Entry tail;

    //容器大小
    private int size;

    /**
     * 初始化双向链表，容器大小
     */
    public LRUCache(int capacity) {
        this.capacity = capacity;
        head = new Entry();
        tail = new Entry();
        head.after = tail;
        tail.before = head;
    }

    public int get(int key) {
        Entry node = cacheMap.get(key);
        if (node == null) {
            return -1;
        }
        // node != null,返回node后需要把访问的node移动到双向链表头部
        moveToHead(node);
        return node.value;
    }

    public void put(int key, int value) {

```

```

        Entry node = cacheMap.get(key);
        if (node == null) {
            //缓存不存在就新建一个节点，放入Map以及双向链表的头部
            Entry newNode = new Entry(key, value);
            cacheMap.put(key, newNode);
            addToHead(newNode);
            size++;
            //如果超出缓存容器大小，就移除队尾元素
            if (size > capacity) {
                Entry removeNode = removeTail();
                cacheMap.remove(removeNode.key);
                size--;
            }
        } else {
            //如果已经存在，就把node移动到头部。
            node.value = value;
            moveToHead(node);
        }
    }

    /**
     * 移动节点到头部：
     * 1、删除节点
     * 2、把节点添加到头部
     */
    private void moveToHead(Entry node) {
        removeNode(node);
        addToHead(node);
    }

    /**
     * 移除队尾元素
     */
    private Entry removeTail() {
        Entry node = tail.before;
        removeNode(node);
        return node;
    }

    private void removeNode(Entry node) {

        node.before.after = node.after;
        node.after.before = node.before;
    }

    /**
     * 把节点添加到头部
     */
    private void addToHead(Entry node) {
        head.after.before = node;
        node.after = head.after;
        head.after = node;
        node.before = head;
    }
}

```


Guava Cache使用案例

Guava Cache是由Google开源的基于LRU替换算法的缓存技术。

但Guava Cache由于被下面即将介绍的Caffeine全面超越而被取代，因此不特意编写示例代码了，有兴趣的读者可以访问Guava Cache主页。

- 优点：支持最大容量限制，两种过期删除策略（插入时间和访问时间），支持简单的统计功能。
- 缺点：springboot2和spring5都放弃了对Guava Cache的支持。

Caffeine使用案例

Caffeine采用了W-TinyLFU（LUR和LFU的优点结合）开源的缓存技术。缓存性能接近理论最优，属于是Guava Cache的增强版。

```
package com.github.benmanes.caffeine.demo;

import com.github.benmanes.caffeine.cache.*;
import org.checkerframework.checker.nullness.qual.NonNull;
import org.checkerframework.checker.nullness.qual.Nullable;

import java.util.concurrent.TimeUnit;

public class Demo1 {
    static System.Logger logger = System.getLogger(Demo1.class.getName());

    public static void hello(String[] args) {
        System.out.println("args = " + args);
    }

    public static void main(String... args) throws Exception {
        Cache<String, String> cache = Caffeine.newBuilder()
            //最大个数限制
            //最大容量1024个，超过会自动清理空间
            .maximumSize(1024)
            //初始化容量
            .initialCapacity(1)
            //访问后过期（包括读和写）
            //5秒没有读写自动删除
            .expireAfterAccess(5, TimeUnit.SECONDS)
            //写后过期
            .expireAfterWrite(2, TimeUnit.HOURS)
            //写后自动异步刷新
            .refreshAfterWrite(1, TimeUnit.HOURS)
            //记录下缓存的一些统计数据，例如命中率等
            .recordStats()
    }
```

```

        .removeListener(((key, value, cause) -> {
            //清理通知 key,value ==> 键值对    cause ==> 清理原因
            System.out.println("removed key="+ key);
        })))
//使用CacheLoader创建一个LoadingCache
.build(new CacheLoader<String, String>() {
    //同步加载数据
    @Nullable
    @Override
    public String load(@NonNull String key) throws Exception {
        System.out.println("loading key="+ key);
        return "value_" + key;
    }

    //异步加载数据
    @Nullable
    @Override
    public String reload(@NonNull String key, @NonNull String
oldValue) throws Exception {
        System.out.println("reloading key="+ key);
        return "value_" + key;
    }
});

//添加值
cache.put("name", "疯狂创客圈");
cache.put("key", "一个高并发 研究社群");

//获取值
@Nullable String value = cache.getIfPresent("name");
System.out.println("value = " + value);
//remove
cache.invalidate("name");
value = cache.getIfPresent("name");
System.out.println("value = " + value);
}

}

```

说明：本文会持续更新，最新PDF电子版本，请从下面的连接获取：[语雀](#) 或者 [码云](#)

Encache使用案例

Ehcache是一个纯java的进程内缓存框架，具有快速、精干的特点。是hibernate默认的cacheprovider。

- 优点：支持多种缓存淘汰算法，包括LFU，LRU和FIFO；缓存支持堆内缓存，堆外缓存和磁盘缓存；支持多种集群方案，解决数据共享问题。
- 缺点：性能比Caffeine差

```

package com.crazymakercircle.cache;

import com.crazymakercircle.im.common.bean.User;
import com.crazymakercircle.util.IOUtil;
import net.sf.ehcache.Cache;
import net.sf.ehcache.CacheManager;
import net.sf.ehcache.Element;
import net.sf.ehcache.config.CacheConfiguration;

import java.io.InputStream;

import static com.crazymakercircle.util.IOUtil.getResourcePath;

public class EhcacheDemo {
    public static void main(String[] args) {

        // 1. 创建缓存管理器
        String inputStream= getResourcePath( "ehcache.xml");
        CacheManager cacheManager = CacheManager.create(inputStream);

        // 2. 获取缓存对象
        Cache cache = cacheManager.getCache("HelloWorldCache");
        CacheConfiguration config = cache.getCacheConfiguration();
        config.setTimeToIdleSeconds(60);
        config.setTimeToLiveSeconds(120);

        // 3. 创建元素
        Element element = new Element("key1", "value1");

        // 4. 将元素添加到缓存
        cache.put(element);

        // 5. 获取缓存
        Element value = cache.get("key1");
        System.out.println("value: " + value);
        System.out.println(value.getObjectValue());

        // 6. 删除元素
        cache.remove("key1");

        User user = new User("1000", "Javaer1");
        Element element2 = new Element("user", user);
        cache.put(element2);
        Element value2 = cache.get("user");
        System.out.println("value2: " + value2);
        User user2 = (User) value2.getObjectValue();
        System.out.println(user2);

        System.out.println(cache.getSize());
    }
}

```

```

        // 7. 刷新缓存
        cache.flush();

        // 8. 关闭缓存管理器
        cacheManager.shutdown();

    }
}

public class EncacheTest {    public static void main(String[] args) throws
Exception {                // 声明一个cacheBuilder                CacheManager cacheManager =
CacheManagerBuilder.newCacheManagerBuilder()
.withCache("encacheInstance", CacheConfigurationBuilder
//声明一个容量为20的堆内缓存
.newCacheConfigurationBuilder(String.class,String.class,
ResourcePoolsBuilder.heap(20)))                .build(true);                // 获取Cache
实例                Cache<String,String> myCache =
cacheManager.getCache("encacheInstance", String.class, String.class);                //
写缓存                myCache.put("key","v");                // 读缓存                String value =
myCache.get("key");                // 移除换粗
cacheManager.removeCache("myCache");                cacheManager.close();    }}

```

caffeine 的使用实操

在 pom.xml 中添加 caffeine 依赖:

```

<dependency>
    <groupId>com.github.ben-manes.caffeine</groupId>
    <artifactId>caffeine</artifactId>
    <version>2.5.5</version>
</dependency>

```

创建一个 Caffeine 缓存 (类似一个map) :

```

Cache<String, Object> manualCache = Caffeine.newBuilder()
    .expireAfterWrite(10, TimeUnit.MINUTES)
    .maximumSize(10_000)
    .build();

```

常见用法:

```

public static void main(String... args) throws Exception {
    Cache<String, String> cache = Caffeine.newBuilder()
        //最大个数限制
        //最大容量1024个, 超过会自动清理空间

```

```

        .maximumSize(1024)
        //初始化容量
        .initialCapacity(1)
        //访问后过期（包括读和写）
        //5秒没有读写自动删除
        .expireAfterAccess(5, TimeUnit.SECONDS)
        //写后过期
        .expireAfterWrite(2, TimeUnit.HOURS)
        //写后自动异步刷新
        .refreshAfterWrite(1, TimeUnit.HOURS)
        //记录下缓存的一些统计数据，例如命中率等
        .recordStats()
        .removalListener(((key, value, cause) -> {
            //清理通知 key,value ==> 键值对    cause ==> 清理原因
            System.out.println("removed key="+ key);
        }))
        //使用CacheLoader创建一个LoadingCache
        .build(new CacheLoader<String, String>() {
            //同步加载数据
            @Nullable
            @Override
            public String load(@NonNull String key) throws Exception {
                System.out.println("loading key="+ key);
                return "value_" + key;
            }

            //异步加载数据
            @Nullable
            @Override
            public String reload(@NonNull String key, @NonNull String
oldValue) throws Exception {
                System.out.println("reloading key="+ key);
                return "value_" + key;
            }
        });

        //添加值
        cache.put("name", "疯狂创客圈");
        cache.put("key", "一个高并发 研究社群");

        //获取值
        @Nullable String value = cache.getIfPresent("name");
        System.out.println("value = " + value);
        //remove
        cache.invalidate("name");
        value = cache.getIfPresent("name");
        System.out.println("value = " + value);
    }
}

```

参数方法：

- initialCapacity(1) 初始缓存长度为1；
- maximumSize(100) 最大长度为100；
- expireAfterWrite(1, TimeUnit.DAYS) 设置缓存策略在1天未写入过期缓存。

过期策略

在Caffeine中分为两种缓存，一个是有界缓存，一个是无界缓存，无界缓存不需要过期并且没有界限。

在有界缓存中提供了三个过期API：

- `expireAfterWrite`：代表着写了之后多久过期。（上面列子就是这种方式）
- `expireAfterAccess`：代表着最后一次访问了之后多久过期。
- `expireAfter`：在`expireAfter`中需要自己实现`Expiry`接口，这个接口支持`create`、`update`、以及`access`了之后多久过期。

注意这个API和前面两个API是互斥的。这里和前面两个API不同的是，需要你告诉缓存框架，它应该在具体的某个时间过期，也就是通过前面的重写`create`、`update`、以及`access`的方法，获取具体的过期时间。

填充 (Population) 特性

填充特性是指如何在key不存在的情况下，如何创建一个对象进行返回，主要分为下面四种

1 手动(Manual)

```
public static void main(String... args) throws Exception {
    Cache<String, Integer> cache = Caffeine.newBuilder().build();

    Integer age1 = cache.getIfPresent("张三");
    System.out.println(age1);

    //当key不存在时，会立即创建出对象来返回，age2不会为空
    Integer age2 = cache.get("张三", k -> {
        System.out.println("k:" + k);
        return 18;
    });
    System.out.println(age2);
}
null
k: 张三
18
```

`Cache`接口允许显式的去控制缓存的检索，更新和删除。

我们可以通过`cache.getIfPresent(key)`方法来获取一个key的值，通过`cache.put(key, value)`方法显示的将数据放入缓存，但是这样子会覆盖缓存原来key的数据。更加建议使用`cache.get(key, k -> value)`的方式，`get`方法将一个参数为key的Function(`createExpensiveGraph`)作为参数传入。如果缓存中不存在该键，则调用这个Function函数，并将返回值作为该缓存的值插入缓存中。`get`方法是以阻塞方式执行调用，即使多个线程同时请求该值也只会调用一次Function方法。这样可以避免与其他线程的写入竞争，这也是为什么使用`get`优于`getIfPresent`的原因。

注意：如果调用该方法返回NULL（如上面的`createExpensiveGraph`方法），则`cache.get`返回null，如果调用该方法抛出异常，则`get`方法也会抛出异常。

可以使用`Cache.asMap()`方法获取`ConcurrentMap`进而对缓存进行一些更改。

2 自动>Loading)

```
public static void main(String... args) throws Exception {

    //此时的类型是 LoadingCache 不是 Cache
    LoadingCache<String, Integer> cache = Caffeine.newBuilder().build(key -> {
        System.out.println("自动填充:" + key);
        return 18;
    });

    Integer age1 = cache.getIfPresent("张三");
    System.out.println(age1);

    // key 不存在时 会根据给定的CacheLoader自动装载进去
    Integer age2 = cache.get("张三");
    System.out.println(age2);
}
null
自动填充:张三
18
```

3 异步手动(Asynchronous Manual)

```
public static void main(String... args) throws Exception {
    AsyncCache<String, Integer> cache = Caffeine.newBuilder().buildAsync();

    //会返回一个 future对象， 调用future对象的get方法会一直卡住直到得到返回，和多线程的
    submit一样
    CompletableFuture<Integer> ageFuture = cache.get("张三", name -> {
        System.out.println("name:" + name);
        return 18;
    });

    Integer age = ageFuture.get();
    System.out.println("age:" + age);
}
name:张三
age:18
```

4 异步自动(Asynchronously Loading)

```

public static void main(String... args) throws Exception {
    //和1.4基本差不多
    AsyncLoadingCache<String, Integer> cache =
    Caffeine.newBuilder().buildAsync(name -> {
        System.out.println("name:" + name);
        return 18;
    });
    CompletableFuture<Integer> ageFuture = cache.get("张三");

    Integer age = ageFuture.get();
    System.out.println("age:" + age);
}

```

refresh刷新策略

何为更新策略？就是在设定多长时间后会自动刷新缓存。

Caffeine提供了refreshAfterWrite()方法，来让我们进行写后多久更新策略：

```

LoadingCache<String, String> build =
CacheBuilder.newBuilder().refreshAfterWrite(1, TimeUnit.DAYS)
    .build(new CacheLoader<String, String>() {
        @Override
        public String load(String key) {
            return "";
        }
    });
}

```

上面的代码我们需要建立一个CacheLodaer来进行刷新，这里是同步进行的，可以通过buildAsync方法进行异步构建。

在实际业务中这里可以把我们代码中的mapper传入进去，进行数据源的刷新。

但是实际使用中，你设置了一天刷新，但是一天后你发现缓存并没有刷新。

这是因为只有在1天后这个缓存再次访问后才能刷新，如果没人访问，那么永远也不会刷新。

我们来看看自动刷新是怎么做的呢？

自动刷新只存在读操作之后，也就是我们的afterRead()这个方法，其中有个方法叫refreshIfNeeeded，它会根据你是同步还是异步然后进行刷新处理。

说明：本文会持续更新，最新PDF电子版本，请从下面的连接获取：[语雀](#) 或者 [码云](#)

驱逐策略 (eviction)

Caffeine提供三类驱逐策略：基于大小（size-based），基于时间（time-based）和基于引用（reference-based）。

基于大小（size-based）

基于大小驱逐，有两种方式：一种是基于缓存大小，一种是基于权重。


```
// Evict based on the number of entries in the cache
// 根据缓存的计数进行驱逐
LoadingCache<Key, Graph> graphs = Caffeine.newBuilder()
    .maximumSize(10_000)
    .build(key -> createExpensiveGraph(key));

// Evict based on the number of vertices in the cache
// 根据缓存的权重来进行驱逐（权重只是用于确定缓存大小，不会用于决定该缓存是否被驱逐）
LoadingCache<Key, Graph> graphs = Caffeine.newBuilder()
    .maximumWeight(10_000)
    .weigher((Key key, Graph graph) -> graph.vertices().size())
    .build(key -> createExpensiveGraph(key));
```

我们可以使用Caffeine.maximumSize(long)方法来指定缓存的最大容量。当缓存超出这个容量的时候，会使用[Window TinyLfu策略](#)来删除缓存。

我们也可以使用权重的策略来进行驱逐，可以使用Caffeine.weigher(Weigher) 函数来指定权重，使用Caffeine.maximumWeight(long) 函数来指定缓存最大权重值。

maximumWeight与maximumSize不可以同时使用。

基于时间 (Time-based)

```
// Evict based on a fixed expiration policy
// 基于固定的到期策略进行退出
LoadingCache<Key, Graph> graphs = Caffeine.newBuilder()
    .expireAfterAccess(5, TimeUnit.MINUTES)
    .build(key -> createExpensiveGraph(key));
LoadingCache<Key, Graph> graphs = Caffeine.newBuilder()
    .expireAfterWrite(10, TimeUnit.MINUTES)
    .build(key -> createExpensiveGraph(key));

// Evict based on a varying expiration policy
// 基于不同的到期策略进行退出
LoadingCache<Key, Graph> graphs = Caffeine.newBuilder()
    .expireAfter(new Expiry<Key, Graph>() {
        @Override
        public long expireAfterCreate(Key key, Graph graph, long currentTime) {
            // Use wall clock time, rather than nanotime, if from an external
resource
            long seconds = graph.creationDate().plusHours(5)
                .minus(System.currentTimeMillis(), MILLIS)
                .toEpochSecond();
            return TimeUnit.SECONDS.toNanos(seconds);
        }

        @Override
        public long expireAfterUpdate(Key key, Graph graph,
            long currentTime, long currentDuration) {
            return currentDuration;
        }

        @Override
```

```

        public long expireAfterRead(Key key, Graph graph,
                                   long currentTime, long currentDuration) {
            return currentDuration;
        }
    })
    .build(key -> createExpensiveGraph(key));

```

Caffeine提供了三种定时驱逐策略：

- `expireAfterAccess(long, TimeUnit)`: 在最后一次访问或者写入后开始计时，在指定的时间后过期。假如一直有请求访问该key，那么这个缓存将一直不会过期。
- `expireAfterWrite(long, TimeUnit)`: 在最后一次写入缓存后开始计时，在指定的时间后过期。
- `expireAfter(Expiry)`: 自定义策略，过期时间由Expiry实现独自计算。

缓存的删除策略使用的是惰性删除和定时删除。这两个删除策略的时间复杂度都是O(1)。

测试定时驱逐不需要等到时间结束。我们可以使用Ticker接口和Caffeine.ticker(Ticker)方法在缓存生成器中指定时间源，而不必等待系统时钟。如：

```

FakeTicker ticker = new FakeTicker(); // Guava's testlib
Cache<Key, Graph> cache = Caffeine.newBuilder()
    .expireAfterWrite(10, TimeUnit.MINUTES)
    .executor(Runnable::run)
    .ticker(ticker::read)
    .maximumSize(10)
    .build();

cache.put(key, graph);
ticker.advance(30, TimeUnit.MINUTES)
assertThat(cache.getIfPresent(key), is(nullValue()));

```

基于引用 (reference-based)

Java4种引用的级别由高到低依次为：强引用 > 软引用 > 弱引用 > 虚引用

引用类型	被垃圾回收时间	用途	生存时间
强引用	从来不会	对象的一般状态	JVM停止运行时终止
软引用	在内存不足时	对象缓存	内存不足时终止
弱引用	在垃圾回收时	对象缓存	gc运行后终止
虚引用	在垃圾回收时	堆外内存	虚引用的通知特性来管理的堆外内存

1、强引用

以前我们使用的大部分引用实际上都是强引用，这是使用最普遍的引用。如果一个对象具有强引用，那就类似于必不可少的生活用品，垃圾回收器绝不会回收它。当内存空间不足，Java虚拟机宁愿抛出OutOfMemoryError错误，使程序异常终止，也不会靠随意回收具有强引用的对象来解决内存不足问题。

如

```
String str = "abc";
List<String> list = new ArrayList<String>();
list.add(str)
在list集合里的数据不会释放，即使内存不足也不会
```

在ArrayList类中定义了一个私有的变量elementData数组，在调用方法清空数组时可以看到为每个数组内容赋值为null。不同于elementData=null，强引用仍然存在，避免在后续调用 add()等方法添加元素时进行重新的内存分配。使用如clear()方法中释放内存的方法对数组中存放的引用类型特别适用，这样就可以及时释放内存。

2、软引用 (SoftReference)

特色：

- 内存溢出之前进行回收，GC时内存不足时回收，如果内存足够就不回收
- 使用场景：在内存足够的情况下进行缓存，提升速度，内存不足时JVM自动回收

如果一个对象只具有软引用，那就类似于可有可物的生活用品。如果内存空间足够，垃圾回收器就不会回收它，如果内存空间不足了，就会回收这些对象的内存。只要垃圾回收器没有回收它，该对象就可以被程序使用。软引用可用来实现内存敏感的高速缓存。

软引用可以和一个引用队列（ReferenceQueue）联合使用，如果软引用所引用的对象被垃圾回收，JAVA虚拟机就会把这个软引用加入到与之关联的引用队列中。

如：

```
public class Test {

    public static void main(String[] args){
        System.out.println("开始");
        A a = new A();
        SoftReference<A> sr = new SoftReference<A>(a);
        a = null;
        if(sr!=null){
            a = sr.get();
        }
        else{
            a = new A();
            sr = new SoftReference<A>(a);
        }
        System.out.println("结束");
    }

}

class A{
    int[] a ;
    public A(){
        a = new int[100000000];
    }
}
```

当内存足够大时可以把数组存入软引用，取数据时就可从内存里取数据，提高运行效率

3. 弱引用 (WeakReference)

特色：

- 每次GC时回收，无论内存是否足够
- 使用场景：a. ThreadLocalMap防止内存泄漏 b. 监控对象是否将要被回收

如果一个对象只具有弱引用，那就类似于可有可物的生活用品。

弱引用与软引用的区别在于：只具有弱引用的对象拥有更短暂的生命周期。在垃圾回收器线程扫描它所管辖的内存区域的过程中，一旦发现了只具有弱引用的对象，不管当前内存空间足够与否，都会回收它的内存。不过，由于垃圾回收器是一个优先级很低的线程，因此不一定会很快发现那些只具有弱引用的对象。

弱引用可以和一个引用队列（ReferenceQueue）联合使用，如果弱引用所引用的对象被垃圾回收，Java虚拟机就会把这个弱引用加入到与之关联的引用队列中。

如：

```
Object c = new Car(); //只要c还指向car object, car object就不会被回收
WeakReference<Car> weakCar = new WeakReference<Car>(car);
```

当要获得weak reference引用的object时，首先需要判断它是否已经被回收：

```
weakCar.get();
```

如果此方法为空，那么说明weakCar指向的对象已经被回收了。

如果这个对象是偶尔的使用，并且希望在使用时随时就能获取到，但又不想影响此对象的垃圾收集，那么你应该用 Weak Reference 来记住此对象。

当你想引用一个对象，但是这个对象有自己的生命周期，你不想介入这个对象的生命周期，这时候你就是用弱引用。

这个引用不会在对象的垃圾回收判断中产生任何附加的影响。

4. 虚引用 (PhantomReference)

“虚引用”顾名思义，就是形同虚设，与其他几种引用都不同，虚引用并不会决定对象的生命周期。

如果一个对象仅持有虚引用，那么它就和没有任何引用一样，在任何时候都可能被垃圾回收。

虚引用主要用来跟踪对象被垃圾回收的活动。

虚引用与软引用和弱引用的一个区别在于：**虚引用必须和引用队列（ReferenceQueue）联合使用。**

当垃圾回收器准备回收一个对象时，如果发现它还有虚引用，就会在回收对象的内存之前，把这个虚引用加入到与之关联的引用队列中。

程序可以通过判断引用队列中是否已经加入了虚引用，来了解被引用的对象是否将要被垃圾回收。程序如果发现某个虚引用已经被加入到引用队列，那么就可以在所引用的对象的内存被回收之前采取必要的行动。

特别注意，在实际程序设计中一般很少使用弱引用与虚引用，使用软引用的情况较多，这是因为软引用可以加速JVM对垃圾内存的回收速度，可以维护系统的运行安全，防止内存溢出（OutOfMemory）等问题的产生。

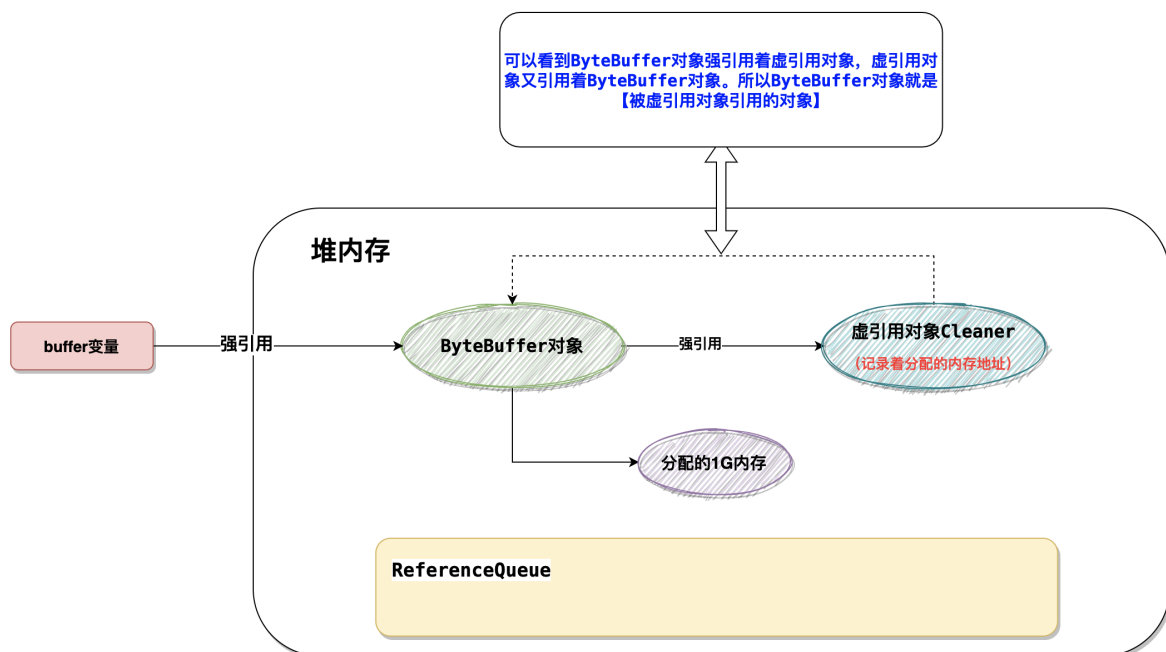
byteBuffer回收对外内存的流程

两种使用堆外内存的方法，一种是依靠unsafe对象，另一种是NIO中的ByteBuffer，直接使用unsafe对象来操作内存，对于一般开发者来说难度很大，并且如果内存管理不当，容易造成内存泄漏。所以不推荐。

推荐使用的是ByteBuffer来操作堆外内存。

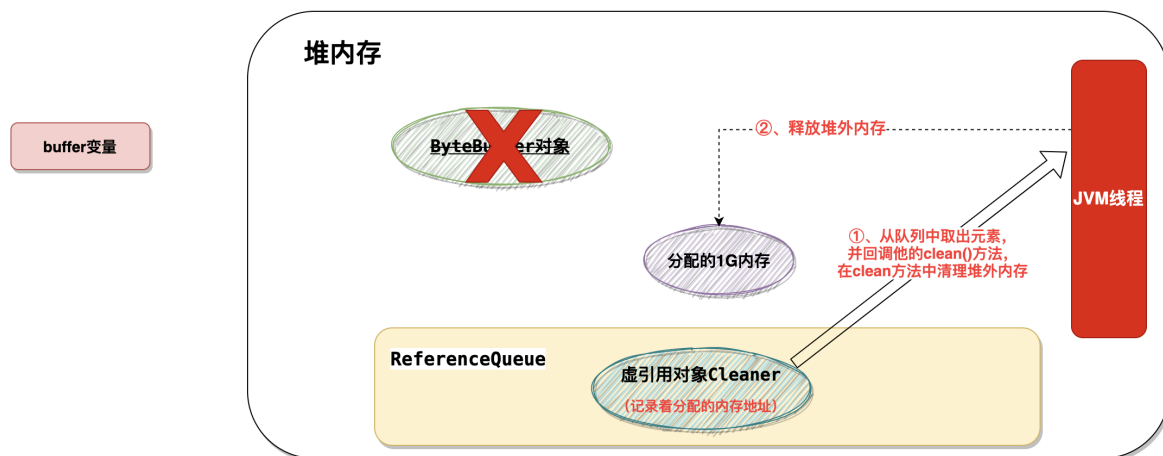
在上面的ByteBuffer如何 触发堆外内存的回收呢？是通过 虚引用的 关联线程是实现的。

- 当byteBuffer被回收后，在进行GC垃圾回收的时候，发现 虚引用对象Cleaner 是 PhantomReference 类型的对象，并且 被该对象引用的对象（ByteBuffer对象） 已经被回收了
- 那么他就将这个对象放入到（ReferenceQueue）队列中
- JVM中会有一个优先级很低的线程会去将该队列中的 虚引用对象 取出来，然后回调 clean（） 方法
- 在 clean（） 方法里做的工作其实就是根据 内存地址 去释放这块内存（内部还是通过unsafe对象去释放的内存）。



可以看到被虚引用引用的对象其实就是这个byteBuffer对象。

所以说需要重点关注的是这个byteBuffer对象被回收了以后会触发什么操作。



说明：本文会持续更新，最新PDF电子版本，请从下面的连接获取：[语雀](#) 或者 [码云](#)

缓存的驱逐配置成基于垃圾回收器

```
// Evict when neither the key nor value are strongly reachable
// 当key和value都没有引用时驱逐缓存
LoadingCache<Key, Graph> graphs = Caffeine.newBuilder()
    .weakKeys()
    .weakValues()
    .build(key -> createExpensiveGraph(key));

// Evict when the garbage collector needs to free memory
// 当垃圾收集器需要释放内存时驱逐
LoadingCache<Key, Graph> graphs = Caffeine.newBuilder()
    .softValues()
    .build(key -> createExpensiveGraph(key));
```

我们可以将缓存的驱逐配置成基于垃圾回收器。

为此，我们可以将key 和 value 配置为弱引用或只将值配置成软引用。

注意：AsyncLoadingCache不支持弱引用和软引用。

Caffeine.weakKeys() 使用弱引用存储key。如果没有其他地方对该key有强引用，那么该缓存就会被垃圾回收器回收。由于垃圾回收器只依赖于身份(identity)相等，因此这会导致整个缓存使用身份(==)相等来比较 key，而不是使用 equals()。

Caffeine.weakValues() 使用弱引用存储value。如果没有其他地方对该value有强引用，那么该缓存就会被垃圾回收器回收。由于垃圾回收器只依赖于身份(identity)相等，因此这会导致整个缓存使用身份(==)相等来比较 key，而不是使用 equals()。

Caffeine.softValues() 使用软引用存储value。当内存满了过后，软引用的对象以将使用最近最少使用(least-recently-used) 的方式进行垃圾回收。由于使用软引用是需要等到内存满了才进行回收，所以我们通常建议给缓存配置一个使用内存的最大值。softValues() 将使用身份相等(identity) (==) 而不是 equals() 来比较值。

注意： Caffeine.weakValues()和Caffeine.softValues()不可以一起使用。

Removal移除特性

概念：

- 驱逐 (eviction)：由于满足了某种驱逐策略，后台自动进行的删除操作
- 无效 (invalidation)：表示由调用方手动删除缓存
- 移除 (removal)：监听驱逐或无效操作的监听器

手动删除缓存：

在任何时候，您都可能明确地使缓存无效，而不用等待缓存被驱逐。

```
// individual key
cache.invalidate(key)
// bulk keys
cache.invalidateAll(keys)
// all keys
cache.invalidateAll()
```

Removal 监听器：

```
Cache<Key, Graph> graphs = Caffeine.newBuilder()
    .removalListener((Key key, Graph graph, RemovalCause cause) ->
        System.out.printf("Key %s was removed (%s)%n", key, cause))
    .build();
```

您可以通过Caffeine.removalListener(RemovalListener) 为缓存指定一个删除侦听器，以便在删除数据时执行某些操作。 RemovalListener可以获取到key、value和RemovalCause（删除的原因）。

删除侦听器的里面的操作是使用Executor来异步执行的。默认执行程序是ForkJoinPool.commonPool(), 可以通过Caffeine.executor(Executor)覆盖。当操作必须与删除同步执行时，请改为使用CacheWrite，CacheWrite将在下面说明。

注意： 由RemovalListener抛出的任何异常都会被记录（使用Logger）并不会抛出。

刷新 (Refresh)

```
LoadingCache<Key, Graph> graphs = Caffeine.newBuilder()
    .maximumSize(10_000)
    // 指定在创建缓存或者最近一次更新缓存后经过固定的时间间隔，刷新缓存
    .refreshAfterWrite(1, TimeUnit.MINUTES)
    .build(key -> createExpensiveGraph(key));
```


刷新和驱逐是不一样的。刷新的是通过LoadingCache.refresh(key)方法来指定，并通过调用CacheLoader.reload方法来执行，刷新key会异步地为这个key加载新的value，并返回旧的值（如果有的话）。驱逐会阻塞查询操作直到驱逐作完成才会进行其他操作。

与expireAfterWrite不同的是，refreshAfterWrite将在查询数据的时候判断该数据是不是符合查询条件，如果符合条件该缓存就会去执行刷新操作。例如，您可以在同一个缓存中同时指定refreshAfterWrite和expireAfterWrite，只有当数据具备刷新条件的时候才会去刷新数据，不会盲目去执行刷新操作。如果数据在刷新后就一直没有被再次查询，那么该数据也会过期。

刷新操作是使用Executor异步执行的。默认执行程序是ForkJoinPool.commonPool()，可以通过Caffeine.executor(Executor)覆盖。

如果刷新时引发异常，则使用log记录日志，并不会抛出。

Writer直接写 (write-through)

```
LoadingCache<Key, Graph> graphs = Caffeine.newBuilder()
    .writer(new CacheWriter<Key, Graph>() {
        @Override public void write(Key key, Graph graph) {
            // write to storage or secondary cache
        }
        @Override public void delete(Key key, Graph graph, RemovalCause cause) {
            // delete from storage or secondary cache
        }
    })
    .build(key -> createExpensiveGraph(key));
```

CacheWriter允许缓存充当一个底层资源的代理，当与CacheLoader结合使用时，所有对缓存的读写操作都可以通过Writer进行传递。Writer可以把操作缓存和操作外部资源扩展成一个同步的原子性操作。并且在缓存写入完成之前，它将会阻塞后续的更新缓存操作，但是读取（get）将直接返回原有的值。如果写入程序失败，那么原有的key和value的映射将保持不变，如果出现异常将直接抛给调用者。

CacheWriter可以同步的监听到缓存的创建、变更和删除操作。加载（例如，LoadingCache.get）、重新加载（例如，LoadingCache.refresh）和计算（例如Map.computeIfPresent）的操作不被CacheWriter监听到。

注意：CacheWriter不能与weakKeys或AsyncLoadingCache结合使用。

写模式 (Write Modes)

CacheWriter可以用来实现一个直接写（write-through）或回写（write-back）缓存的操作。

write-through式缓存中，写操作是一个同步的过程，只有写成功了才会去更新缓存。这避免了同时去更新资源和缓存的条件竞争。

write-back式缓存中，对外部资源的操作是在缓存更新后异步执行的。这样可以提高写入的吞吐量，避免数据不一致的风险，比如如果写入失败，则在缓存中保留无效的状态。这种方法可能有助于延迟写操作，直到指定的时间，限制写速率或批写操作。

通过对write-back进行扩展，我们可以实现以下特性：

- 批处理和合并操作

- 延迟操作并到一个特定的时间执行
- 如果超过阈值大小，则在定期刷新之前执行批处理
- 如果操作尚未刷新，则从写入后缓冲器（write-behind）加载
- 根据外部资源的特点，处理重审，速率限制和并发

可以参考一个简单的[例子](#)，使用RxJava实现。

分层（Layering）

CacheWriter可能用来集成多个缓存进而实现多级缓存。

多级缓存的加载和写入可以使用系统外部高速缓存。这允许缓存使用一个小并且快速的缓存去调用一个大的并且速度相对慢一点的缓存。典型的off-heap、file-based和remote 缓存。

受害者缓存（Victim Cache）是一个多级缓存的变体，其中被删除的数据被写入二级缓存。

这个delete(K, V, RemovalCause) 方法允许检查为什么该数据被删除，并作出相应的操作。

同步监听器（Synchronous Listeners）

同步监听器会接收一个key在缓存中的进行了那些操作的通知。

监听器可以阻止缓存操作，也可以将事件排队以异步的方式执行。

这种类型的监听器最常用于复制或构建分布式缓存。

统计（Statistics） 特性

```
Cache<Key, Graph> graphs = Caffeine.newBuilder()
    .maximumSize(10_000)
    .recordStats()
    .build();
```

使用Caffeine.recordStats()，您可以打开统计信息收集。Cache.stats() 方法返回提供统计信息的CacheStats，如：

- hitRate(): 返回命中与请求的比率
- hitCount(): 返回命中缓存的总数
- evictionCount(): 缓存逐出的数量
- averageLoadPenalty(): 加载新值所花费的平均时间

Cleanup 清理特性

缓存的删除策略使用的是惰性删除和定时删除，但是我也可以自己调用cache.cleanUp()方法手动触发一次回收操作。

cache.cleanUp()是一个同步方法。

策略（Policy） 特性

在创建缓存的时候，缓存的策略就指定好了。

但是我们可以在运行时可以获得和修改该策略。这些策略可以通过一些选项来获得，以此来确定缓存是否支持该功能。

Size-based

```
cache.policy().eviction().ifPresent(eviction -> {  
    eviction.setMaximum(2 * eviction.getMaximum());  
});
```

如果缓存配置时基于权重来驱逐，那么我们可以使用`weightedSize()`来获取当前权重。这与获取缓存中的记录数的`Cache.estimatedSize()`方法有所不同。

缓存的最大值(maximum)或最大权重(weight)可以通过`getMaximum()`方法来读取，并使用`setMaximum(long)`进行调整。当缓存量达到新的阈值的时候缓存才会去驱逐缓存。

如果有需用我们可以通过`hottest(int)`和`coldest(int)`方法来获取最有可能命中的数据和最有可能驱逐的数据快照。

Time-based

```
cache.policy().expireAfterAccess().ifPresent(expiration -> ...);  
cache.policy().expireAfterWrite().ifPresent(expiration -> ...);  
cache.policy().expireVariably().ifPresent(expiration -> ...);  
cache.policy().refreshAfterWrite().ifPresent(expiration -> ...);
```

`ageOf(key, TimeUnit)`提供了从`expireAfterAccess`、`expireAfterWrite`或`refreshAfterWrite`策略的角度来看条目已经空闲的时间。

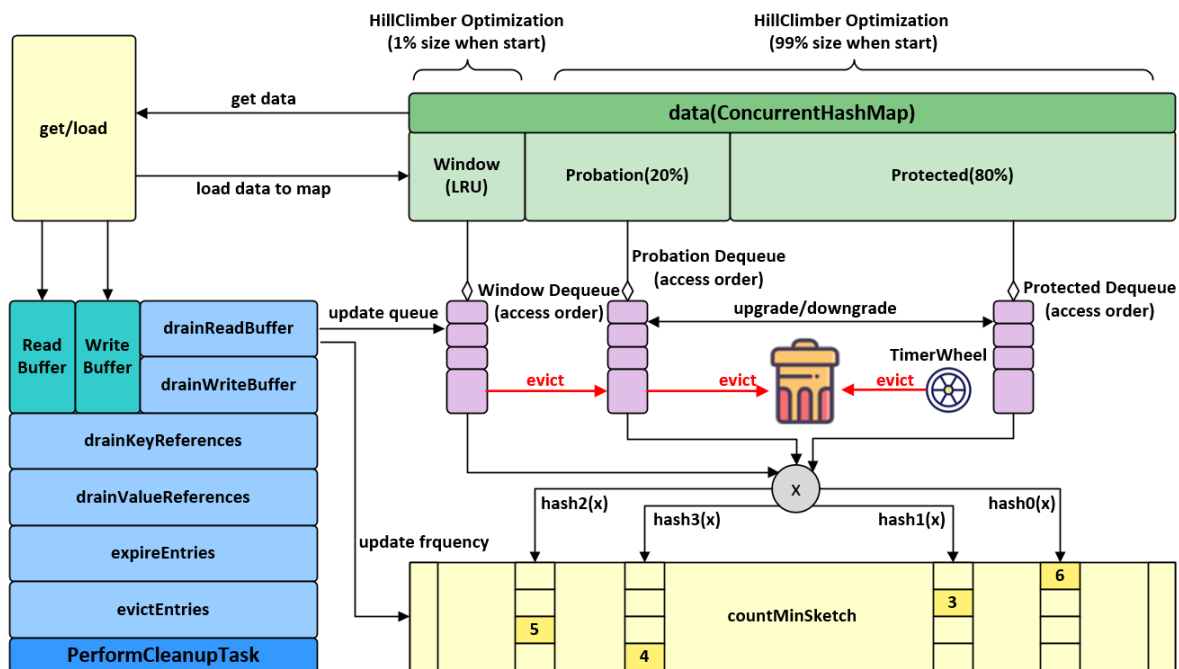
最大持续时间可以从`getExpiresAfter(TimeUnit)`读取，并使用`setExpiresAfter(long, TimeUnit)`进行调整。

如果有需用我们可以通过`hottest(int)`和`coldest(int)`方法来获取最有可能命中的数据和最有可能驱逐的数据快照。

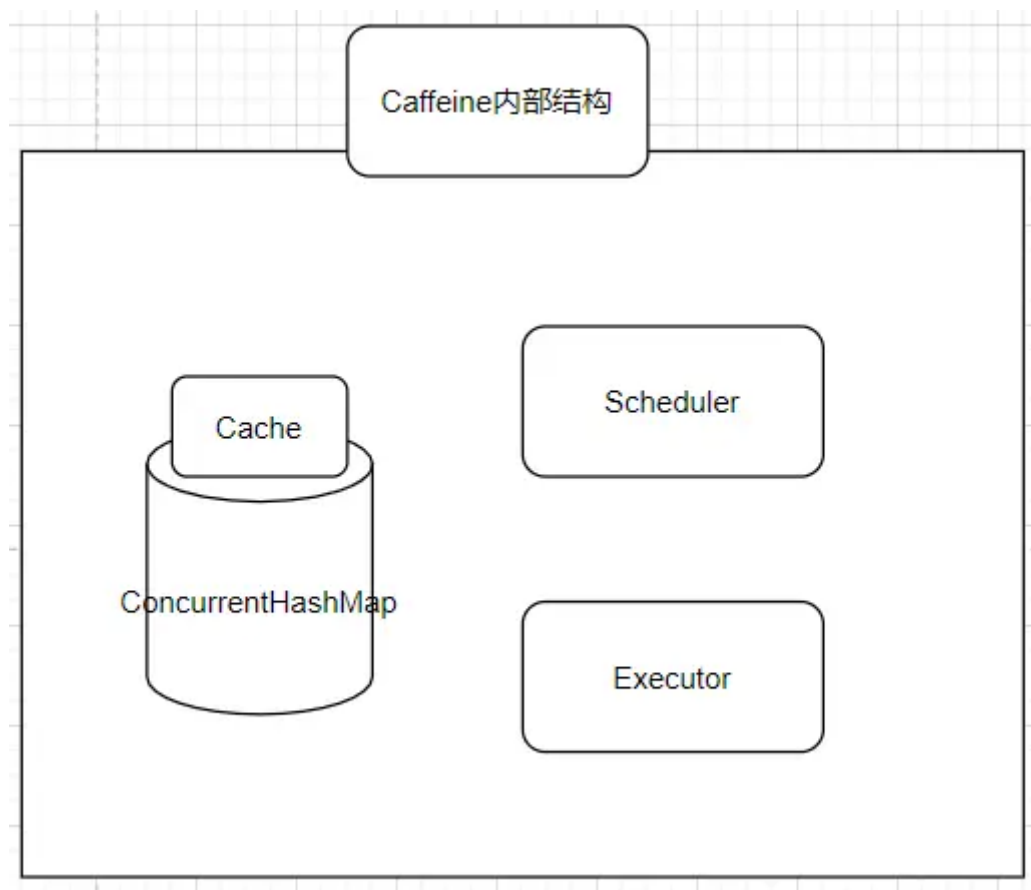
说明：本文会持续更新，最新PDF电子版本，请从下面的连接获取：[语雀](#) 或者 [码云](#)

Caffeine 架构分析

caffeine底层架构图



caffeine的宏观结构图



说明：本文会持续更新，最新PDF电子版本，请从下面的连接获取：[语雀](#) 或者 [码云](#)

Cache的内部数据容器

包含着一个ConcurrentHashMap,

这也是存放我们所有缓存数据的地方, 众所周知, ConcurrentHashMap是一个并发安全的容器, 这点很重要, 可以说Caffeine其实就是一个被强化过的ConcurrentHashMap;

Scheduler (定时器)

定期清空数据的一个机制, 可以不设置, 如果不设置则不会主动的清空过期数据;

Executor 异步线程池

指定运行异步任务时要使用的线程池。

可以不设置, 如果不设置则会使用默认的线程池, 也就是ForkJoinPool.commonPool();

缓存操作执行流程

1. 通过put操作将数据放入data属性中 (ConcurrentHashMap)
2. 创建AddTask任务, 放入(offer)写缓存: writeBuffer
3. 从writeBuffer中获取任务, 并执行其run方法, 追加记录频率: frequencySketch().increment(key)
4. 往window区写入数据
5. 如果数据超过window区大小, 将数据移到probation区
6. 比较从window区晋升的数据和probation区的老数据的频率, 输者被淘汰, 从data中删除

访问顺序队列

双向链表对哈希表中的所有条目进行排序。

通过在哈希, 我们可以在 $O(1)$ 时间内找到条目并操作其相邻元素。

这里的访问包括条目的创建、更新和读取 (CUR) 。

最近使用最少的条目在头部, 最多的在末尾。

这为基于大小的淘汰 (maximumSize) 和 time-to-idle 淘汰 (expireAfterAccess) 提供了支持。

问题的关键在于, 每次访问都需要对该列表进行更改, 这很难实现的并行且高效。

写入顺序队列

写入顺序指的是条目的创建或更新 (CU) 。

与访问顺序队列类似, 写顺序队列操作的时间复杂度是 $O(1)$ 的。

这个队列用于 time-to-live 过期 (expireAfterWrite) 。

读缓冲区

典型的缓存锁定每个操作，以安全地对 **访问队列中的条目进行重新排序**。

一种替代方法是将每个重新排序操作存储在缓冲区中，然后分批应用更改。

可以将其视为页面替换策略的预写日志。

当缓冲区满的时候，将会立即尝试获取锁并挂起，直到缓冲区内的操作被处理完毕后将会立即返回。

读缓冲区被实现为条带状环形缓冲区。

条带用于减少竞争，线程通过特定的哈希选择条带。

环形缓冲区是一个固定大小的数组，使其高效并最大程度地减少了 GC 开销。条带数可以根据竞争检测算法动态增长。

写缓冲区

与读取缓冲区类似，此缓冲区用于**重放写入事件**。

读缓冲区是允许丢记录的，因为这只用于用于 **优化驱逐策略的命中率**，

但是写入不允许丢记录的，因此必须用 有效的有界队列实现。

由于每次填充（populate）时都要优先清空写缓冲区，因此它通常保持为空或很小。

缓冲区被实现为可扩展的循环数组，该数组可调整为最大大小。

调整大小时，将分配并产生一个新数组。先前的数组包括供消费者遍历的转发链接，然后允许将旧的数组释放。

通过使用这种分块机制，缓冲区的初始大小较小，读取和写入的成本较低，并且产生的垃圾最少。

当缓冲区已满且无法增长时，生产者会不停自旋重试并触发维护操作，然后 yield 一小段时间。

这样可以使消费者线程根据线程优先级来清空缓存区重放写操作。

锁定摊销

传统的缓存会锁定每个操作，而 Caffeine 会批量处理工作、并将成本分散到许多线程中。

这将对锁定的代价进行摊销，而不会创建锁竞争。

维护的开销一般会委托给配置的执行器，不过如果任务被拒绝或使用了调用者运行策略，还是会由用户线程执行。

批处理的一个优点是，由于锁的排他性，缓冲区仅在给定的时间被单个线程清空。

这允许使用更有效的基于多生产者-单消费者的缓冲区实现。

通过利用 CPU 缓存效率的优势，它也可以更好地与硬件特性保持一致。

Entry 状态切换

当缓存不受排他锁保护时，会存在操作可能以错误的顺序记录和重放的问题。

由于竞争的原因，创建-读取-更新-删除序列可能无法以相同顺序存储在缓冲区中。

这样做将需要粗粒度的锁，会导致降低性能。

与并发数据结构中的典型情况一样，Caffeine 使用原子状态转换解决这个问题。

Entry 有三种状态——活跃（alive）、已退休（retired）、已失效（dead）。

活跃状态意味着它在哈希表和访问/写入队列中都存在。

从哈希表中删除 entry 时，该 entry 被标记为已退休，需要从队列中删除。

删除完成后，该 entry 将被标记已失效并且可以进行垃圾回收。

驱逐策略

Caffeine 使用 Window TinyLfu 策略来提供几乎最佳的命中率。

访问队列分为两个空间：主空间和准入窗口。

如果 TinyLfu 策略接受，则退出准入窗口并进入主空间。

TinyLfu 会比较窗口中的受害者和主空间的受害者之间的访问频率，选择保留两者之间之前被访问频率更高的 entry。

频率在 CountMinSketch 中通过 4 bit 存储，每个 entry 要求 8 bytes 以保证准确。

此配置使缓存能够以 $O(1)$ 时间根据频率和新近度进行淘汰，同时占用空间也很小。

适应性

准入窗口和主空间的大小是根据工作负载特征动态确定的。

如果偏向新近度，则倾向于使用大窗口，而偏向频率倾向使用较小的窗口。

Caffeine 使用 hill climbing 算法来采样命中率，进行调整并将其配置为最佳平衡。

快速处理

当缓存低于其最大容量的 50% 时，驱逐策略并不完全启用。

暂不初始化记录频率的 sketch 可以减少内存占用，因为缓存可能会被配置了一个极其高的阈值。

除非其他特性要求，否则不记录访问，以避免读取缓冲区操作和重放访问并清空操作的竞争。

Hash DoS 保护

DoS是Denial of Service的简称，即拒绝服务，造成DoS的攻击行为被称为DoS攻击，其目的是使计算机或网络无法提供正常的服务。最常见的DoS攻击有计算机网络带宽攻击和连通性攻击。

DoS攻击是指恶意的攻击网络协议实现的缺陷或直接通过野蛮手段残忍地耗尽被攻击对象的资源，目的是让目标计算机或网络无法提供正常的服务或资源访问，使目标系统服务系统停止响应甚至崩溃，而在此攻击中并不包括侵入目标服务器或目标网络设备。这些服务资源包括网络带宽，文件系统空间容量，开放的进程或者允许的连接。这种攻击会导致资源的匮乏，无论计算机的处理速度多快、内存容量多大、网络带宽的速度多快都无法避免这种攻击带来的后果。

哈希碰撞攻击

采用链地址法的情况下发生冲突时会在哈希冲突处构造一个链表，但是在极端情况下，有些恶意的攻击者，可能会通过精心构造的数据，使得所有的数据经过哈希函数之后，都映射到同一个位置，这时候哈希表就会退化为链表，查询的时间复杂度就从 $O(1)$ 急剧退化为 $O(n)$ 。

这样就有可能发生因为查询操作消耗大量 CPU 或者线程资源，而导致系统无法响应其他请求的情况，从而达到拒绝服务攻击（DoS）的目的。

要防止哈希碰撞攻击，就要求我们在设计一个哈希表的时候要考虑到在极端情况下性能也不能退化到无法接受的情况，

防止哈希碰撞 Dos 攻击

一般我们为了防止哈希碰撞攻击需要从以下几个方面着手：

哈希函数需要设计好，即使只有细微改动，经过哈希函数后得到的哈希值也要大不相同，这样可以增加伪造数据的难度。

设计负载因子，支持动态扩容。也就是不要等到哈希表满了才开始扩容，而要达到一定百分比之后就要开始扩容。

选择合适的方法来解决哈希冲突，

如果选择链地址法，可以引入红黑树或者跳表等数据结构来避免出现过长的链表，从而导致性能急剧下降。

当 key 之间的 hash 值相同，或者 hash 到了同一个位置，这类的 hash 冲突可能会导致性能降低。

hash 表采用将链表降级为红黑树来解决这一问题。

TinyLFU的Dos攻击

一种针对 TinyLFU 的攻击行为是人为地提高驱逐策略下的元素的预估频率。

这将导致所有后续进入的元素被频率过滤器所拒绝，导致缓存失效。

一种解决方案是在比较过程中，加入少量抖动使得最后的结果具有一定的不确定性。

这通过 1% 以下的概率选择保留一个将要被驱逐的窗口中的新进度更高的中等访问频率元素来实现。

代码生成

Cache 有许多不同的配置，只有使用特定功能的子集的时候，相关字段才有意义。

如果默认情况下所有字段都被存在，将会导致缓存和每个缓存中的元素的内存开销的浪费。

而通过代码生成的最合理实现，将会减少运行时的内存开销，但是会需要磁盘上更大的二进制文件。

这项技术有通过算法优化的潜力。

也许在构造的时候用户可以根据用法指定最适合的特性。

一个移动应用可能更需要更高的并发率，而服务器可能需要在一定内存开销下更高的命中率。

也许不需要通过不断尝试在所有用法中选择最佳的平衡，而可以通过驱动算法进行选择。

封装 hash map

缓存通过在 ConcurrentHashMap 之上进行封装来添加所需要的特性。

缓存和 hash 表的并发算法非常复杂。通过将两者分开，可以更便利地应用 hash 表的设计的优秀之处，也可以避免更粗粒度的锁覆盖全表和驱逐所引发的问题。

这种方式的成本是额外的运行时开销。

这些字段可以直接内联到表中的元素上，而不是通过包装容纳额外的元数据。缺少包装可以提供单次表操作的快速路径（比如 lambdas）而不是多次 map 调用和短暂存活的对象实例。

之前的项目中探索了两种途径：基于 ConcurrentLinkedHashMap 的封装和 Guava 中 hash 表的分支开发。在最后的設計里，分支开发的想法最后没有实施，因为工程实在是太复杂了。

分层 TimerWheel

一个时间感知 (time-aware) 的优先级队列，该队列使用哈希和双向链接列表在 $O(1)$ 时间内执行操作。

此队列用于变量到期 (`expireAfter(Expiry)`)。

Caffeine 的空间优化与时间优化

Caffeine 的空间优化

Caffeine 如何对一个 key 如何可以节省空间呢？

Caffeine 如何对一个 key 进行统计，但又可以节省空间呢？

在 TinyLFU 中，近似频率的统计如下图所示：

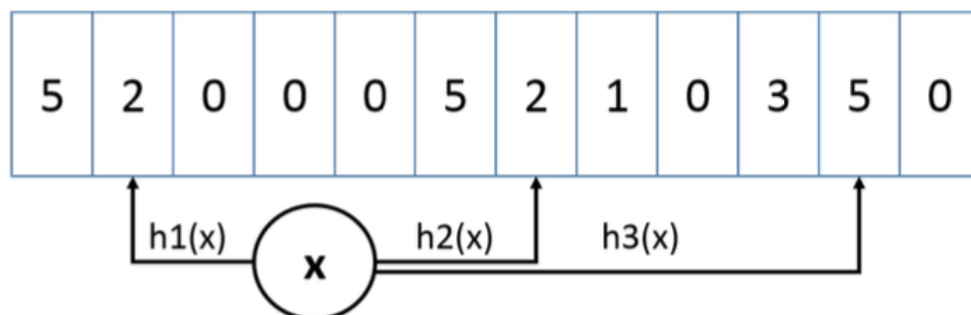


Figure 2: A counting Bloom filter example

Caffeine 对这个算法的实现在 `FrequencySketch` 类。

但 Caffeine 对此有进一步的优化，例如：

- Count-Min Sketch 使用了二维数组，Caffeine 只是用了一个一维的数组；
- Caffeine 认为缓存的访问频率不需要用到那么大，访问频率又是数值类型，这个数需要用 `int` 或 `long` 来存储，但是，只需要 15 就足够，一般认为达到 15 次的频率算是很高的了，而且 Caffeine 还有另外一个机制来使得这个频率进行衰退减半。

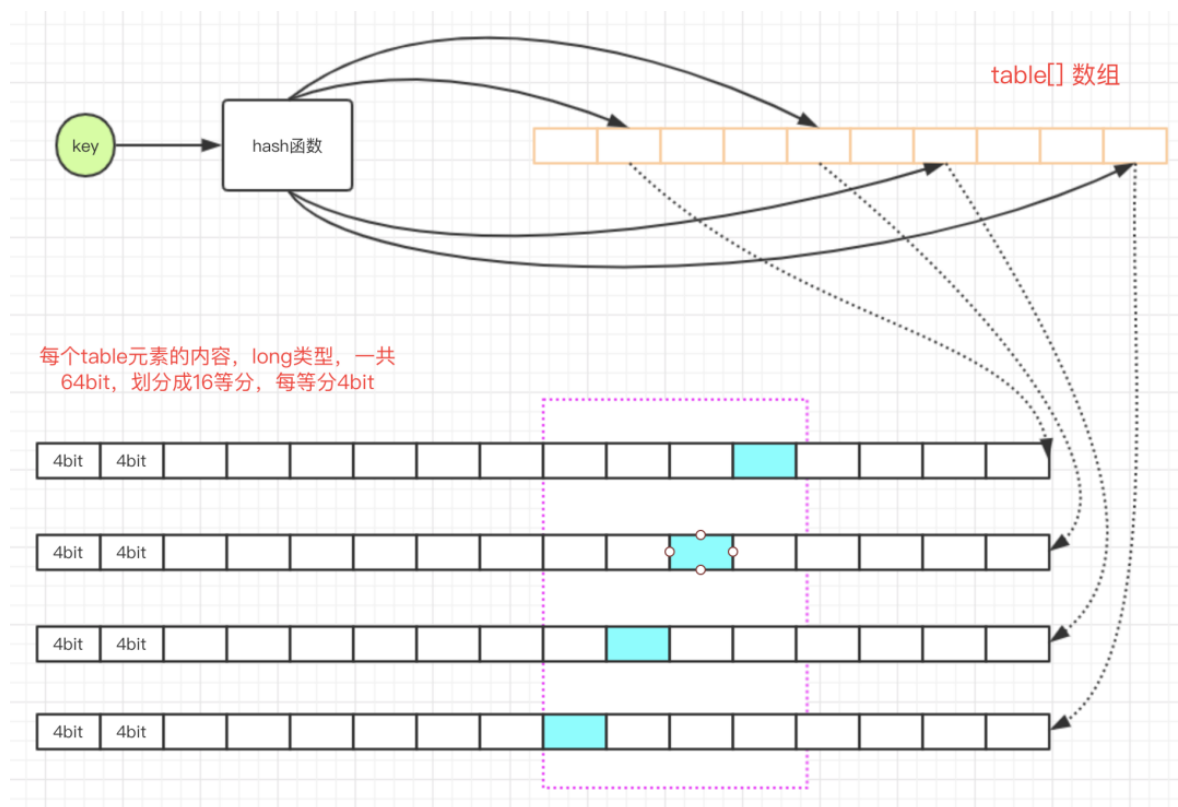
如果最大是 15 的话，那么只需要 4 个 bit 就可以满足了，一个 `long` 有 64bit，可以存储 16 个这样的统计数，Caffeine 就是这样的设计，使得存储效率提高了 16 倍。

Caffeine 对缓存的读写 (`afterRead` 和 `afterwrite` 方法) 都会调用 `onAccess` 方法，而 `onAccess` 方法里有一句：

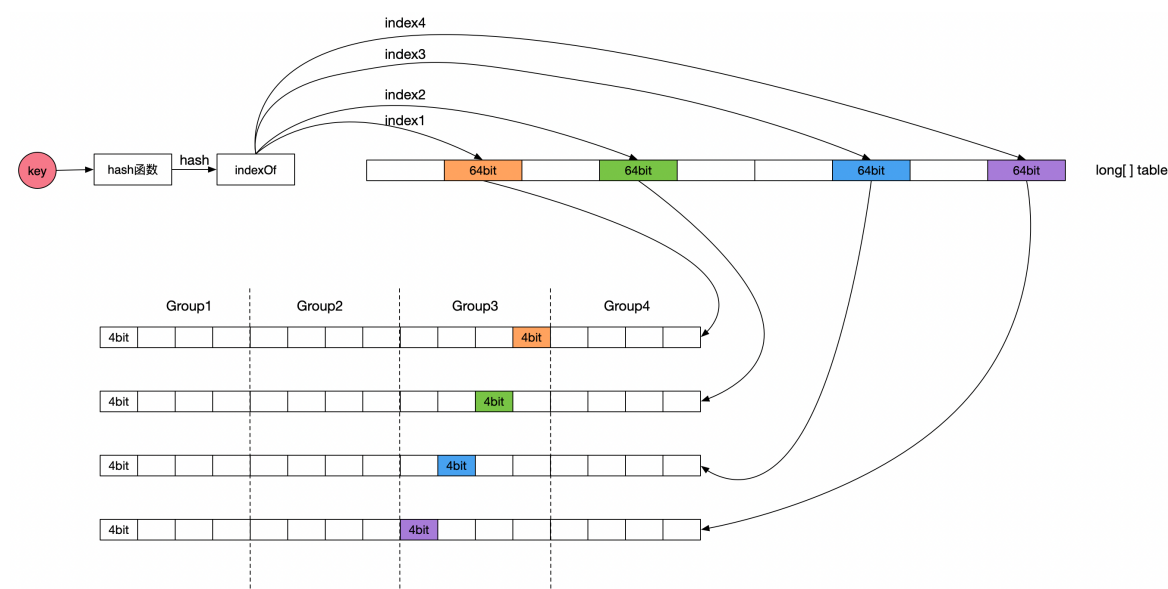
```
frequencySketch().increment(key);
```


通过代码和注释或者读者可能难以理解，下图是我画出来帮助大家理解的结构图。

注意紫色虚线框，其中蓝色小格就是需要计算的位置：



Count-Min Sketch算法详细实现方案如下：



Count-Min Sketch原理

Count-Min Sketch维护了一个 `long[] table` 数组，数组的大小为缓存空间容量（缓存项最大数量）向上取整为2的n次方。Count-Min Sketch的计数器是4bit，table数组的每个元素大小是64bit，相当于table元素包含了16个计数器，这16个计数器进一步分为4个group，那么每个group包含4个计数器，正好等于bloom hash函数的个数，同一个key的四个计数器分别使用group内相应位置的计数器。每个table元素包含16个计数器，4个hash计数器在相应table元素内计数器的偏移不一样，可以有效降低hash冲突。

缓存项计数统计过程为：先计算缓存项key的hash值，然后使用4个不同的种子值分别计算得到table数组四个元素的下标。然后根据hash值的低2bit确定table数组元素中的group，那么第一个计数器位置为第一个table数组元素相应group中的第一个计数器，第二个计数器位置为第二个table数组元素相应group中的第二个计数器，第三个计数器位置为第三个table数组元素相应group中的第三个计数器，第四个计数器位置为第四个table数组元素相应group中的第四个计数器。

从Count-Min Sketch频率统计算法描述可知，由于计数器大小只有4bit，极大地降低了LFU频率统计对存储空间的要求。同时，计数器统计上限是15，并在计数总和达到阈值时所有计数器值减半，相当于引入了计数饱和和衰减机制，可以有效解决短时间内突发大流量不能有效淘汰的问题。比如出现了一个突发热点事件，它的访问量是其他事件的成百上千倍，但是该热点事件很快冷却下去，传统的LFU淘汰机制会让该事件的缓存长时间地保留在缓存中而无法淘汰掉，虽然该类型事件已经访问量非常小或无人问津了。

Caffeine 的保鲜机制

传统LFU一般使用key-value形式来记录每个key的频率，优点是数据结构非常简单，并且能跟缓存本身的数据结构复用，增加一个属性记录频率就行了，它的缺点也比较明显就是频率这个属性会占用很大的空间，但如果改用压缩方式存储频率呢？

频率占用空间肯定可以减少，但会引出另外一个问题：怎么从压缩后的数据里获得对应key的频率呢？

TinyLFU的解决方案是类似位图的方法，将key取hash值获得它的位下标，然后用这个下标来找频率，但位图只有0、1两个值，那频率明显可能会非常大，这要怎么处理呢？另外使用位图需要预占非常大的空间，这个问题怎么解决呢？

TinyLFU根据最大数据量设置生成一个long数组，然后将频率值保存在其中的四个long的4个bit位中（4个bit位不会大于15），取频率值时则取四个中的最小一个。

Caffeine认为频率大于15已经很高了，是属于热数据，所以它只需要4个bit位来保存，long有8个字节64位，这样可以保存16个频率。取hash值的后左移两位，然后加上hash四次，这样可以利用到16个中的13个，利用率挺高的，或许有更好的算法能将16个都利用到。

为了让缓存保证“新鲜度”，剔除掉过往频率很高，但之后不经常的缓存，Caffeine 有一个 Freshness Mechanism。

做法很简答，就是当整体的统计计数（当前所有记录的频率统计之和，这个数值内部维护）达到某一个值时，那么所有记录的频率统计除以 2。

```
*/
@SuppressWarnings("ShortCircuitBoolean")
public void increment(E e) {
    if (isNotInitialized()) {
        return;
    }
    //统计 tinyLFU 的计数
    int[] index = new int[8];
    int blockHash = spread(e.hashCode());
    int counterHash = rehash(blockHash);
    int block = (blockHash & blockMask) << 3;
    for (int i = 0; i < 4; i++) {
        int h = counterHash >>> (i << 3); //0 8 16 24
        index[i] = (h >>> 1) & 15;
        int offset = h & 1;
        index[i + 4] = block + offset + (i << 1); //i << 1: 0,2,4,6
    }
    boolean added =
        incrementAt(index[4], index[0])
```

```

        | incrementAt(index[5], index[1])
        | incrementAt(index[6], index[2])
        | incrementAt(index[7], index[3]);

//当数据写入次数达到数据长度时就重置
if (added && (++size == sampleSize)) {
    reset();
}
}

```

看到 reset 方法就是做这个事情

```

/** Reduces every counter by half of its original value. */
void reset() {
    int count = 0;
    for (int i = 0; i < table.length; i++) {
        count += Long.bitCount(table[i] & ONE_MASK);
        table[i] = (table[i] >>> 1) & RESET_MASK;
    }
    size = (size - (count >>> 2)) >>> 1;
}
}

```

关于这个 reset 方法，为什么是除以 2，而不是其他，及其正确性，在最下面的参考资料的 TinyLFU 论文中 3.3 章节给出了数学证明，大家有兴趣可以看看。

W-TinyLFU 整体设计

上面说到淘汰策略是影响缓存命中率的因素之一，

一般比较简单的缓存就会直接用到 **LFU(Least Frequently Used，即最不经常使用)** 或者 **LRU(Least Recently Used，即最近最少使用)**，而 Caffeine 就是使用了 **W-TinyLFU** 算法。

W-TinyLFU 看名字就能大概猜出来，它是 LFU 的变种，也是一种缓存淘汰算法。

那为什么要使用 W-TinyLFU 呢？

淘汰策略 (eviction policy)

当 window 区满了，就会根据 LRU 把 candidate（即淘汰出来的元素）放到 probation 区，

如果 probation 区也满了，就把 candidate 和 probation 将要淘汰的元素 victim，两个进行“PK”，胜者留在 probation，输者就要被淘汰了。

而且经过实验发现当 window 区配置为总容量的 1%，剩余的 99% 当中的 80% 分给 protected 区，20% 分给 probation 区时，这时整体性能和命中率表现得最好，所以 Caffeine 默认的比例设置就是这个。

不过这个比例 Caffeine 会在运行时根据统计数据 (statistics) 去动态调整，

如果你的应用程序的缓存随着时间变化比较快的话，那么增加 window 区的比例可以提高命中率，相反缓存都是比较固定不变的话，增加 Main Cache 区（protected 区 + probation 区）的比例会有较好的效果。

下面我们看看上面说到的淘汰策略是怎么实现的：

一般缓存对读写操作后都有后续的一系列“维护”操作，Caffeine 也不例外，这些“维护”操作都在 `maintenance` 方法，我们将要说的淘汰策略也在里面。

Caffeine的TinyLFU数据模式

Caffeine的TinyLFU数据模式，来说它使用了三个双端队列：

- `accessOrderEdenDeque`,
- `accessOrderProbationDeque`,
- `accessOrderProtectedDeque`,

使用双端队列的原因是支持LRU算法比较方便。

`accessOrderEdenDeque`属于eden区，缓存1%的数据，其余的99%缓存在main区。

`accessOrderProbationDeque`属于main区，缓存main内数据的20%，这部分是属于冷数据，即将被淘汰。

`accessOrderProtectedDeque`属于main区，缓存main内数据的80%，这部分是属于热数据，是整个缓存的主存区。

我们先看一下淘汰方法入口：

```
/**
 * Evicts entries if the cache exceeds the maximum.
 */
@GuardedBy("evictionLock")
void evictEntries() {
    if (!evicts()) {
        return;
    }
    //todo 高并发异步删除 4.1 伊甸园的候选人，和考察区的牺牲者PK之战 by nien at
    2022/11/30
    // 淘汰window区的记录
    // candidate 第一个 准备晋升的元素
    var candidate = evictFromWindow();

    //淘汰Main区的记录
    evictFromMain(candidate);
}
```

`accessOrderEdenDeque`对应W-TinyLFU的W(window)，这里保存的是最新写入数据的引用，它使用LRU淘汰，

这里面的数据主要是应对突发流量的问题，淘汰后的数据进入`accessOrderProbationDeque`。

代码如下：

```

@GuardedBy("evictionLock")
@Nullable Node<K, V> evictFromWindow() {
    Node<K, V> first = null;
    //todo 高并发异步处理 5.2 从伊甸园升级到考察区 by nien at 2022/11/30

    //node = window queue的头部节点
    Node<K, V> node = accessOrderWindowDeque().peekFirst();

    //一直循环: 如果window区超过了最大的限制, 那么就要把“多出来”的记录做处理
    while (windowWeightedSize() > windowMaximum()) {
        // The pending operations will adjust the size to reflect the
correct weight
        if (node == null) {
            break;
        }
        //下一个节点
        Node<K, V> next = node.getNextInAccessOrder();
        if (node.getPolicyWeight() != 0) {
            //设置 node 的类型: 为观察类型 probation
            node.makeMainProbation();

            //todo 高并发写的关键代码 5.3 的呼应代码: 节点从 lru 队列 移除 by
nien at 2022/11/28
            // 从window区去掉
            // node = accessOrderWindowDeque().peekFirst()
            accessOrderWindowDeque().remove(node);

            //加入到probation queue, 相当于把节点移动到probation区 (开始准备晋升)
            accessOrderProbationDeque().offerLast(node);

            // 记录一下第一个 准备晋升的元素
            if (first == null) {
                first = node;
            }
            //因为window移除了一个节点, 所以需要调整 size
            setWindowWeightedSize(windowWeightedSize() -
node.getPolicyWeight());
        }
        node = next;
    }
    //第一个 准备晋升的元素
    return first;
}

```

数据进入probation队列后, 继续执行以下代码:

```

@GuardedBy("evictionLock")
void evictFromMain(@Nullable Node<K, V> candidate) {
    int victimQueue = PROBATION;
    int candidateQueue = PROBATION;

    // 迭代处理 考察区 probation queue
    // victim 是probation queue的头部
    Node<K, V> victim = accessOrderProbationDeque().peekFirst();
    while (weightedSize() > maximum()) {
        // Search the admission window for additional candidates

```

```

// candidate 刚从window晋升来的，最先晋升的那个 元素
if ((candidate == null) && (candidateQueue == PROBATION)) {
    candidate = accessOrderWindowDeque().peekFirst();
    candidateQueue = WINDOW;
}

// Try evicting from the protected and window queues
if ((candidate == null) && (victim == null)) {
    if (victimQueue == PROBATION) {

        //todo 高并发异步删除 4.2 考察区的牺牲者 by nien at 2022/11/30

        victim = accessOrderProtectedDeque().peekFirst();
        victimQueue = PROTECTED;
        continue;
    } else if (victimQueue == PROTECTED) {
        victim = accessOrderWindowDeque().peekFirst();
        victimQueue = WINDOW;
        continue;
    }

    // The pending operations will adjust the size to reflect the
correct weight
    break;
}

// Skip over entries with zero weight
if ((victim != null) && (victim.getPolicyWeight() == 0)) {
    victim = victim.getNextInAccessOrder();
    continue;
} else if ((candidate != null) && (candidate.getPolicyWeight() ==
0)) {

    candidate = candidate.getNextInAccessOrder();
    continue;
}

// Evict immediately if only one of the entries is present
if (victim == null) {
    @SuppressWarnings("NullAway")
    Node<K, V> previous = candidate.getNextInAccessOrder();
    Node<K, V> evict = candidate;
    candidate = previous;
    evictEntry(evict, RemovalCause.SIZE, 0L);
    continue;
} else if (candidate == null) {
    Node<K, V> evict = victim;
    victim = victim.getNextInAccessOrder();
    evictEntry(evict, RemovalCause.SIZE, 0L);
    continue;
}

// Evict immediately if both selected the same entry
if (candidate == victim) {
    victim = victim.getNextInAccessOrder();
    evictEntry(candidate, RemovalCause.SIZE, 0L);
    candidate = null;
    continue;
}
}

```

```

// Evict immediately if an entry was collected
K victimKey = victim.getKey();
K candidateKey = candidate.getKey();
if (victimKey == null) {
    Node<K, V> evict = victim;
    victim = victim.getNextInAccessOrder();
    evictEntry(evict, RemovalCause.COLLECTED, 0L);
    continue;
} else if (candidateKey == null) {
    Node<K, V> evict = candidate;
    candidate = candidate.getNextInAccessOrder();
    evictEntry(evict, RemovalCause.COLLECTED, 0L);
    continue;
}

// Evict immediately if an entry was removed
if (!victim.isAlive()) {
    Node<K, V> evict = victim;
    victim = victim.getNextInAccessOrder();
    evictEntry(evict, RemovalCause.SIZE, 0L);
    continue;
} else if (!candidate.isAlive()) {
    Node<K, V> evict = candidate;
    candidate = candidate.getNextInAccessOrder();
    evictEntry(evict, RemovalCause.SIZE, 0L);
    continue;
}

// Evict immediately if the candidate's weight exceeds the maximum
if (candidate.getPolicyWeight() > maximum()) {
    Node<K, V> evict = candidate;
    candidate = candidate.getNextInAccessOrder();
    evictEntry(evict, RemovalCause.SIZE, 0L);
    continue;
}

//todo 高并发异步删除 4.3 candidate和 victim的频率PK之战 by nien at
2022/11/30

// Evict the entry with the lowest frequency
//根据节点的统计频率frequency来做比较，看看要处理掉victim还是candidate
// admit = ture : 准许 candidate 加入 ; false: 淘汰 candidate

if (admit(candidateKey, victimKey)) {
    Node<K, V> evict = victim;
    victim = victim.getNextInAccessOrder();
    // 删除 victim , 从而留下 candidate
    evictEntry(evict, RemovalCause.SIZE, 0L);
    candidate = candidate.getNextInAccessOrder();
} else {
    Node<K, V> evict = candidate;
    candidate = candidate.getNextInAccessOrder();
    // 删除 candidate , 从而留下 victim
    evictEntry(evict, RemovalCause.SIZE, 0L);
}
}
}

```


上面的代码逻辑是从probation的头尾取出两个node进行比较频率，频率更小者将被remove，其中尾部元素就是上一部分从eden中淘汰出来的元素，

如果将两步逻辑合并起来讲是这样的：

在eden队列通过lru淘汰出来的“候选者”与probation队列通过lru淘汰出来的“被驱逐者”进行频率比较，失败者将被从cache中真正移除。

下面看一下它的比较逻辑admit：

```
//准许 candidate 加入：
boolean admit(K candidateKey, K victimKey) {
    //分别获取victim和candidate的统计频率
    int victimFreq = frequencySketch().frequency(victimKey);
    int candidateFreq = frequencySketch().frequency(candidateKey);
    // 伊甸园的末位 > 考察区的末位
    if (candidateFreq > victimFreq) {
        return true;    //ture : 准许 candidate 加入, 淘汰 victimKey ;
    } else if (candidateFreq >= ADMIT_HASHDOS_THRESHOLD) {
        // The maximum frequency is 15 and halved to 7 after a reset to age
        // the history. An attack
        // exploits that a hot candidate is rejected in favor of a hot
        // victim. The threshold of a warm
        // candidate reduces the number of random acceptances to minimize
        // the impact on the hit rate.
        int random = ThreadLocalRandom.current().nextInt();
        return ((random & 127) == 0);
    }
    // 伊甸园的末位 太小    false: 淘汰 candidate, 留下victimKey
    return false;
}
```

从frequencySketch取出候选者与被驱逐者的频率，如果候选者的频率高就淘汰被驱逐者，如果被驱逐者比候选者的频率高，并且候选者频率小于等于5则淘汰者，如果前面两个条件都不满足则随机淘汰。

probation中这个数据, 如何移动到protected中去的呢？

```
@GuardedBy("evictionLock")
void reorderProbation(Node<K, V> node) {
    if (!accessOrderProbationDeque().contains(node)) {
        // Ignore stale accesses for an entry that is no longer present
        return;
    } else if (node.getPolicyweight() > mainProtectedMaximum()) {
        reorder(accessOrderProbationDeque(), node);
        return;
    }

    // If the protected space exceeds its maximum, the LRU items are demoted
    // to the probation space.
    // This is deferred to the adaption phase at the end of the maintenance
    // cycle.
    setMainProtectedWeightedSize(mainProtectedWeightedSize() +
    node.getPolicyweight());
    accessOrderProbationDeque().remove(node);
}
```



```
        accessOrderProtectedDeque().offerLast(node);
        node.makeMainProtected();
    }
```

当数据被访问时并且该数据在probation中，这个数据就会移动到protected中去，同时通过lru从protected中淘汰一个数据进入到probation中。

这样数据流转的逻辑全部通了：

新数据都会进入到eden中，通过lru淘汰到probation,并与probation中通过lru淘汰的数据进行使用频率pk,如果胜利了就继续留在probation中，如果失败了就会被直接淘汰，当这条数据被访问了，则移动到protected。当其它数据被访问了，则它可能会从protected中通过lru淘汰到probation中。

读写操作后都有后续的一系列“维护”操作

一般缓存对读写操作后都有后续的一系列“维护”操作，Caffeine也不例外，这些“维护”操作都在 `maintenance` 方法，我们将要说的淘汰策略也在里面。

maintenance 过程

这方法比较重要，下面也会提到，

所以这里只先说跟“淘汰策略”有关的 `evictEntries` 和 `climb`。

```
/**
 * Performs the pending maintenance work and sets the state flags during
 * processing to avoid
 * excess scheduling attempts.
 *
 * 排空：
 * 1 The read buffer
 * 2 write buffer
 * 3. reference queues
 *
 * The read buffer, write buffer, and reference queues are drained,
 * followed by expiration, and size-based eviction.
 *
 * @param task an additional pending task to run, or {@code null} if not
 * present
 */
@GuardedBy("evictionLock")
void maintenance(@Nullable Runnable task) {
    setDrainStatusRelease(PROCESSING_TO_IDLE);

    try {
        drainReadBuffer();

        drainWriteBuffer();
        if (task != null) {
```

```

        task.run();
    }

    drainKeyReferences();
    drainValueReferences();
    //过期符合条件的记录
    expireEntries();
    //淘汰符合条件的记录
    evictEntries();

    //动态调整window区和protected区的大小
    climb();
} finally {
    if ((drainStatusOpaque() != PROCESSING_TO_IDLE) ||
!casDrainStatus(PROCESSING_TO_IDLE, IDLE)) {
        setDrainStatusOpaque(REQUIRED);
    }
}
}

```

1. 设置状态位为 PROCESSING_TO_IDLE
2. 清空读缓存
3. 清空写缓存
4. 一般只有 afterWrite 的情况有正在执行的 task（比如添加缓存项时发现已达到最大上限，此时 task 就是正在进行的添加缓存的操作），如果有则执行 task
5. 清空 key 引用和 value 引用队列
6. 处理过期项
7. 淘汰项
8. 调整窗口比例（climbing hill 算法）
9. 尝试将状态从 PROCESSING_TO_IDLE 改成 IDLE，否则记为 REQUIRED

这里有一个小设计：`BLCHeader.DrainStatusRef<K, V>` 包含一个 volatile 的 drainStatus 状态位 + 15 个 long 的填充位。

注：`lazySetDrainStatus` 本质调用的是 `unsafe` 的 `putOrderedInt` 方法，可以 lazy set 一个 volatile 的变量

W-TinyLFU 策略的实现用到的数据结构

先说一下 Caffeine 对上面说到的 W-TinyLFU 策略的实现用到的数据结构：

```

//最大的个数限制
long maximum;
//当前的个数
long weightedSize;
//window区的最大限制
long windowMaximum;
//window区当前的个数
long windowWeightedSize;
//protected区的最大限制
long mainProtectedMaximum;

```

```

//protected区当前的个数
long mainProtectedWeightedSize;
//下一次需要调整的大小（还需要进一步计算）
double stepSize;
//window区需要调整的大小
long adjustment;
//命中计数
int hitsInSample;
//不命中的计数
int missesInSample;
//上一次的缓存命中率
double previousSampleHitRate;

final FrequencySketch<K> sketch;
//window区的LRU queue (FIFO)
final AccessOrderDeque<Node<K, V>> accessOrderWindowDeque;
//probation区的LRU queue (FIFO)
final AccessOrderDeque<Node<K, V>> accessOrderProbationDeque;
//protected区的LRU queue (FIFO)
final AccessOrderDeque<Node<K, V>> accessOrderProtectedDeque;

```

以及默认比例设置（意思看注释）

```

/** The initial percent of the maximum weighted capacity dedicated to the main
space. */
static final double PERCENT_MAIN = 0.99d;
/** The percent of the maximum weighted capacity dedicated to the main's
protected space. */
static final double PERCENT_MAIN_PROTECTED = 0.80d;
/** The difference in hit rates that restarts the climber. */
static final double HILL_CLIMBER_RESTART_THRESHOLD = 0.05d;
/** The percent of the total size to adapt the window by. */
static final double HILL_CLIMBER_STEP_PERCENT = 0.0625d;
/** The rate to decrease the step size to adapt by. */
static final double HILL_CLIMBER_STEP_DECAY_RATE = 0.98d;
/** The maximum number of entries that can be transfered between queues. */

/** The initial percent of the maximum weighted capacity dedicated to the main
space. */

static final double PERCENT_MAIN = 0.99d;

/** The percent of the maximum weighted capacity dedicated to the main's
protected space. */
static final double PERCENT_MAIN_PROTECTED = 0.80d;

/** The difference in hit rates that restarts the climber. */

static final double HILL_CLIMBER_RESTART_THRESHOLD = 0.05d;
/** The percent of the total size to adapt the window by. */

static final double HILL_CLIMBER_STEP_PERCENT = 0.0625d;
/** The rate to decrease the step size to adapt by. */

static final double HILL_CLIMBER_STEP_DECAY_RATE = 0.98d;

/** The maximum number of entries that can be transfered between queues. */

```

重点来了，`evictEntries` 和 `climb` 方法：

```
/** Evicts entries if the cache exceeds the maximum. */
@GuardedBy("evictionLock")
void evictEntries() {
    if (!evicts()) {
        return;
    }
    //淘汰window区的记录
    int candidates = evictFromWindow();
    //淘汰Main区的记录
    evictFromMain(candidates);
}
```

evictFromWindow方法

```
/**
 * Evicts entries from the window space into the main space while the window
 * size exceeds a
 * maximum.
 *
 * @return the number of candidate entries evicted from the window space
 */
//根据w-TinyLFU，新的数据都会无条件的加到admission window
//但是window是有大小限制，所以要“定期”做一下“维护”
@GuardedBy("evictionLock")
int evictFromWindow() {
    int candidates = 0;
    //查看window queue的头部节点
    Node<K, V> node = accessOrderWindowDeque().peek();
    //如果window区超过了最大的限制，那么就要把“多出来”的记录做处理
    while (windowWeightedSize() > windowMaximum()) {
        // The pending operations will adjust the size to reflect the correct weight
        if (node == null) {
            break;
        }
        //下一个节点
        Node<K, V> next = node.getNextInAccessOrder();
        if (node.getWeight() != 0) {
            //把node定位在probation区
            node.makeMainProbation();
            //从window区去掉
            accessOrderWindowDeque().remove(node);
            //加入到probation queue，相当于把节点移动到probation区（晋升了）
            accessOrderProbationDeque().add(node);
            candidates++;
            //因为移除了一个节点，所以需要调整window的size
            setWindowWeightedSize(windowWeightedSize() - node.getPolicyWeight());
        }
        //处理下一个节点
        node = next;
    }
}
```

```

    }

    return candidates;
}

```

evictFromMain方法:

```

//根据W-TinyLFU，从window晋升过来的要跟probation区的进行“PK”，胜者才能留下

@GuardedBy("evictionLock")
void evictFromMain(@Nullable Node<K, V> candidate) {
    int victimQueue = PROBATION;
    int candidateQueue = PROBATION;

    // 迭代处理 考察区 probation queue
    // victim 是probation queue的头部
    Node<K, V> victim = accessOrderProbationDeque().peekFirst();
    while (weightedSize() > maximum()) {
        // Search the admission window for additional candidates
        // candidate 刚从window晋升来的，最先晋升的那个 元素
        if ((candidate == null) && (candidateQueue == PROBATION)) {
            candidate = accessOrderWindowDeque().peekFirst();
            candidateQueue = WINDOW;
        }

        // Try evicting from the protected and window queues
        if ((candidate == null) && (victim == null)) {
            if (victimQueue == PROBATION) {
                victim = accessOrderProtectedDeque().peekFirst();
                victimQueue = PROTECTED;
                continue;
            } else if (victimQueue == PROTECTED) {
                victim = accessOrderWindowDeque().peekFirst();
                victimQueue = WINDOW;
                continue;
            }

            // The pending operations will adjust the size to reflect the correct
            weight
            break;
        }

        // Skip over entries with zero weight
        if ((victim != null) && (victim.getPolicyweight() == 0)) {
            victim = victim.getNextInAccessOrder();
            continue;
        } else if ((candidate != null) && (candidate.getPolicyweight() == 0)) {
            candidate = candidate.getNextInAccessOrder();
            continue;
        }

        // Evict immediately if only one of the entries is present
        if (victim == null) {
            @SuppressWarnings("NullAway")
            Node<K, V> previous = candidate.getNextInAccessOrder();

```

```

    Node<K, V> evict = candidate;
    candidate = previous;
    evictEntry(evict, RemovalCause.SIZE, 0L);
    continue;
} else if (candidate == null) {
    Node<K, V> evict = victim;
    victim = victim.getNextInAccessOrder();
    evictEntry(evict, RemovalCause.SIZE, 0L);
    continue;
}

// Evict immediately if both selected the same entry
if (candidate == victim) {
    victim = victim.getNextInAccessOrder();
    evictEntry(candidate, RemovalCause.SIZE, 0L);
    candidate = null;
    continue;
}

// Evict immediately if an entry was collected
K victimKey = victim.getKey();
K candidateKey = candidate.getKey();
if (victimKey == null) {
    Node<K, V> evict = victim;
    victim = victim.getNextInAccessOrder();
    evictEntry(evict, RemovalCause.COLLECTED, 0L);
    continue;
} else if (candidateKey == null) {
    Node<K, V> evict = candidate;
    candidate = candidate.getNextInAccessOrder();
    evictEntry(evict, RemovalCause.COLLECTED, 0L);
    continue;
}

// Evict immediately if an entry was removed
if (!victim.isAlive()) {
    Node<K, V> evict = victim;
    victim = victim.getNextInAccessOrder();
    evictEntry(evict, RemovalCause.SIZE, 0L);
    continue;
} else if (!candidate.isAlive()) {
    Node<K, V> evict = candidate;
    candidate = candidate.getNextInAccessOrder();
    evictEntry(evict, RemovalCause.SIZE, 0L);
    continue;
}

// Evict immediately if the candidate's weight exceeds the maximum
if (candidate.getPolicyweight() > maximum()) {
    Node<K, V> evict = candidate;
    candidate = candidate.getNextInAccessOrder();
    evictEntry(evict, RemovalCause.SIZE, 0L);
    continue;
}

// Evict the entry with the lowest frequency
//根据节点的统计频率frequency来做比较，看看要处理掉victim还是candidate
// admit = true : 准许 candidate 加入 ; false: 淘汰 candidate

```

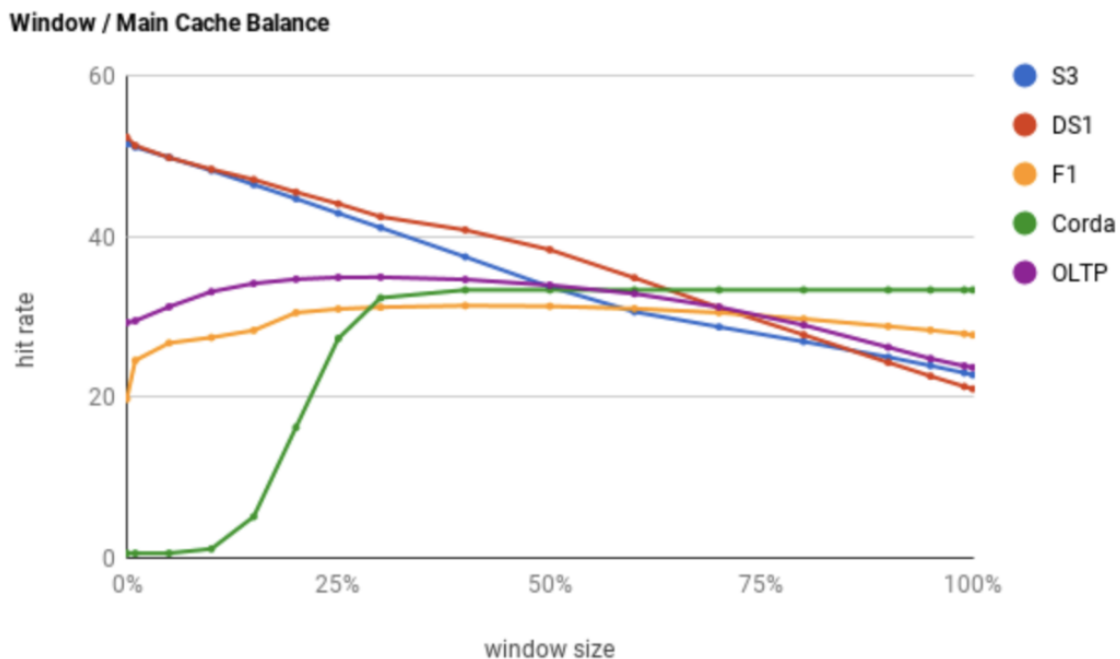
```

if (admit(candidateKey, victimKey)) {
    Node<K, V> evict = victim;
    victim = victim.getNextInAccessOrder();
    // 删除 victim，从而留下 candidate
    evictEntry(evict, RemovalCause.SIZE, 0L);
    candidate = candidate.getNextInAccessOrder();
} else {
    Node<K, V> evict = candidate;
    candidate = candidate.getNextInAccessOrder();
    // 删除 candidate，从而留下 victim
    evictEntry(evict, RemovalCause.SIZE, 0L);
}
}
}
}

```

`climb` 方法主要是用来调整 window size 的，使得 Caffeine 可以适应你的应用类型（如 OLAP 或 OLTP）表现出最佳的命中率。

下图是官方测试的数据：



调整时用到的默认比例数据：

```

//与上次命中率之差的阈值
static final double HILL_CLIMBER_RESTART_THRESHOLD = 0.05d;
//步长（调整）的大小（跟最大值maximum的比例）
static final double HILL_CLIMBER_STEP_PERCENT = 0.0625d;
//步长的衰减比例
static final double HILL_CLIMBER_STEP_DECAY_RATE = 0.98d;
/** Adapts the eviction policy to towards the optimal recency / frequency
configuration. */
//climb方法的主要作用就是动态调整window区的大小（相应的，main区的大小也会发生变化，两个之和为
100%）。
//因为区域的大小发生了变化，那么区域内的数据也可能需要发生相应的移动。
@GuardedBy("evictionLock")
void climb() {
    if (!evicts()) {

```

```

        return;
    }
    //确定window需要调整的大小
    determineAdjustment();
    //如果protected区有溢出，把溢出部分移动到probation区。因为下面的操作有可能需要调整到protected区。
    demoteFromMainProtected();
    long amount = adjustment();
    if (amount == 0) {
        return;
    } else if (amount > 0) {
        //增加window的大小
        increasewindow();
    } else {
        //减少window的大小
        decreasewindow();
    }
}

```

下面分别展开每个方法来解释：

```

/** calculates the amount to adapt the window by and sets {@link #adjustment()} accordingly. */
@GuardedBy("evictionLock")
void determineAdjustment() {
    //如果frequencysketch还没初始化，则返回
    if (frequencysketch().isNotInitialized()) {
        setPreviousSampleHitRate(0.0);
        setMissesInSample(0);
        setHitsInSample(0);
        return;
    }
    //总请求量 = 命中 + miss
    int requestCount = hitsInSample() + missesInSample();
    //没达到sampleSize则返回
    //默认下sampleSize = 10 * maximum。用sampleSize来判断缓存是否足够“热”。
    if (requestCount < frequencysketch().sampleSize) {
        return;
    }

    //命中率的公式 = 命中 / 总请求
    double hitRate = (double) hitsInSample() / requestCount;
    //命中率的差值
    double hitRateChange = hitRate - previousSampleHitRate();
    //本次调整的大小，是由命中率的差值和上次的stepSize决定的
    double amount = (hitRateChange >= 0) ? stepSize() : -stepSize();
    //下次的调整大小：如果命中率的之差大于0.05，则重置为0.065 * maximum，否则按照0.98来进行衰减
    double nextStepSize = (Math.abs(hitRateChange) >=
HILL_CLIMBER_RESTART_THRESHOLD)
        ? HILL_CLIMBER_STEP_PERCENT * maximum() * (amount >= 0 ? 1 : -1)
        : HILL_CLIMBER_STEP_DECAY_RATE * amount;
    setPreviousSampleHitRate(hitRate);
    setAdjustment((long) amount);
    setStepSize(nextStepSize);
    setMissesInSample(0);
    setHitsInSample(0);
}

```



```

}

/** Transfers the nodes from the protected to the probation region if it exceeds
the maximum. */

//这个方法比较简单，减少protected区溢出的部分
@GuardedBy("evictionLock")
void demoteFromMainProtected() {
    long mainProtectedMaximum = mainProtectedMaximum();
    long mainProtectedWeightedSize = mainProtectedWeightedSize();
    if (mainProtectedWeightedSize <= mainProtectedMaximum) {
        return;
    }

    for (int i = 0; i < QUEUE_TRANSFER_THRESHOLD; i++) {
        if (mainProtectedWeightedSize <= mainProtectedMaximum) {
            break;
        }

        Node<K, V> demoted = accessOrderProtectedDeque().poll();
        if (demoted == null) {
            break;
        }
        demoted.makeMainProbation();
        accessOrderProbationDeque().add(demoted);
        mainProtectedWeightedSize -= demoted.getPolicyWeight();
    }
    setMainProtectedWeightedSize(mainProtectedWeightedSize);
}

/**
 * Increases the size of the admission window by shrinking the portion allocated
to the main
 * space. As the main space is partitioned into probation and protected regions
(80% / 20%), for
 * simplicity only the protected is reduced. If the regions exceed their
maximums, this may cause
 * protected items to be demoted to the probation region and probation items to
be demoted to the
 * admission window.
 */

//增加window区的大小，这个方法比较简单，思路就像我上面说的
@GuardedBy("evictionLock")
void increasewindow() {
    if (mainProtectedMaximum() == 0) {
        return;
    }

    long quota = Math.min(adjustment(), mainProtectedMaximum());
    setMainProtectedMaximum(mainProtectedMaximum() - quota);
    setWindowMaximum(windowMaximum() + quota);
    demoteFromMainProtected();

    for (int i = 0; i < QUEUE_TRANSFER_THRESHOLD; i++) {
        Node<K, V> candidate = accessOrderProbationDeque().peek();
        boolean probation = true;
        if ((candidate == null) || (quota < candidate.getPolicyWeight())) {

```

```

        candidate = accessOrderProtectedDeque().peek();
        probation = false;
    }
    if (candidate == null) {
        break;
    }

    int weight = candidate.getPolicyWeight();
    if (quota < weight) {
        break;
    }

    quota -= weight;
    if (probation) {
        accessOrderProbationDeque().remove(candidate);
    } else {
        setMainProtectedWeightedSize(mainProtectedWeightedSize() - weight);
        accessOrderProtectedDeque().remove(candidate);
    }
    setWindowWeightedSize(windowWeightedSize() + weight);
    accessOrderWindowDeque().add(candidate);
    candidate.makeWindow();
}

setMainProtectedMaximum(mainProtectedMaximum() + quota);
setWindowMaximum(windowMaximum() - quota);
setAdjustment(quota);
}

/** Decreases the size of the admission window and increases the main's
protected region. */
//同上increaseWindow差不多，反操作
@GuardedBy("evictionLock")
void decreaseWindow() {
    if (windowMaximum() <= 1) {
        return;
    }

    long quota = Math.min(-adjustment(), Math.max(0, windowMaximum() - 1));
    setMainProtectedMaximum(mainProtectedMaximum() + quota);
    setWindowMaximum(windowMaximum() - quota);

    for (int i = 0; i < QUEUE_TRANSFER_THRESHOLD; i++) {
        Node<K, V> candidate = accessOrderWindowDeque().peek();
        if (candidate == null) {
            break;
        }

        int weight = candidate.getPolicyWeight();
        if (quota < weight) {
            break;
        }

        quota -= weight;
        setMainProtectedWeightedSize(mainProtectedWeightedSize() + weight);
        setWindowWeightedSize(windowWeightedSize() - weight);
        accessOrderWindowDeque().remove(candidate);
        accessOrderProbationDeque().add(candidate);
    }
}

```

```

        candidate.makeMainProbation();
    }

    setMainProtectedMaximum(mainProtectedMaximum() - quota);
    setWindowMaximum(windowMaximum() + quota);
    setAdjustment(-quota);
}

```

以上，是 Caffeine 的 W-TinyLFU 策略的设计原理及代码实现解析。

说明：本文会持续更新，最新PDF电子版本，请从下面的连接获取：[语雀](#) 或者 [码云](#)

清空读缓冲

清空就是将所有的 readBuffer 使用 accessPolicy 清空。

```

/** Drains the read buffer. */
@GuardedBy("evictionLock")
void drainReadBuffer() {
    if (!skipReadBuffer()) {
        readBuffer.drainTo(accessPolicy);
    }
}

```

accessPolicy 在前面设置了的

```
accessPolicy = (evicts() || expiresAfterAccess()) ? this::onAccess : e -> {};
```

onAccess的代码如下：

```

/** Updates the node's location in the page replacement policy. */
@GuardedBy("evictionLock")
void onAccess(Node<K, V> node) {
    if (evicts()) {
        K key = node.getKey();
        if (key == null) {
            return;
        }
        frequencySketch().increment(key);
        if (node.inwindow()) {
            reorder(accessOrderWindowDeque(), node);
        } else if (node.inMainProbation()) {
            reorderProbation(node);
        } else {
            reorder(accessOrderProtectedDeque(), node);
        }
        setHitsInSample(hitsInSample() + 1);
    }
}

```

```

    } else if (expiresAfterAccess()) {
        reorder(accessOrderWindowDeque(), node);
    }
    if (expiresVariable()) {
        timerwheel().reschedule(node);
    }
}

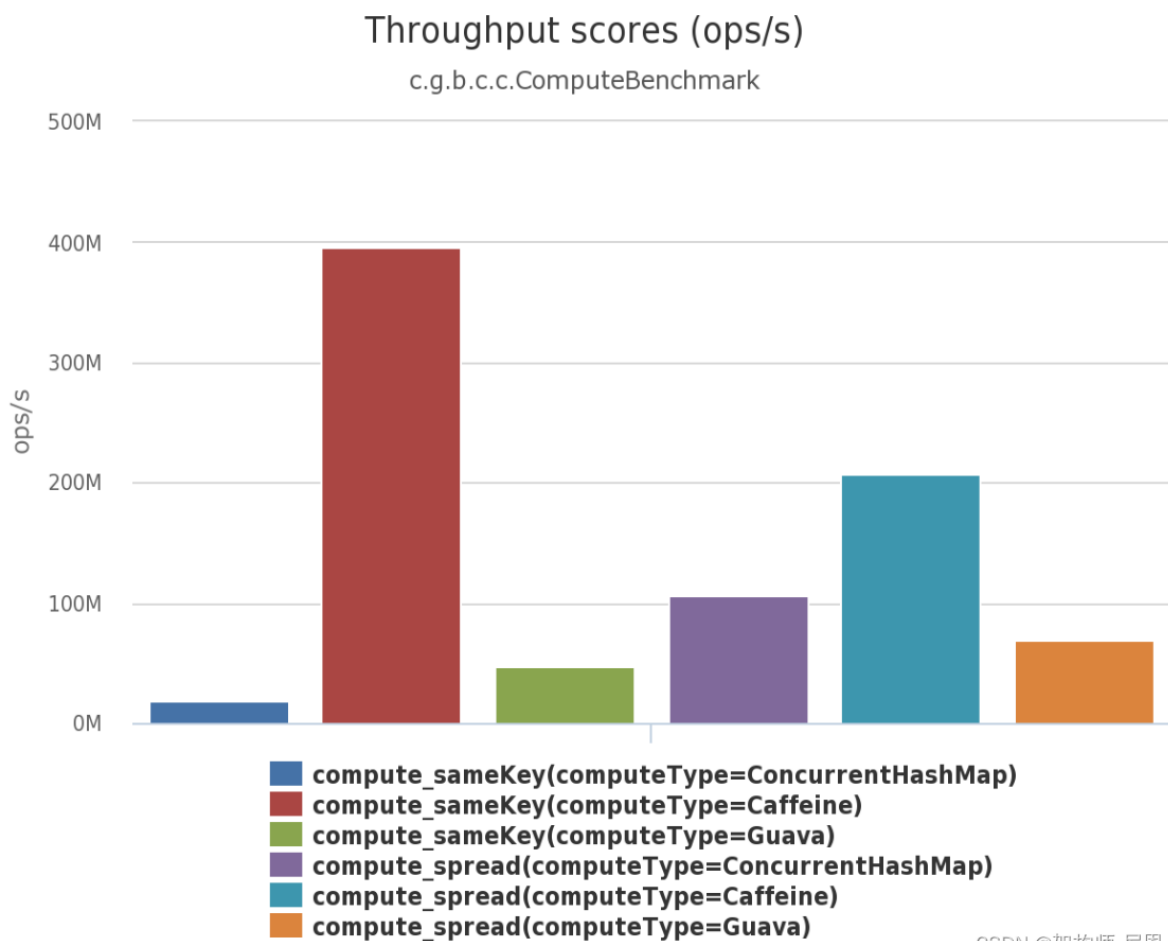
```

这个 onAccess 主要是：

- 统计 tinyLFU 的计数
- 将节点在队列中重排序，
- 以及更新统计信息。

Caffeine 的性能比较

吞吐量 PK

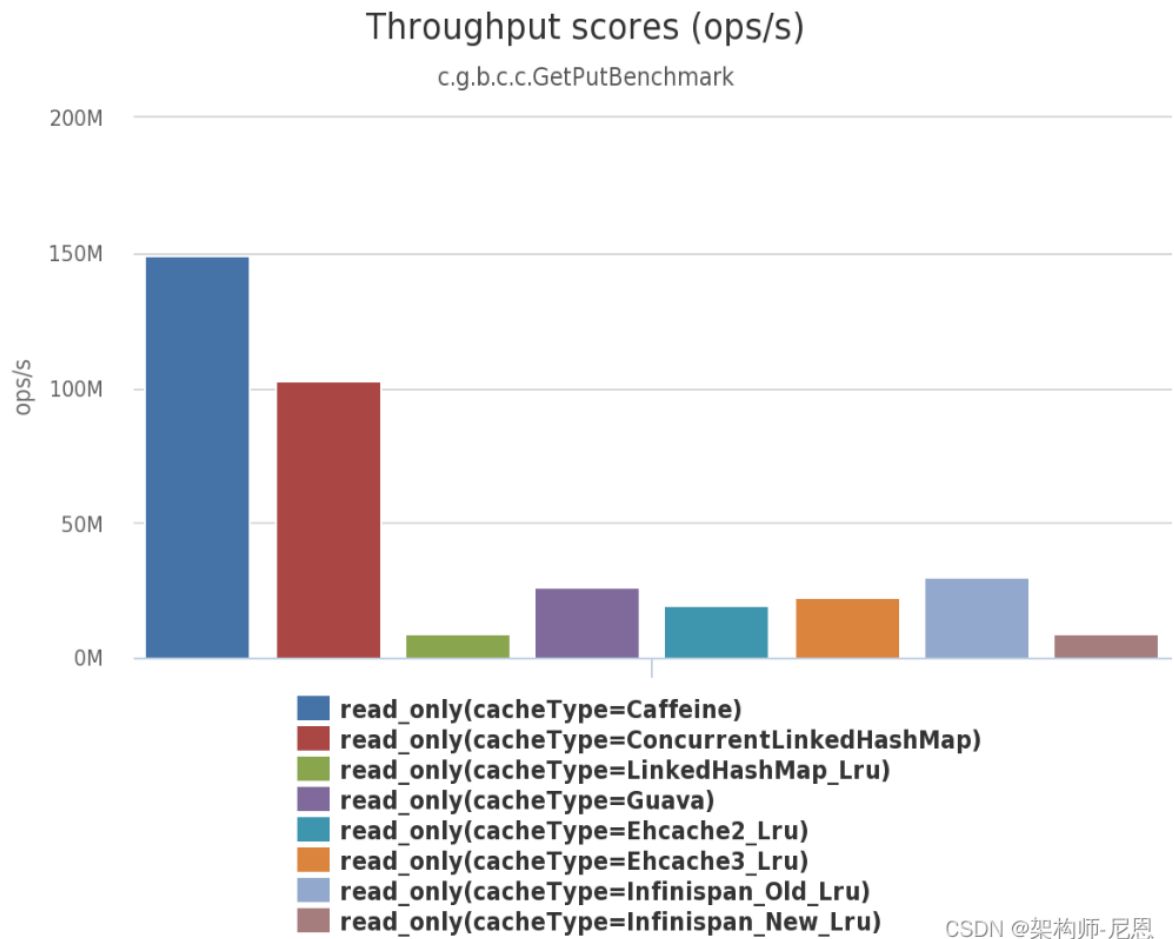


CSDN @架构师-尼恩

可以清楚的看到Caffeine效率明显的高于其他缓存。

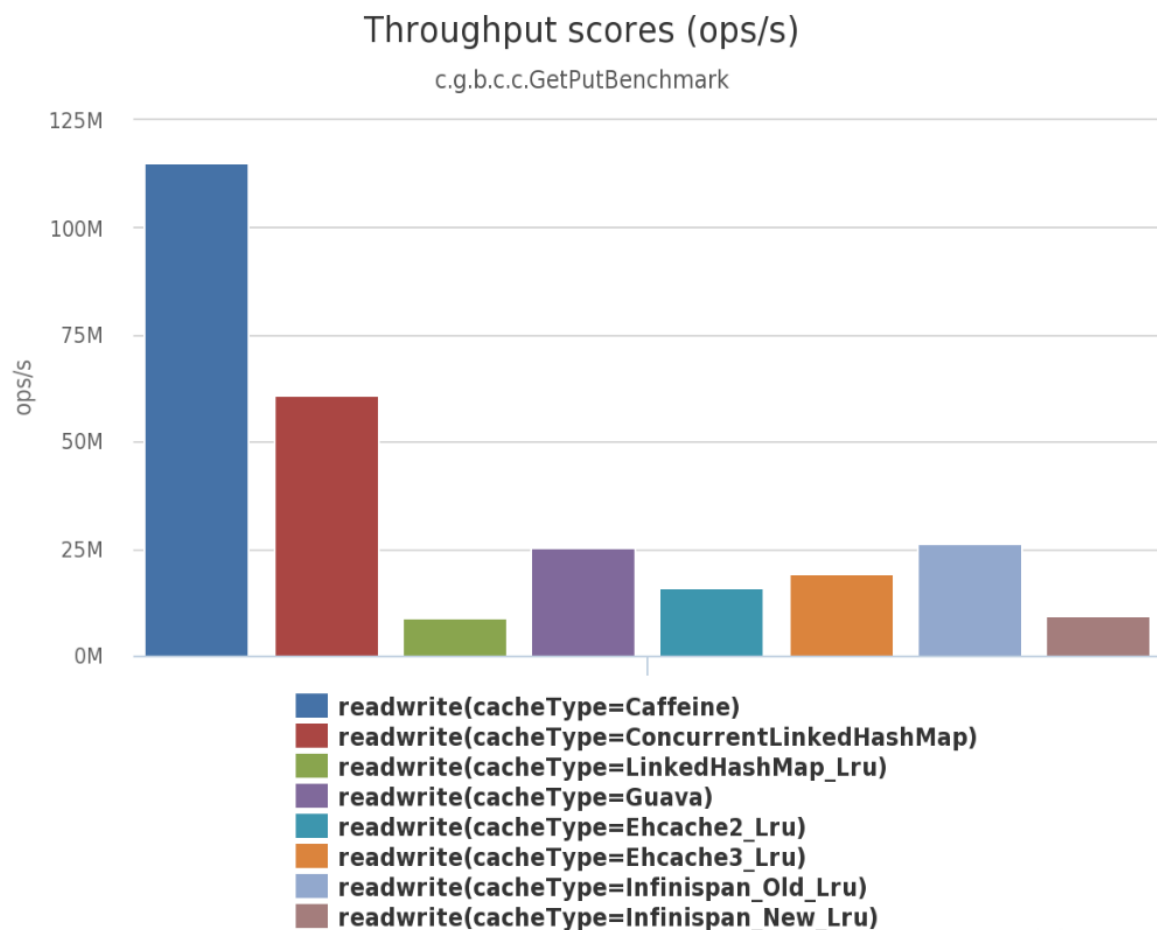
Read (100%)

In this benchmark 8 threads concurrently read from a cache configured with a maximum size.



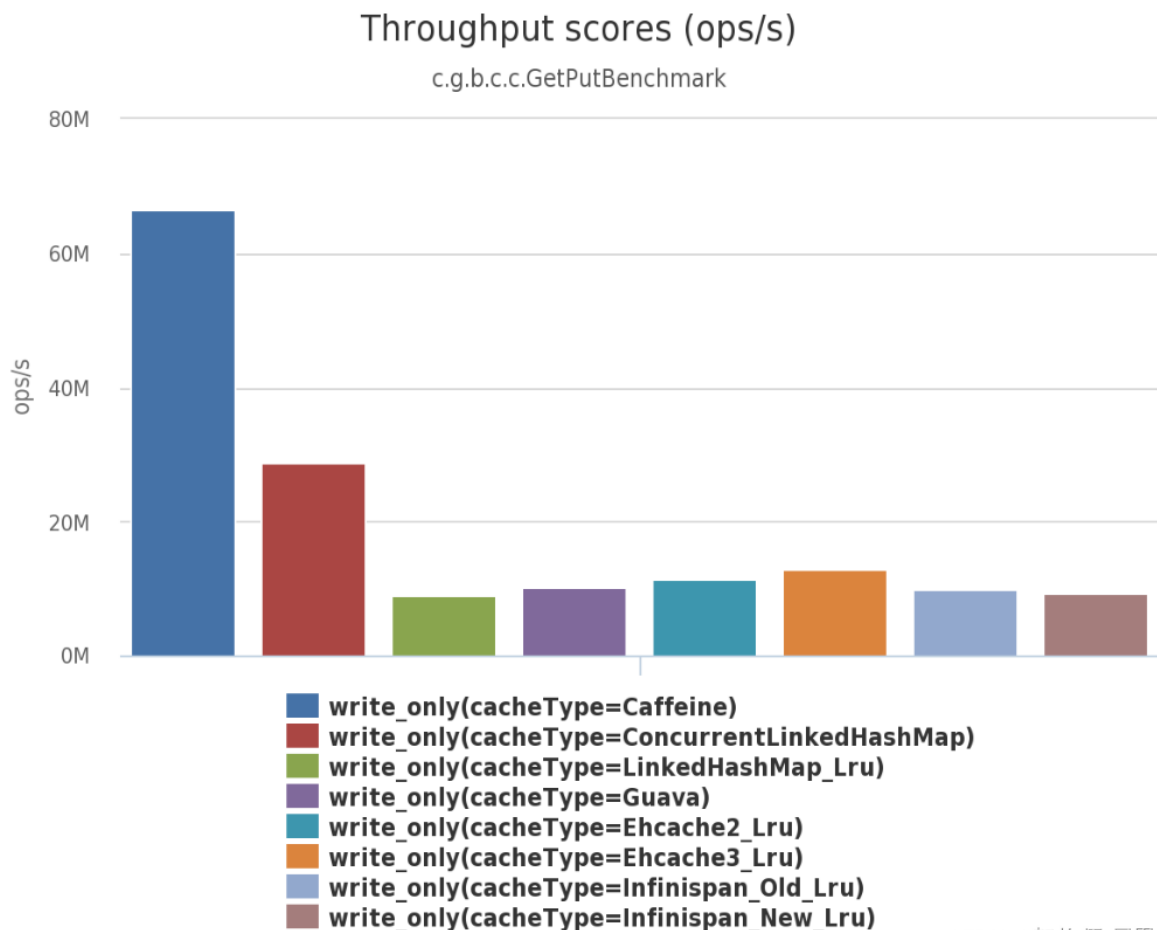
Read (75%) / Write (25%)

In this benchmark 6 threads concurrently read from and 2 threads write to a cache configured with a maximum size.



Write (100%)

In this benchmark 8 threads concurrently write to a cache configured with a maximum size.



Server-class

The benchmarks were run on an Azure G4 instance, the largest available during a free trial from the major cloud providers. The machine was reported as a single socket Xeon E5-2698B v3 @ 2.00GHz (16 core, hyperthreading disabled), 224 GB, Ubuntu 15.04.

Compute

Cache	same key	spread
ConcurrentHashMap	29,679,839	65,726,864
Caffeine	1,581,524,763	530,182,873
Guava	25,132,366	114,608,951

Read (100%)

Unbounded	ops/s (8 threads)	ops/s (16 threads)
ConcurrentHashMap (v8)	560,367,163	1,171,389,095
ConcurrentHashMap (v7)	301,331,240	542,304,172
Bounded		
Caffeine	181,703,298	382,355,194
ConcurrentLinkedHashMap	154,771,582	313,892,223
LinkedHashMap_Lru	9,209,065	13,598,576
Guava (default)	12,434,655	10,647,238
Guava (64)	24,533,922	43,101,468
Ehcache2_Lru	11,252,172	20,750,543
Ehcache3_Lru	11,415,248	17,611,169

Unbounded	ops/s (8 threads)	ops/s (16 threads)
Infinispan_Old_Lru	29,073,439	49,719,833
Infinispan_New_Lru	4,888,027	4,749,506

Read (75%) / Write (25%)

Unbounded	ops/s (8 threads)	ops/s (16 threads)
ConcurrentHashMap (v8)	441,965,711	790,602,730
ConcurrentHashMap (v7)	196,215,481	346,479,582
Bounded		
Caffeine	144,193,725	279,440,749
ConcurrentLinkedHashMap	63,968,369	122,342,605
LinkedHashMap_Lru	8,668,785	12,779,625
Guava (default)	11,782,063	11,886,673
Guava (64)	22,782,431	37,332,090
Ehcache2_Lru	9,472,810	8,471,016
Ehcache3_Lru	10,958,697	17,302,523
Infinispan_Old_Lru	22,663,359	37,270,102
Infinispan_New_Lru	4,753,313	4,885,061

Write (100%)

Unbounded	ops/s (8 threads)	ops/s (16 threads)
ConcurrentHashMap (v8)	60,477,550	50,591,346
ConcurrentHashMap (v7)	46,204,091	36,659,485
Bounded		
Caffeine	55,281,751	48,295,360
ConcurrentLinkedHashMap	23,819,597	39,797,969
LinkedHashMap_Lru	10,179,891	10,859,549
Guava (default)	4,764,056	5,446,282
Guava (64)	8,128,024	7,483,986
Ehcache2_Lru	4,205,936	4,697,745

Ehcache3_Lru Unbounded	10,051,020 ops/s (8 threads)	13,939,317 ops/s (16 threads)
Infinispan_Old_Lru	7,538,859	7,332,973
Infinispan_New_Lru	4,797,502	5,086,305

异步的高性能读写

解决CAS恶性空自旋的有效方式之一是以空间换时间，较为常见的方案有两种：分散操作热点、使用队列削峰。

一般的缓存每次对数据处理完之后（读的话，已经存在则直接返回，不存在则 load 数据，保存，再返回；写的话，则直接插入或更新）

，但是因为要维护一些淘汰策略，则需要一些额外的操作，诸如：

- 计算和比较数据的是否过期
- 统计频率（像 LFU 或其变种）
- 维护 read queue 和 write queue
- 淘汰符合条件的数据
- 等等。。。

这种数据的读写伴随着缓存状态的变更，Guava Cache 的做法是把这些操作和读写操作放在一起，在一个同步加锁的操作中完成，

虽然 Guava Cache 巧妙地利用了 JDK 的 ConcurrentHashMap（分段锁或者无锁 CAS）来降低锁的密度，达到提高并发度的目的。

但是，对于一些热点数据，这种做法还是避免不了频繁的锁竞争。

Caffeine 借鉴了数据库系统的 WAL（Write-Ahead Logging）思想，即：**先写日志，再执行操作**，这种思想同样适合缓存的，

执行读写操作时，先把操作记录在缓冲区，然后在合适的时机异步、批量地执行缓冲区中的内容。

但在执行缓冲区的内容时，也是需要在缓冲区加上同步锁的，不然存在并发问题，

只不过这样就可以把对锁的竞争从缓存数据转移到对缓冲区上。

ReadBuffer 读缓冲

在 Caffeine 的内部实现中，为了更好的支持不同的 Features（如 Eviction，Removal，Refresh，Statistics，Cleanup，Policy 等等），扩展了很多子类，

它们共同的父类是BoundedLocalCache，而readBuffer就是作为它们共有的属性，即都是用一样的readBuffer，

ReadBuffer 读缓冲定义：

```
final Buffer<Node<K, V>> readBuffer;
```

ReadBuffer 读缓冲初始化：

```
readBuffer = evicts() || collectKeys() || collectValues() ||
expiresAfterAccess()
    ? new BoundedBuffer<>()
    : Buffer.disabled();
```

readBuffer 的类型是 BoundedBuffer，它的实现是一个 Striped Ring（条带隔离的 环形）的 buffer 首先考虑到 readBuffer 的特点是多生产者-单消费者（MPSC），所以只需要考虑**写入端**的并发问题。

生产者并行（可能存在竞争）读取计数，检查是否有可用的容量，如果可用，则尝试一下 CAS 写入计数的操作。

如果增量成功，则生产者会懒发布这个元素。

由于 CAS 失败或缓冲区已满而失败时，生产方不会重试或阻塞。

消费者读取计数并获取可用元素，然后清除元素的并懒设置读取计数。

缓冲区分成很多条带（这就是 Striped 的含义）。

如果检测到竞争，则重新哈希、并动态添加新缓冲区来进一步提高并发性，直到一个内部最大值。

当重新哈希发现了可用的缓冲区时，生产者可以重试添加元素以确定是否找到了满足的缓冲区，或者是否有必要调整大小。

具体代码不再列举，一些关键参数：

- 每条 ring buffer 允许 16 个元素（每个元素一个 4 字节的引用）
- 最大允许 $4 * \text{ceilingNextPowerOfTwo}(\text{CPU 数})$ 条 ring buffer

ceilingNextPowerOfTwo 表示向上取 2 的整数幂

触发afterRead

Caffeine 对每次缓存的读操作都会触发afterRead

```
/**
 * Performs the post-processing work required after a read.
 *
 * @param node the entry in the page replacement policy
 * @param now the current time, in nanoseconds
 * @param recordHit if the hit count should be incremented
```

```

*/
void afterRead(Node<K, V> node, long now, boolean recordHit) {
    if (recordHit) {
        statsCounter().recordHits(1);
    }
    //把记录加入到readBuffer
    //判断是否需要立即处理readBuffer
    //注意这里无论offer是否成功都可以走下去的，即允许写入readBuffer丢失，因为这个

    boolean delayable = skipReadBuffer() || (readBuffer.offer(node) !=
Buffer.FULL);
    if (shouldDrainBuffers(delayable)) {
        scheduleDrainBuffers();
    }
    refreshIfNeeded(node, now);
}

/**
 * Returns whether maintenance work is needed.
 *
 * @param delayable if draining the read buffer can be delayed
 */

//caffeine用了一组状态来定义和管理“维护”的过程
boolean shouldDrainBuffers(boolean delayable) {
    switch (drainStatus()) {
        case IDLE:
            return !delayable;
        case REQUIRED:
            return true;
        case PROCESSING_TO_IDLE:
        case PROCESSING_TO_REQUIRED:
            return false;
        default:
            throw new IllegalStateException();
    }
}
}

```

重点看BoundedBuffer

```

/**
 * A striped, non-blocking, bounded buffer.
 *
 * @author ben.manes@gmail.com (Ben Manes)
 * @param <E> the type of elements maintained by this buffer
 */
final class BoundedBuffer<E> extends StripedBuffer<E>

```

它是一个 striped、非阻塞、有界限的 buffer，继承于StripedBuffer类。

下面看看StripedBuffer的实现：

```

/**
 * A base class providing the mechanics for supporting dynamic striping of
 * bounded buffers. This
 * implementation is an adaption of the numeric 64-bit {@link
 * java.util.concurrent.atomic.Striped64}
 * class, which is used by atomic counters. The approach was modified to lazily
 * grow an array of
 * buffers in order to minimize memory usage for caches that are not heavily
 * contended on.
 *
 * @author dl@cs.oswego.edu (Doug Lea)
 * @author ben.manes@gmail.com (Ben Manes)
 */

abstract class StripedBuffer<E> implements Buffer<E>

```

分散操作热点

解决CAS恶性空自旋的有效方式之一是以空间换时间，

较为常见的方案有两种：

- 分散操作热点、
- 使用队列削峰。

这个StripedBuffer设计的思想是跟Striped64类似的，通过扩展结构把**分散操作热点（/竞争热点分离）**。

具体实现是这样的，StripedBuffer维护一个Buffer[]数组，每个元素就是一个RingBuffer，

每个线程用自己threadLocalRandomProbe属性作为 hash 值，这样就相当于每个线程都有自己“专属”的RingBuffer，就不会产生竞争啦，

而不是用 key 的hashCode作为 hash 值，因为会产生热点数据问题。

看看StripedBuffer的属性

```

/** Table of buffers. when non-null, size is a power of 2. */
//RingBuffer数组
transient volatile Buffer<E> @Nullable[] table;

//当进行resize时，需要整个table锁住。tableBusy作为CAS的标记。
static final long TABLE_BUSY =
    UnsafeAccess.objectFieldOffset(StripedBuffer.class, "tableBusy");
static final long PROBE = UnsafeAccess.objectFieldOffset(Thread.class,
    "threadLocalRandomProbe");

/** Number of CPUs. */
static final int NCPU = Runtime.getRuntime().availableProcessors();

/** The bound on the table size. */
//table最大size
static final int MAXIMUM_TABLE_SIZE = 4 * ceilingNextPowerOfTwo(NCPU);

```

```

/** The maximum number of attempts when trying to expand the table. */
//如果发生竞争时（CAS失败）的尝试次数
static final int ATTEMPTS = 3;

/** Table of buffers. When non-null, size is a power of 2. */
//核心数据结构
transient volatile Buffer<E> @Nullable[] table;

/** Spinlock (locked via CAS) used when resizing and/or creating Buffers. */
transient volatile int tableBusy;

/** CASes the tableBusy field from 0 to 1 to acquire lock. */
final boolean casTableBusy() {
    return UnsafeAccess.UNSAFE.compareAndSwapInt(this, TABLE_BUSY, 0, 1);
}

/**
 * Returns the probe value for the current thread. Duplicated from
 * ThreadLocalRandom because of
 * packaging restrictions.
 */
static final int getProbe() {
    return UnsafeAccess.UNSAFE.getInt(Thread.currentThread(), PROBE);
}

/** Table of buffers. When non-null, size is a power of 2. */
//RingBuffer数组
transient volatile Buffer<E> @Nullable[] table; //当进行resize时，需要整个table锁
住。tableBusy作为CAS的标记。static final long TABLE_BUSY =
UnsafeAccess.objectFieldOffset(StripedBuffer.class, "tableBusy");static final
long PROBE = UnsafeAccess.objectFieldOffset(Thread.class,
"threadLocalRandomProbe"); /** Number of CPUs. */static final int NCPU =
Runtime.getRuntime().availableProcessors(); /** The bound on the table size.
*///table最大sizestatic final int MAXIMUM_TABLE_SIZE = 4 *
ceilingNextPowerOfTwo(NCPU); /** The maximum number of attempts when trying to
expand the table. */
//如果发生竞争时（CAS失败）的尝试次数static final int ATTEMPTS =
3; /** Table of buffers. When non-null, size is a power of 2. */
//核心数据结构
transient volatile Buffer<E> @Nullable[] table; /** Spinlock (locked via CAS)
used when resizing and/or creating Buffers. */
transient volatile int tableBusy; /** CASes the tableBusy field from 0 to 1 to
acquire lock. */
final boolean casTableBusy() { return UnsafeAccess.UNSAFE.compareAndSwapInt(this, TABLE_BUSY,
0, 1);} /** * Returns the probe value for the current thread. Duplicated from
ThreadLocalRandom because of * packaging restrictions. */
static final int getProbe() { return UnsafeAccess.UNSAFE.getInt(Thread.currentThread(), PROBE);}

```

offer方法，当没初始化或存在竞争时，则扩容为 2 倍。

实际是调用RingBuffer的 offer 方法，把数据追加到RingBuffer后面。

```

@Override
public int offer(E e) {
    int mask;
    int result = 0;
    Buffer<E> buffer;
    //是否不存在竞争
    boolean uncontended = true;
    Buffer<E>[] buffers = table
    //是否已经初始化

```

```

    if ((buffers == null)
        || (mask = buffers.length - 1) < 0
        //用thread的随机值作为hash值，得到对应位置的RingBuffer
        || (buffer = buffers[getProbe() & mask]) == null
        //检查追加到RingBuffer是否成功
        || !(uncontended = ((result = buffer.offer(e)) != Buffer.FAILED))) {
        //其中一个符合条件则进行扩容
        expandOrRetry(e, uncontended);
    }
    return result;
}

/**
 * Handles cases of updates involving initialization, resizing, creating new
 * Buffers, and/or
 * contention. See above for explanation. This method suffers the usual non-
 * modularity problems of
 * optimistic retry code, relying on rechecked sets of reads.
 *
 * @param e the element to add
 * @param wasUncontended false if CAS failed before call
 */
//这个方法比较长，但思路还是相对清晰的。
@SuppressWarnings("PMD.ConfusingTernary")
final void expandOrRetry(E e, boolean wasUncontended) {
    int h;
    if ((h = getProbe()) == 0) {
        ThreadLocalRandom.current(); // force initialization
        h = getProbe();
        wasUncontended = true;
    }
    boolean collide = false; // True if last slot nonempty
    for (int attempt = 0; attempt < ATTEMPTS; attempt++) {
        Buffer<E>[] buffers;
        Buffer<E> buffer;
        int n;
        if (((buffers = table) != null) && ((n = buffers.length) > 0)) {
            if ((buffer = buffers[(n - 1) & h]) == null) {
                if ((tableBusy == 0) && castTableBusy()) { // Try to attach new Buffer
                    boolean created = false;
                    try { // Recheck under lock
                        Buffer<E>[] rs;
                        int mask, j;
                        if (((rs = table) != null) && ((mask = rs.length) > 0)
                            && (rs[j = (mask - 1) & h] == null)) {
                            rs[j] = create(e);
                            created = true;
                        }
                    } finally {
                        tableBusy = 0;
                    }
                    if (created) {
                        break;
                    }
                }
                continue; // Slot is now non-empty
            }
            collide = false;

```

```

    } else if (!wasUncontended) { // CAS already known to fail
        wasUncontended = true;      // Continue after rehash
    } else if (buffer.offer(e) != Buffer.FAILED) {
        break;
    } else if (n >= MAXIMUM_TABLE_SIZE || table != buffers) {
        collide = false; // At max size or stale
    } else if (!collide) {
        collide = true;
    } else if (tableBusy == 0 && casTableBusy()) {
        try {
            if (table == buffers) { // Expand table unless stale
                table = Arrays.copyOf(buffers, n << 1);
            }
        } finally {
            tableBusy = 0;
        }
        collide = false;
        continue; // Retry with expanded table
    }
    h = advanceProbe(h);
} else if ((tableBusy == 0) && (table == buffers) && casTableBusy()) {
    boolean init = false;
    try { // Initialize table
        if (table == buffers) {
            @SuppressWarnings({"unchecked", "rawtypes"})
            Buffer<E>[] rs = new Buffer[1];
            rs[0] = create(e);
            table = rs;
            init = true;
        }
    } finally {
        tableBusy = 0;
    }
    if (init) {
        break;
    }
}
}
}
}

```

最后看看RingBuffer，注意RingBuffer是BoundedBuffer的内部类。

```

/** The maximum number of elements per buffer. */
static final int BUFFER_SIZE = 16;

// Assume 4-byte references and 64-byte cache line (16 elements per line)
//256长度，但是是以16为单位，所以最多存放16个元素
static final int SPACED_SIZE = BUFFER_SIZE << 4;
static final int SPACED_MASK = SPACED_SIZE - 1;
static final int OFFSET = 16;
//RingBuffer数组
final AtomicReferenceArray<E> buffer;

//插入方法

```

```

@Override
public int offer(E e) {
    long head = readCounter;
    long tail = relaxedWriteCounter();
    //用head和tail来限制个数
    long size = (tail - head);
    if (size >= SPACED_SIZE) {
        return Buffer.FULL;
    }
    //tail追加16
    if (casWriteCounter(tail, tail + OFFSET)) {
        //用tail“取余”得到下标
        int index = (int) (tail & SPACED_MASK);
        //用unsafe.putOrderedObject设值
        buffer.lazySet(index, e);
        return Buffer.SUCCESS;
    }
    //如果CAS失败则返回失败
    return Buffer.FAILED;
}

//用consumer来处理buffer的数据
@Override
public void drainTo(Consumer<E> consumer) {
    long head = readCounter;
    long tail = relaxedWriteCounter();
    //判断数据多少
    long size = (tail - head);
    if (size == 0) {
        return;
    }
    do {
        int index = (int) (head & SPACED_MASK);
        E e = buffer.get(index);
        if (e == null) {
            // not published yet
            break;
        }
        buffer.lazySet(index, null);
        consumer.accept(e);
        //head也跟tail一样，每次递增16
        head += OFFSET;
    } while (head != tail);
    lazySetReadCounter(head);
}

```

注意，ring buffer 的 size（固定是 16 个）是不变的，变的是 head 和 tail 而已。

总的来说ReadBuffer有如下特点：

- 使用 Striped-RingBuffer来提升对 buffer 的读写
- 用 thread 的 hash 来避开热点 key 的竞争
- 允许写入的丢失

WriteBuffer

本来缓存的一般场景是**读多写少**的，读的并发会更高，且 afterRead 显得没那么重要，允许延迟甚至丢失。

writeBuffer跟readBuffer不一样，主要体现在使用场景的不一样。

写不一样，**写afterWrite不允许丢失，且要求尽量马上执行。**

Caffeine 使用MPSC（Multiple Producer / Single Consumer）作为 buffer 数组，

实现在MpscGrowableArrayQueue类，它是仿照JTools的MpscGrowableArrayQueue来写的。

MPSC 允许无锁的高并发写入，但只允许一个消费者，同时也牺牲了部分操作。

TimerWheel

除了支持expireAfterAccess和expireAfterWrite之外（Guava Cache 也支持这两个特性），Caffeine 还支持expireAfter。

因为expireAfterAccess和expireAfterWrite都只能是固定的过期时间，这可能满足不了某些场景，譬如记录的过期时间是需要根据某些条件而不一样的，这就需要用户自定义过期时间。

先看看expireAfter的用法

```
package com.github.benmanes.caffeine.demo;

import com.github.benmanes.caffeine.cache.Cache;
import com.github.benmanes.caffeine.cache.CacheLoader;
import com.github.benmanes.caffeine.cache.Caffeine;
import com.github.benmanes.caffeine.cache.Expiry;
import org.checkerframework.checker.index.qual.NonNegative;
import org.checkerframework.checker.nullness.qual.NonNull;
import org.checkerframework.checker.nullness.qual.Nullable;

import java.util.concurrent.TimeUnit;

public class ExpireAfterDemo {
    static System.Logger logger =
        System.getLogger(ExpireAfterDemo.class.getName());

    public static void hello(String[] args) {
        System.out.println("args = " + args);
    }

    public static void main(String... args) throws Exception {
        Cache<String, String> cache = Caffeine.newBuilder()
            //最大个数限制
            //最大容量1024个，超过会自动清理空间
            .maximumSize(1024)
            //初始化容量
            .initialCapacity(1)
            //访问后过期（包括读和写）
            //5秒没有读写自动删除
            //
            .expireAfterAccess(5, TimeUnit.SECONDS)
            //写后过期
            //
            .expireAfterWrite(2, TimeUnit.HOURS)
            //写后自动异步刷新
    }
}
```

```

//      .refreshAfterWrite(1, TimeUnit.HOURS)
//记录缓存的一些统计数据，例如命中率等
.recordStats()
.removeAllListener(((key, value, cause) -> {
    //清理通知 key,value ==> 键值对    cause ==> 清理原因
    System.out.println("removed key="+ key);
})))
.expireAfter(new Expiry<String, String>() {
    //返回创建后的过期时间
    @Override
    public long expireAfterCreate(@NonNull String key, @NonNull
String value, long currentTime) {
        System.out.println("1. expireAfterCreate key="+ key);
        return 0;
    }

    //返回更新后的过期时间
    @Override
    public long expireAfterUpdate(@NonNull String key, @NonNull
String value, long currentTime, @NonNegative long currentDuration) {
        System.out.println("2. expireAfterUpdate key="+ key);
        return 0;
    }

    //返回读取后的过期时间
    @Override
    public long expireAfterRead(@NonNull String key, @NonNull
String value, long currentTime, @NonNegative long currentDuration) {
        System.out.println("3. expireAfterRead key="+ key);
        return 0;
    }
})
.recordStats()
//使用CacheLoader创建一个LoadingCache
.build(new CacheLoader<String, String>() {
    //同步加载数据
    @Nullable
    @Override
    public String load(@NonNull String key) throws Exception {
        System.out.println("loading key="+ key);
        return "value_" + key;
    }

    //异步加载数据
    @Nullable
    @Override
    public String reload(@NonNull String key, @NonNull String
oldValue) throws Exception {
        System.out.println("reloading key="+ key);
        return "value_" + key;
    }
});

//添加值
cache.put("name", "疯狂创客圈");
cache.put("key", "一个高并发 研究社群");

//获取值

```

```

        @Nullable String value = cache.getIfPresent("name");
        System.out.println("value = " + value);
        //remove
        cache.invalidate("name");
        value = cache.getIfPresent("name");
        System.out.println("value = " + value);
    }
}

```

通过自定义过期时间，使得不同的 key 可以动态的得到不同的过期时间。

注意，我把expireAfterAccess和expireAfterWrite注释了，因为这两个特性不能跟expireAfter一起使用。

而当使用了expireAfter特性后，Caffeine 会启用一种叫“时间轮”的算法来实现这个功能。

为什么要用时间轮

好，重点来了，为什么要用时间轮？

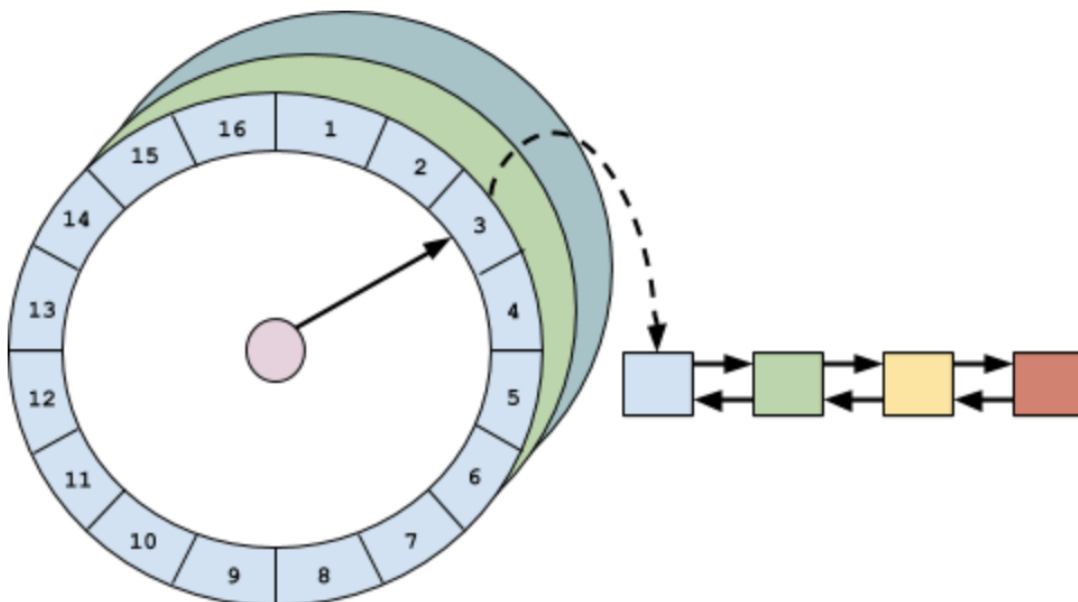
对expireAfterAccess和expireAfterWrite的实现是用一个AccessOrderDeque双端队列，它是 FIFO 的

因为**它们的过期时间是固定的**，所以在队列头的数据肯定是最早过期的，要处理过期数据时，只需要首先看看头部是否过期，然后再挨个检查就可以了。

但是，如果过期时间不一样的话，这需要对accessOrderQueue进行排序&插入，这个代价太大了。

于是，Caffeine 用了一种更加高效、优雅算法-时间轮。

时间轮的结构：



Caffeine 对时间轮的实现在TimerWheel，它是一种多层时间轮（hierarchical timing wheels）。

看看元素加入到时间轮的schedule方法：

```

/**
 * Schedules a timer event for the node.
 *
 * @param node the entry in the cache

```

```

*/
public void schedule(@NonNull Node<K, V> node) {
    Node<K, V> sentinel = findBucket(node.getVariableTime());
    link(sentinel, node);
}

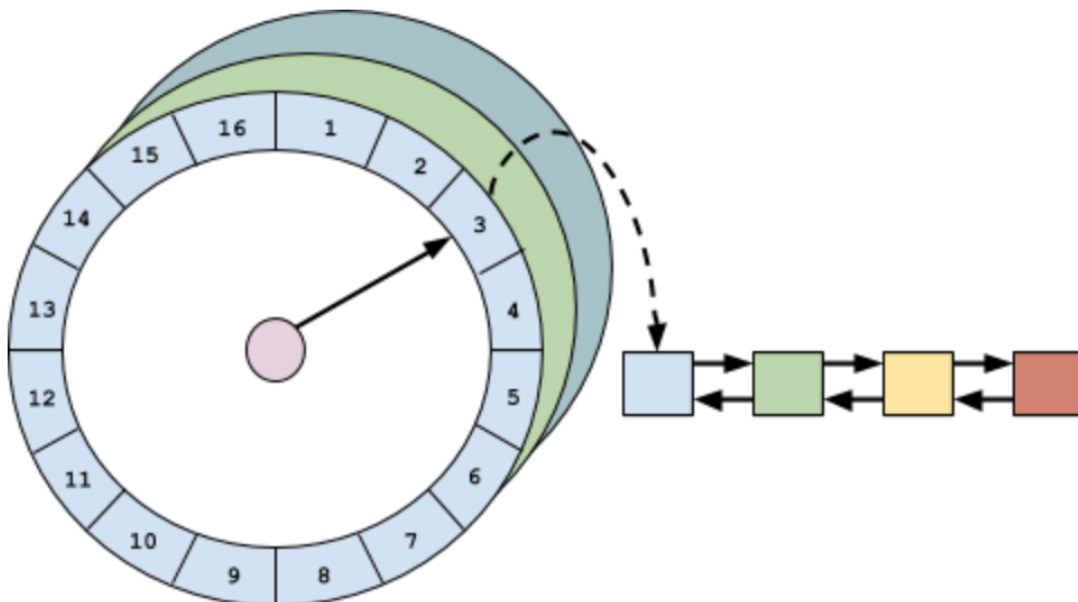
/**
 * Determines the bucket that the timer event should be added to.
 *
 * @param time the time when the event fires
 * @return the sentinel at the head of the bucket
 */
Node<K, V> findBucket(long time) {
    long duration = time - nanos;
    int length = wheel.length - 1;
    for (int i = 0; i < length; i++) {
        if (duration < SPANS[i + 1]) {
            long ticks = (time >>> SHIFT[i]);
            int index = (int) (ticks & (wheel[i].length - 1));
            return wheel[i][index];
        }
    }
    return wheel[length][0];
}

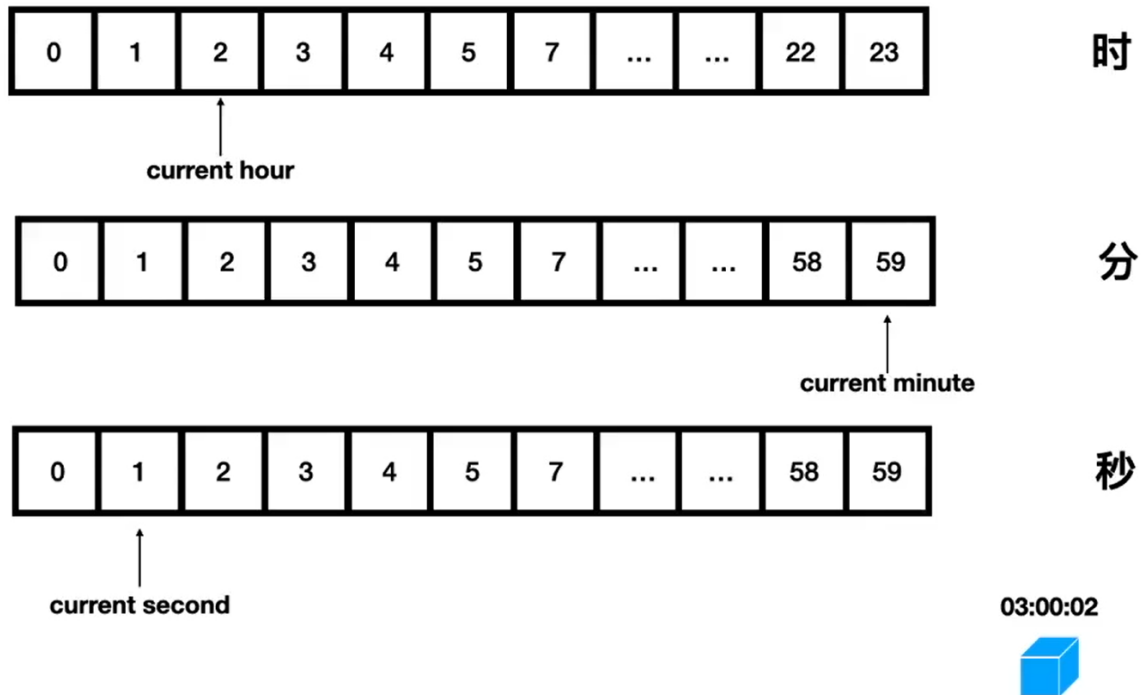
/** Adds the entry at the tail of the bucket's list. */
void link(Node<K, V> sentinel, Node<K, V> node) {
    node.setPreviousInVariableOrder(sentinel.getPreviousInVariableOrder());
    node.setNextInVariableOrder(sentinel);

    sentinel.getPreviousInVariableOrder().setNextInVariableOrder(node);
    sentinel.setPreviousInVariableOrder(node);
}

```

分层时间轮实现定时器





本质就是多个时间轮共同一起作用，分时间层级！

以上述图片为样例，当前时间为2时59分1秒，新建一个3时0分2秒的定时任务，先将定时任务存储在小时单位的时间轮上，存放位置为3时；

然后分层时间轮以秒进行驱动步进，秒驱动到59向0切换时，分钟时间轮也随之步进1，同理小时时间轮；

如果小时时间轮步进到3时，发现该节点上有一个定时任务，那么就将该任务转移到对应的分钟时间轮上，存放位置为0分；同理如果分钟时间轮发现当前的节点中有定时任务，那么就将其转移到秒时间轮上，存放位置为1；秒时间轮发现当前节点有任务，那么就直接执行！

时间复杂度：新增 $O(1)$ ，移除 $O(1)$ ，检测 $O(1)$

说明：本文会持续更新，最新PDF电子版本，请从下面的连接获取：[语雀](#) 或者 [码云](#)

Caffeine 中的宽松读写

Caffeine 对充分利用 volatile 操作花费了很多精力。

内存屏障提供了一种从硬件角度出发的视角，来代替从语言层面思考 volatile 的读写。

通过了解具体哪些屏障被建立以及它们对硬件和数据可视化的影响，将具有实现更好性能的潜力。

当在锁内保证独占访问时，由于锁获取的内存屏障提供了数据可见性，因此 Caffeine 使用宽松读。

在数据竞争无法避免的情况下，比如在读取元素时校验是否过期来模拟缓存丢失，是可以接受的。

和宽松读类似，Caffeine 还使用宽松写。

当一个元素在锁定状态进行排他写，那么写入可以在解锁时释放的内存屏障返回。

这也可以用来支持解决写偏序问题，比如在读取一个数据的时候更新其时间戳。

宽松读写相关的基础知识：VarHandle变量句柄（指针）技术

在JDK9引入了VarHandle，

变量句柄（VarHandle）是对于一个变量的强类型引用，或者是一组参数化定义的变量族，包括了静态字段、非静态字段、数组元素等，

VarHandle支持不同访问模型下对于变量的访问，包括简单的read/write访问，volatile read/write访问，以及CAS访问。

VarHandle相比于传统的对于变量的并发操作具有巨大的优势，

在JDK9引入了VarHandle之后，JUC包中对于变量的访问基本上都使用VarHandle，比如AQS中的CLH队列中使用到的变量等。

VarHandle的作用与优势

在开始讨论VarHandle之前，我们先来回忆一下并发操作里面的三大特性：**原子性、可见性、有序性**。

volatile变量可以保证可见性、有序性（防止指令重拍），

加锁或者原子操作、CAS等可以保证原子性，

只有同时满足这三个特性才能够保证对于一个变量的并发操作是**线程安全的、合乎预期的**。

加锁的话，需要对线程进行同步，而线程上下文切换之间带来的开销是很大的，所以这里不予考虑，考虑一下几种方式：

- 使用**Atomic**包下的原子类进行间接管理，但增加了开销，也可能导致额外的问题如ABA问题；
- 使用原子性的**FieldUpdaters**，利用反射机制，开销也会增大；
- 使用**sun.misc.Unsafe**提供的JVM内置函数，但直接操作JVM可能会损害安全性和可移植性；

针对以上的问题，VarHandle就是用来替代上述方式的一种方案，它提供了一系列标准的内存屏障操作，用于更细粒度的控制指令排序，

在安全性、可用性、性能等方面都要优于现有的AIP，且基本上能够和任何类型的变量相关联。

创建VarHandle

VarHandle通过**MethodHandles**类中的内部类**Lookup**来创建：

```
package com.github.benmanes.caffeine.demo;

import java.lang.invoke.MethodHandles;
import java.lang.invoke.VarHandle;

public class VarHandleTest {
    private String plainStr;    //普通变量
    private static String staticStr;    //静态变量
    private String reflectStr;    //通过反射生成句柄的变量
    private String[] arrayStr = new String[10];    //数组变量
```

```

private static final VarHandle plainVar;    //普通变量句柄
private static final VarHandle staticVar;  //静态变量句柄
private static final VarHandle reflectVar; //反射字段句柄
private static final VarHandle arrayVar;   //数组句柄

static {
    try {
        MethodHandles.Lookup lookup = MethodHandles.lookup();
        plainVar = lookup.findVarHandle(VarHandleTest.class, "plainStr",
String.class);
        staticVar = lookup.findStaticVarHandle(VarHandleTest.class,
"staticStr", String.class);
        reflectVar =
lookup.unreflectVarHandle(VarHandleTest.class.getDeclaredField("reflectStr"));
        arrayVar = MethodHandles.arrayElementVarHandle(String[].class);
    } catch (ReflectiveOperationException e) {
        throw new ExceptionInInitializerError(e);
    }
}
}

```

来分析一下上述创建VarHandle的代码：

1、通过MethodHandles类里面的lookup()函数创建一个Lookup类，

这个Lookup类是MethodHandles的内部类，作用就是用于创建方法句柄和变量句柄的一个工厂类，源码如下：

```

@CallerSensitive
@ForceInline // to ensure Reflection.getCallerClass optimization
public static Lookup lookup() {
    return new Lookup(Reflection.getCallerClass());
}

```

2、通过Lookup里面的工厂方法生成不同类型的VarHandle，

拿生成普通变量的findVarHandle方法来看：

```

/**
 * Produces a VarHandle giving access to a non-static field {@code name}
 * of type {@code type} declared in a class of type {@code recv}.
 * The VarHandle's variable type is {@code type} and it has one
 * coordinate type, {@code recv}.
 * <p>
 * Access checking is performed immediately on behalf of the lookup
 * class.
 * <p>
 * Certain access modes of the returned VarHandle are unsupported under
 * the following conditions:
 * <ul>
 * <li>if the field is declared {@code final}, then the write, atomic
 *     update, numeric atomic update, and bitwise atomic update access
 *     modes are unsupported.
 * <li>if the field type is anything other than {@code byte},

```

```

*      {@code short}, {@code char}, {@code int}, {@code long},
*      {@code float}, or {@code double} then numeric atomic update
*      access modes are unsupported.
* <li>if the field type is anything other than {@code boolean},
*      {@code byte}, {@code short}, {@code char}, {@code int} or
*      {@code long} then bitwise atomic update access modes are
*      unsupported.
* </li>
* <p>
* If the field is declared {@code volatile} then the returned VarHandle
* will override access to the field (effectively ignore the
* {@code volatile} declaration) in accordance to its specified
* access modes.
* <p>
* If the field type is {@code float} or {@code double} then numeric
* and atomic update access modes compare values using their bitwise
* representation (see {@link Float#floatToRawIntBits} and
* {@link Double#doubleToRawLongBits}, respectively).
* @apiNote
* Bitwise comparison of {@code float} values or {@code double} values,
* as performed by the numeric and atomic update access modes, differ
* from the primitive {@code ==} operator and the {@link Float#equals}
* and {@link Double#equals} methods, specifically with respect to
* comparing NaN values or comparing {@code -0.0} with {@code +0.0}.
* Care should be taken when performing a compare and set or a compare
* and exchange operation with such values since the operation may
* unexpectedly fail.
* There are many possible NaN values that are considered to be
* {@code NaN} in Java, although no IEEE 754 floating-point operation
* provided by Java can distinguish between them. Operation failure can
* occur if the expected or witness value is a NaN value and it is
* transformed (perhaps in a platform specific manner) into another NaN
* value, and thus has a different bitwise representation (see
* {@link Float#intBitsToFloat} or {@link Double#longBitsToDouble} for
more
* details).
* The values {@code -0.0} and {@code +0.0} have different bitwise
* representations but are considered equal when using the primitive
* {@code ==} operator. Operation failure can occur if, for example, a
* numeric algorithm computes an expected value to be say {@code -0.0}
* and previously computed the witness value to be say {@code +0.0}.
* @param recv the receiver class, of type {@code R}, that declares the
* non-static field
* @param name the field's name
* @param type the field's type, of type {@code T}
* @return a VarHandle giving access to non-static fields.
* @throws NoSuchFieldException if the field does not exist
* @throws IllegalAccessException if access checking fails, or if the
field is {@code static}
* @exception SecurityException if a security manager is present and it
*
refuses access
* @throws NullPointerException if any argument is null
* @since 9
*/
public VarHandle findVarHandle(Class<?> recv, String name, Class<?>
type) throws NoSuchFieldException, IllegalAccessException {
    MemberName getField = resolveOrFail(REF_getField, recv, name, type);

```



```

        MemberName putField = resolveOrFail(REF_putField, recv, name, type);
        return getFieldVarHandle(REF_getField, REF_putField, recv, getField,
putField);
    }

```

代码很简单，就是根据传入的参数来生成字段的访问对象MemberName，

MemberName是用来描述一个方法或字段引用的数据结构，再根据MemberName生成句柄，熟悉JVM的同学看到REF_getField应该能够联想到JVM字节码，这个就是对应着访问字段的字节码。

对于findStaticVarHandle、unreflectVarHandle方法的实现其实也很类似，大致就是将REF_getField改为REF_getStatic的过程。

VarHandle的访问

一开始我们就讲过，VarHandle支持不同访问模型下对于变量的访问，

包括简单的read/write访问，volatile read/write访问，以及CAS访问等，

那么分别来看一下VarHandle是如何支持这些访问模型下对于变量的访问的。

1、简单的read/write访问

```

public void plainUse(String[] args) {
    plainVar.set(this, "I am a plain string");    //实例变量的普通write操作
    staticVar.set("I am a static string");        //    静态变量的普通write操作
    reflectVar.set(this, "I am a string created by reflection");    //反射字
段的普通write操作
    arrayVar.set(arrayStr, 0, "I am a string array element");    //数组变量的
普通write操作

    String plainString = (String) plainVar.get(this);    //实例变量的普通read操
作
    String staticString = (String) staticVar.get();    //    静态变量的普通
read操作
    String reflectString = (String) staticVar.get(this);    //反射字段的普通
read操作
    String arrayStrElem = (String) arrayVar.get(arrayStr, 0);    //数组变量的
普通read操作， 第二个参数是指数组下标，即第0个元素
}

```

2、volatile read/write访问

对于不同类型的变量的访问方法跟上面其实大同小异，下面就以普通变量来举例：

```

public void volatileUse(String[] args) {
    volatileVar.setVolatile(this, "I am volatile string");    //volatile
write
    String volatileString = (String) volatileVar.getVolatile(this);
    //volatile read
}

```

3、CAS访问

```

public void casUse(String[] args) {
    String casString = (String) plainVar.get(this);    //先读取当前值作为cas的
预期值
    plainVar.compareAndSet(this, casString, "I am a new cas string");    //
第二个参数为预期值，第三个参数为修改值
}

```

VarHandle中的指令重排序影响

VarHandle中定义了多种不同的访问模式，定义在VarHandle内部的枚举类里面：

```

enum AccessType {
    GET(Object.class),
    SET(void.class),
    COMPARE_AND_SET(boolean.class),
    COMPARE_AND_EXCHANGE(Object.class),
    GET_AND_UPDATE(Object.class);

    final Class<?> returnType;
    final boolean isMonomorphicInReturnType;
}

```

上面只是VarHandle中定义的访问模式中的一小部分，实际上还有很多，

```

/**
 * The set of access modes that specify how a variable, referenced by a
 * VarHandle, is accessed.
 */
public enum AccessMode {
    /**
     * The access mode whose access is specified by the corresponding
     * method
     * {@link VarHandle#get VarHandle.get}
     */
    GET("get", AccessType.GET),
    /**
     * The access mode whose access is specified by the corresponding
     * method
     * {@link VarHandle#set VarHandle.set}
     */
    SET("set", AccessType.SET),
    /**
     * The access mode whose access is specified by the corresponding
     * method
     * {@link VarHandle#getVolatile VarHandle.getVolatile}
     */
    GET_VOLATILE("getVolatile", AccessType.GET),
    /**
     * The access mode whose access is specified by the corresponding
     * method
     * {@link VarHandle#setVolatile VarHandle.setVolatile}
     */
    SET_VOLATILE("setVolatile", AccessType.SET),
}

```

```
* The access mode whose access is specified by the corresponding
* method
* {@link VarHandle#getAcquire VarHandle.getAcquire}
*/
```

不同的访问模式对于指令重排的效果都可能不一样，因此需要慎重选择访问模式。

其次，在创建VarHandle的过程中，VarHandle内部的访问模式会覆盖变量声明时的任何指令排序效果。

比如说一个变量被声明为volatile类型，但是调用varHandle.get()方法时，其访问模式就是get模式，即简单读取方法，因此需要多加注意。

参考文献

1. 疯狂创客圈JAVA 高并发 总目录

ThreadLocal（史上最全）

<https://www.cnblogs.com/crazymakercircle/p/14491965.html>

2. 3000页《尼恩 Java 面试宝典》的 35个面试专题：

<https://www.cnblogs.com/crazymakercircle/p/13917138.html>

3. 价值10W的架构师知识图谱

<https://www.processon.com/view/link/60fb9421637689719d246739>

4、尼恩 架构师哲学

<https://www.processon.com/view/link/616f801963768961e9d9aec8>

5、尼恩 3高架构知识宇宙

<https://www.processon.com/view/link/635097d2e0b34d40be778ab4>

Guava Cache主页：<https://github.com/google/guava/wiki/CachesExplained>

Caffeine的官网：<https://github.com/ben-manes/caffeine/wiki/Benchmarks>

<https://gitee.com/jd-platform-opensource/hotkey>

https://developer.aliyun.com/article/788271?utm_content=m_1000291945

<https://b.alipay.com/page/account-manage-oc/approval/setList>

Caffeine: <https://github.com/ben-manes/caffeine>

这里: <https://albenw.github.io/posts/df42dc84/>

Benchmarks: <https://github.com/ben-manes/caffeine/wiki/Benchmarks>

官方API说明文档: <https://github.com/ben-manes/caffeine/wiki>

这里: <https://github.com/ben-manes/caffeine/wiki/Guava>

HashedWheelTimer时间轮原理分析: <https://albenw.github.io/posts/ec8df8c/>

TinyLFU论文: <https://arxiv.org/abs/1512.00727>

Design Of A Modern Cache: <http://highscalability.com/blog/2016/1/25/design-of-a-modern-cache.html>

Design Of A Modern Cache—Part Deux: <http://highscalability.com/blog/2019/2/25/design-of-a-modern-cache-part-deux.html>

Caffeine的github: <https://github.com/ben-manes/caffeine>

<https://github.com/axinSoochow/redis-caffeine-cache-starter>

<https://www.cnblogs.com/liang24/p/14210542.html>

<https://www.jianshu.com/p/62757d2a592c>

<https://blog.csdn.net/tongkongyu/article/details/124842847>

<https://www.163.com/dy/article/FSC51E7G0511FQO9.html>

<https://blog.csdn.net/Hellowenpan/article/details/121264731>

<https://www.jianshu.com/p/3c6161e5337b>

<https://blog.csdn.net/darin1997/article/details/89397862>

https://blog.csdn.net/weixin_42297591/article/details/112658262

<https://www.cnblogs.com/liujinhua306/p/9808500.html>

<https://blog.csdn.net/varyall/article/details/81172725>

<http://www.cnblogs.com/zhaoxinshanwei/p/8519717.html>

https://segmentfault.com/a/1190000016091569?utm_source=tag-newest

<https://www.javadevjournal.com/spring-boot/spring-boot-with-caffeine-cache/>

<https://sunitc.dev/2020/08/27/springboot-implement-caffeine-cache/>

<https://github.com/ben-manes/caffeine/wiki/Population-zh-CN>

<https://www.cnblogs.com/Mufasa/p/15994714.html>

<https://www.cnblogs.com/liujinhua306/p/9808500.html>

硬核推荐：尼恩Java硬核架构班

又名疯狂创客圈社群 VIP

详情：<https://www.cnblogs.com/crazymakercircle/p/9904544.html>



The banner features a dark red background with a subtle pattern of small white triangles. At the top, two golden circular ornaments with the Chinese character '福' (blessing) are positioned on either side of the main title. The title '尼恩java 硬核架构班' is written in large, bold, golden characters. Below the title, the text '已经发布' (Already Released) is centered in a golden font. A list of course topics is presented on the left side, each preceded by a golden star icon. The topics are written in white and yellow text. The list includes:

- 《高能性能RPC的基础实操之：从0到1开始IM撸一个IM》
- 《分布式高能性能RPC的基础实操之：千万级用户分布式IM实操-含简历指导》
- 《亿级用户超高并发秒杀实操-含简历指导》
- 亮点：助力小伙伴搞定70W年薪，N个涨薪50%，2023春招面试涨薪神器
- 《横扫全网，工业级elasticsearch底层原理与高并发、高可用架构实操》
- 亮点：40岁老架构师细致解读，处处透着分布式、高性能中间件的原理和精髓
- 《第1部曲：超级底层：葵花宝典（高性能秘籍）__架构师视角解读OS操作系统》
- 亮点：大制作解读OS操作系统，并揭秘mmap、pagecache、zerocopy等底层的底层原理
- 2023春招面试涨薪大神器
- 《Rocketmq视频第2部曲：横扫全网工业级 rocketmq 高可用（HA）底层原理和实操》
- 亮点：起底式、绞杀式解读 rocketmq如何保障消息的可靠性？
- 《Rocketmq视频第3部曲：超级内功篇、横扫全网 rocketmq 源码学习以及3高架构模式解读》
- 亮点：大制作解读 Rocketmq源码以及3高架构模式，助力大家内力猛增
- 《Rocketmq视频第4部曲：10Wqps消息推送中台架构、设计、编码、测试实操》
- 亮点：Netty实操、分库分表实操、Rocketmq工业级使用实操
- 《架构师内功篇：横扫全网 netty 高性能、高并发架构 底层原理、源码学习》
- 《架构师实操篇：redis cluster 工业级高可用实操》
- 《架构师实操篇：100W级别QPS日志平台实操》
- 《彻底穿透：skywalking 源码(代表链路跟踪)+Java agent+bytebuddy 探针》

规划中

《架构师实操篇：基于netty 手写 rpc 框架- 参考 dubbo、seata rpc框架》

《架构师实操篇：go语言学习，以及基于 go 手写 rpc 框架》

《架构师实操篇：千万级任务调度平台 架构与实操- 基于尼恩17年的亿级搜索项目》

《架构师实操篇：工业级 亿级文档搜索 平台 架构与实操- 基于尼恩17年的亿级搜索项目》

特色

会员制

提供技术方向指导，
职业生涯指导，少躺坑，少弯路

简历指导

这个很重要，
对于挪窝涨薪来说

实操性

以上项目，都是老架构师
在生产上实操过的项目

非水货

40岁老架构师，不是水货架构师
《Java高并发三部曲》为证

架构班（社群 VIP）的起源：

最初的视频，主要是给读者加餐。很多的读者，需要一些高质量的实操、理论视频，所以，我就围绕书，和底层，做了几个实操、理论视频，然后效果还不错，后面就做成迭代模式了。

架构班（社群 VIP）的功能：

提供高质量实操项目整刀真枪的架构指导、快速提升大家的：

- 开发水平
- 设计水平
- 架构水平

弥补业务中 CRUD 开发短板，帮助大家尽早脱离具备 3 高能力，掌握：

- 高性能
- 高并发
- 高可用

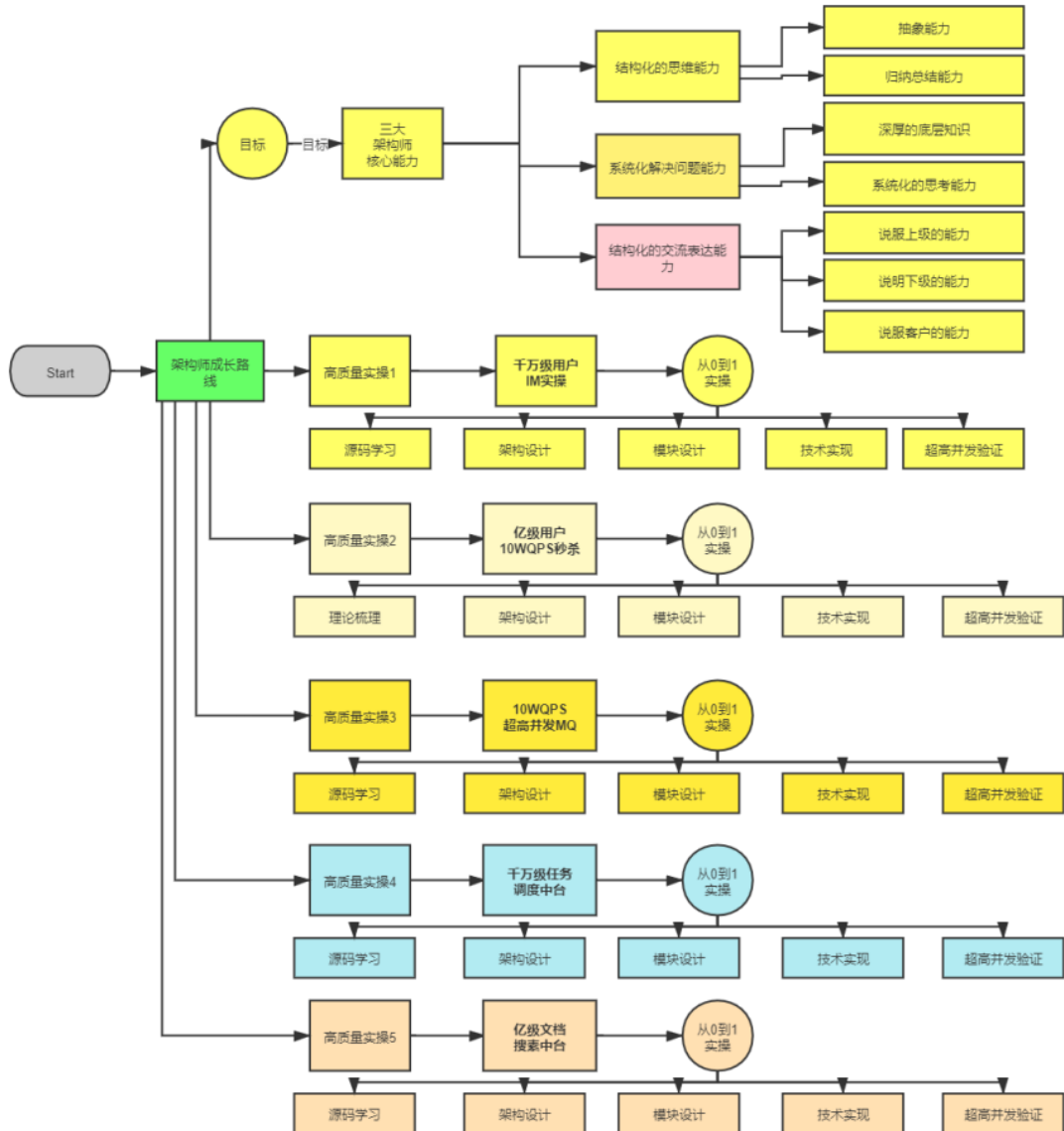
作为一个高质量的架构师成长、人脉社群，把所有的卷王聚焦起来，一起卷：

- 卷高并发实操
- 卷底层原理
- 卷架构理论、架构哲学
- 最终成为顶级架构师，实现人生理想，走向人生巅峰

架构班（社群 VIP）的目的：

- 高质量的实操，大大提升简历的含金量，吸引力，增强面试的召唤率
- 为大家提供九阳真经、葵花宝典，快速提升水平
- 进大厂、拿高薪
- 一路陪伴，提供助学视频和指导，辅导大家成为架构师
- 自学为主，和其他卷王一起，卷高并发实操，卷底层原理、卷大厂面试题，争取狠卷 3 月成高手，狠卷 3 年成为顶级架构师

N 个超高并发实操项目：简历压轴、个顶个精彩



【样章】第 17 章:横扫全网Rocketmq 视频第 2 部曲: 工业级 rocketmq 高可用(HA) 底层原理和实操

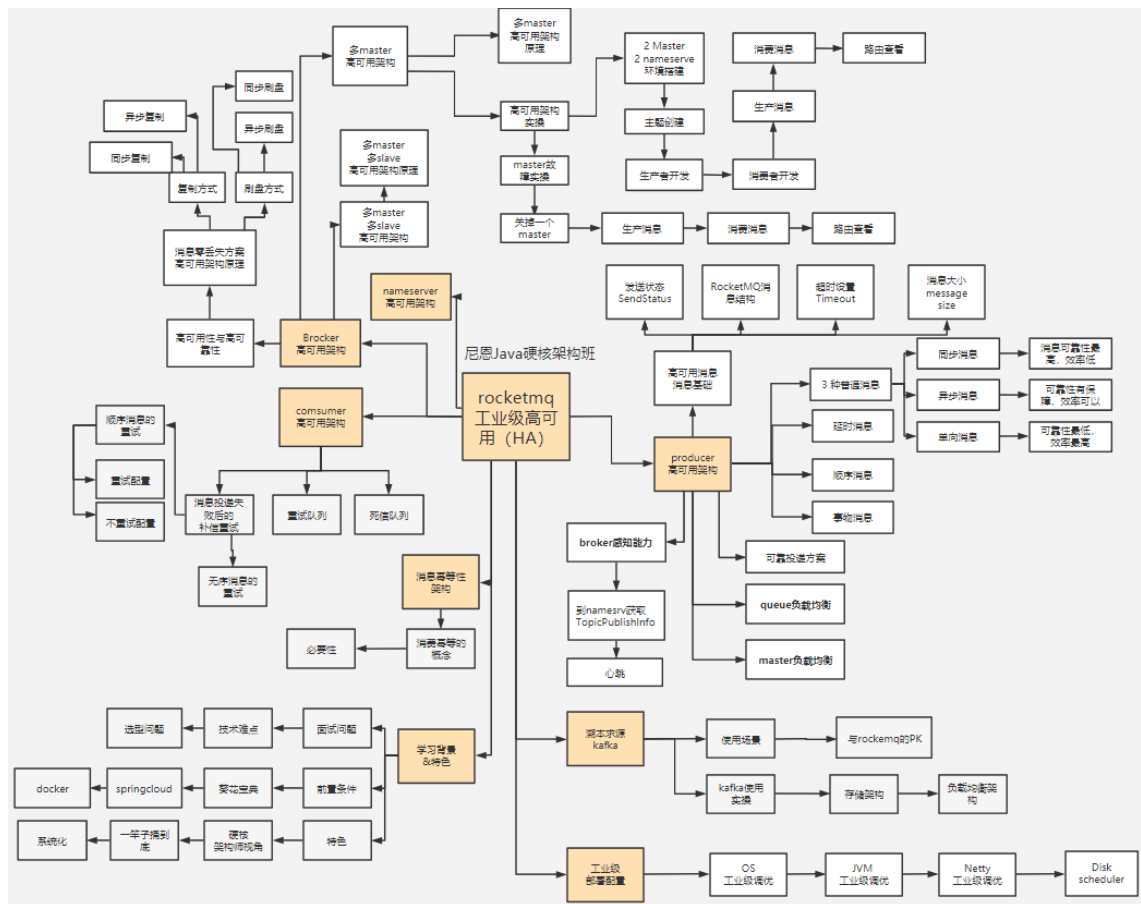
工业级 rocketmq 高可用底层原理, 包含: 消息消费、同步消息、异步消息、单向消息等不同消息的底层原理和源码实现; 消息队列非常底层的主从复制、高可用、同步刷盘、异步刷盘等底层原理。

工业级 rocketmq 高可用底层原理和搭建实操, 包含: 高可用集群的搭建。

解决以下难题:

- 1、技术难题: RocketMQ 如何最大限度的保证消息不丢失的呢? RocketMQ 消息如何做到高可靠投递?
- 2、技术难题: 基于消息的分布式事务, 核心原理不理解
- 3、选型难题: kafka or rocketmq , 该娶谁?

下图链接: <https://www.proceson.com/view/6178e8ae0e3e7416bde9da19>



成功案例：2 年翻 3 倍，35 岁卷王成功转型为架构师

详情: <http://topcoder.cloud/forum.php?mod=forumdisplay&fid=43&page=1>

[最新](#) [最后发表](#) [热门](#) [精华](#)

☐ 成功案例: [1057号卷王] 3年小伙拿到外企offer, 薪酬涨了200%

① 卷王1号 超级版主 前天 17:41

☐ 成功案例: [645号卷王] 4年经验卷王逆袭, 被毕业后, 反涨24W

① 卷王1号 超级版主 2022-9-21

☐ 成功案例: [878号卷王] 小伙8年经验, 年薪60W

① 卷王1号 超级版主 2022-8-13

☐ 年薪70W案例: 通过尼恩的指导, 小伙伴年薪从40W涨到70W

① 卷王1号 超级版主 2022-2-11

☐ 成功案例: [493号卷王] 5年小伙拿满意offer, 就业寒冬季逆涨30%

① 卷王1号 超级版主 前天 17:43

☐ 成功案例: [250号卷王] 就业极寒时代, 收offer 涨25%

① 卷王1号 超级版主 前天 17:38

☐ 成功案例: [612号卷王] 就业极寒时代, 从外包到自研

① 卷王1号 超级版主 前天 17:15

☐ 成功案例: [913号卷王] 热烈祝贺6年经验卷王, 年薪40W

① 卷王1号 超级版主 2022-9-21

☐ 成功案例: [959号卷王] 4年经验卷王, 喜获百度、Boss直聘等N个优质offer, 最高涨100%

① 卷王1号 超级版主 2022-9-21

☐ 成功案例: [529号卷王] 5年经验卷王喜收2大offer, 最高涨5K

① 卷王1号 超级版主 2022-9-21

☐ 成功案例: [811号卷王] 热烈祝贺7年经验卷王, 薪酬涨30%

① 卷王1号 超级版主 2022-9-21

☐ 成功案例: [287号卷王] 不惧大寒潮, 卷王逆市收4 offer, 涨30%, 可喜可贺

① 卷王1号 超级版主 2022-5-30

☐ 成功案例: [1002号卷王] 5月份“被毕业”, 改简历后, 斩获顶级央企Offer, 涨薪7000+

① 卷王1号 超级版主 2022-7-5

成功案例: [7号卷王] 热烈祝贺小伙伴涨薪120%

1 卷王1号 超级版主 2022-8-13

成功案例: [134号卷王] 大三小伙卷1年, 斩获顶级央企Offer, 成功逆袭

1 卷王1号 超级版主 2022-7-6

成功案例: [1008号卷王] 5年经验卷王收42W offer, 月涨8000, 可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例: [453号卷王] 非全日制 6年卷王喜提3 offer, 年薪30W, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [924号卷王] 6年卷王喜提4 offer, 最高涨薪9000, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [15号卷王] 4年卷王入职 微软, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-12

成功案例: [527号卷王] 4年卷王喜提2 offer, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-13

成功案例: [788号卷王] 3年卷王喜提优质Offer, 涨薪60%

1 卷王1号 超级版主 2022-5-11

成功案例: 热烈祝贺: 非全日制卷王, 喜提2个心仪offer, 面3家过2家

1 卷王1号 超级版主 2022-4-21

成功案例: [693号卷王] 二线城市6年卷王喜提4大优质Offer, 含央企offer, 最高薪酬35W

1 卷王1号 超级版主 2022-4-16

成功案例: [85号卷王] 双非2本小伙, 春招大捷, 喜提9个offer, 最高薪酬近30万

1 卷王1号 超级版主 2022-4-14

成功案例: [741号卷王] 卷王逆袭! 6年小伙从很少面试机会到搞定35K*14薪Offer

1 卷王1号 超级版主 2022-4-12

成功案例: [642号卷王] 热烈祝贺, 6年卷王喜提优质国企offer

1 卷王1号 超级版主 2022-4-7

成功案例: [796号卷王] 热烈祝贺, 36岁卷王喜提52万优质offer

1 卷王1号 超级版主 2022-3-25

成功案列: [15号卷王] 小伙卷1年, 涨薪9K+, 喜收ebay等多个优质offer

1 卷王1号 超级版主 2022-3-24

成功案列: [821号卷王] 小伙狠卷3个月, 喜提10多个offer

1 卷王1号 超级版主 2022-3-21

成功案列: [736号卷王] 3年半经验收22k offer, 但是小伙志存高远, 冲击25k+

1 卷王1号 超级版主 2022-3-20

成功案列: 热烈祝贺1群小卷王offer拿到手软, 甚至拒了阿里offer

1 卷王1号 超级版主 2022-3-16

简历案列: 简历一改, 腾讯的邀请就来了! 热烈祝贺, 小伙收到一大堆面试邀请

1 卷王1号 超级版主 2022-3-10

成功案列: 祝贺我国两大超级卷王, 一个过了阿里HR面, 一个过了阿里2面

1 卷王1号 超级版主 2022-3-10

成功案列: 小伙伴php转Java, 卷1.5年Java, 涨薪50%, 喜收多个优质offer

1 卷王1号 超级版主 2022-3-10

成功案列: 4年小伙狠卷半年, 拿到 移动、京东 两大顶级offer

尼恩 超级版主 2022-3-5

成功案列: [267号卷王] 助力3年经验卷王, 拿到蜂巢的17k x 14薪的offer

1 卷王1号 超级版主 2022-2-27

成功案列: [143号卷王] 二本院校00后卷神, 毕业没到一年跳到字节, 年薪45W

1 卷王1号 超级版主 2022-2-27

成功案列: [494号卷王] 尼恩分布式事务助力卷王拿到 中信银行offer

1 卷王1号 超级版主 2022-2-27

成功案列: [76号卷王] 2线城市卷王, 狠卷1.5年, 喜收22K offer

1 卷王1号 超级版主 2022-2-27

成功案列: [429号卷王] 小伙伴在社群卷5个月, 涨8k+

1 卷王1号 超级版主 2022-2-27

成功案列: [154号卷王] 双非学校毕业卷王, 连拿 京东到家&滴滴 两个大厂Offer

1 卷王1号 超级版主 2022-2-27

成功案例: [232号卷王] 涨薪10K, 继续卷向食物链顶端

1 卷王1号 超级版主 2022-2-27

成功案例: 狠卷1年技术, 喜收 腾讯、阿里、微软三大Offer, 最高年薪56W

1 卷王1号 超级版主 2022-2-27

成功案例: [449号卷王] 应届毕业卷王喜收 滴滴offer, 年薪33W

1 卷王1号 超级版主 2022-2-27

成功案例: [551号卷王] 小伙伴学完后, 成功进入大厂, 并且推荐自己的朋友加VIP学习

1 卷王1号 超级版主 2022-2-10

成功案例: [214号卷王] 助力2年经验卷王, 成功拿到17K月薪

1 卷王1号 超级版主 2022-2-10

成功案例: [92号卷王] 课程实操助力社群小伙伴喜收 喜马拉雅Offer

1 卷王1号 超级版主 2022-2-10

成功案例: 社群卷王小伙伴成功过了滴滴三面 获滴滴Offer

1 卷王1号 超级版主 2022-2-10

成功案例: [612号卷王]滴滴小伙伴, 蹲点考察半年, 觉得靠谱后加入 疯狂创客圈

1 卷王1号 超级版主 2022-2-10

成功案例: [732号卷王] 尼恩助力3年经验卷王收获 京东offer, 年薪35W

1 卷王1号 超级版主 2022-2-27

成功案例: [558号卷王] 2年经验卷王, 喜收 网易和阿里子公司两个优质offer

1 卷王1号 超级版主 2022-2-27

成功案例: [569号卷王] 双非应届生卷王, 喜收字节跳动实习offer

1 卷王1号 超级版主 2022-2-25

成功案例: [420号卷王] 狠卷1年, 卷王涨薪80%, 涨薪12000元!

1 卷王1号 超级版主 2022-2-25

成功案例: [76号卷王] 通过尼恩1年半的指导, 专科学历小伙伴从0.8K涨到22K

1 卷王1号 超级版主 2022-2-10

简历优化后的成功涨薪案例（VIP 含免费简历优化）

9年小伙伴拿到年薪90W offer

9月11日改简历 11月29日晒offer

秘诀:
简历指导+ 狠狠卷

薪资高 稳

这个是你微调的

主要的工作是啥?

省路号的地方, 需要你再补充一点

提升了自己的能力, 就不用怕

9月11日 下午17:38

易所 关于数字货币的

上面你留着这个就行

也有打盹的时候, 该裁员, 照样一个不少

年包比 多19w

这么多

估计有 70 万

那你不有 90 个 W?

po 给我 68

就是吗

Java 开发 9 年-修改.docx 34.1 KB

小伙8年经验 年薪60w

7月12日改简历 8月10日晒offer

秘诀:
改简历+ 狠狠卷

涨幅多少呀?

或者, 幅度多少呀?

之前 36*15, 现在这个 39*15

今年行情不太好, 还有一些 offer 基本都是平衡, 没降薪的。

OK

这个马上来

刚在指导简历

哈哈

恭喜呀

这次找工作, 您的简历调整起到效果了。

我这次练习基本看的都是咱们课程的 简历。

OK

那就上午 11 点左右吧

好的

6年小伙伴 年薪40w

9月6日改简历 9月21日晒offer

秘诀:
简历指导+ 狠狠卷

张这多

今年这行情, 也算可以了

总包多少呀, 让我也围观一下

大概 40w 吧

谢谢恩师的指导和鼓励

是在深圳

深圳的行情尤其难

能有面试电话就不错了

是的

太牛啦

哈哈哈哈哈, 恩师的鼓励指导也很重要

给你前面调整了一下

简历 - Java - 6 年.docx 24.7 KB

简历 - Java - 6 年(1).docx 24.5 KB

5年小伙喜提3个offer 年薪 35个W

5月22日改简历 11月29日晒offer

秘诀:
简历指导+ 狠狠卷

恩恩老师晚上好, 汇报下那说的 offer 情况, 最近面试收到了三个 offer, 两个是年薪, 一个是跨境电商公司的 offer, 涨幅暂时不到 20%, 通过这三次面试也让我知道自己距离高级开发还有一点点距离, 还要再多卷才能突破。

最后结合自己的情况, 先选择去跨境电商的公司再往下

之前是 20k*13.5, 跨境电商这个是 23k* (14-15)

恭喜恭喜

3 个 offer

就业寒冬, 能拿到 3 个 offer, 已经很不错了, 已经是高手了

很多小伙伴, 面试电话一个都不接到, 简历海投 7000 份, 只收到 3 次面试机会, 没有一个机会拿到最终 offer

您这个年薪, 算下来也有 35W 了吧

年终奖部分还要确认下, 大部分人听说只有两个月

这个时间点, 拿到这个水平, 挺不错的啦

持续加油卷哈

嘿嘿! 看看明年自己有没有能力冲击离开

把你视频都吃透

辛苦老师再帮忙指导下哈

我报好号, 回头找你哈

OK

1.5年小伙搞定15K offer 就业寒冬涨100%

5月7日改简历 11月21日晒offer

秘诀:
简历指导+ 狠狠卷

他那个不止再修修嘛，我才是两位，原先7k，现在15k

那也很牛

都翻倍了，都是牛人

正常就30%

慢下来

下一步可以瞄准25k

其实我工作一年，

一年经验，搞定15k offer，舍你其谁

那种练习项目也行

卷王逆袭成功案例 6年小伙从很少面试机会到 搞定35K*14薪

3月5日改简历 4月11日拿offer

一个月拿到了理想的offer

面试邀请少 小伙很苦恼

理想很丰满 目标35K

面试法宝 rocketmq四部曲

6年 经验小伙伴 喜收25K offer

3月12日改简历 12月1日晒offer

秘诀:
简历指导+ 狠狠卷

咱们以这个项目为模板改哈

项目有多少人，你带领多少人？

你工作几年了？

6年了

改简历那晚，20涨到25k，高级开发，P7带的后端团队

任务重，道路远，并不是没有方向

7年经验卷王 薪酬涨30%

7月11日改简历 9月1日晒offer

秘诀:
改简历+ 狠狠卷

只要本事好，拿offer比较容易的

入职了，最终并不输于面试官，还差旧的大牛，只涨了30%

恭喜你

现在不比往年

能涨30%是大牛

拿到offer都算大牛

现在很多人投几百发，这个电话都没有

你已经很牛了

4年经验卷王逆袭 被毕业后，反涨24W

7月改简历 8月30日晒offer

秘诀：
改简历 + 狠卷

这就是你的简历
差得太多啦
老哥 我八月十号开始找工作，今天已入职了
能涨24W
原因大概是啥？
很多小伙伴，面试机会都没有
这个地方的话要这样写
项目被终止
方便语音沟通不

小伙5月份"被毕业"，改简历后 斩获顶级央企Offer 涨薪7000+

5月29日改简历 7月5日晒offer

秘诀：
简历指导 + 狠卷3高

快速看书，就能不求甚解，把自谈和场景大概一下，然后重点的地方，用到的地方，再去回顾
我被"毕业"了
这周末或下周找你改一下简历
毕业没有关系
ok，发我吧
R行业，就来说去，太复杂了
嗯，其实有点心理准备
不太会写简历
我拿到手写的offer了
涨20%，没数更多。结果人家都
看起来里面不差钱呀
超过了8000没
平均算下来
7000多
好的
有啥面试的心得吗
可以分享给其他小伙伴的
1.面试前多向面试官了解，面试官了解你的背景不感兴趣，可以提前准备好问题

卷王逆袭成功案例 武汉6年喜收4个优质offer 最高的年薪35W

2月9日改简历 4月15日晒offer

面试法宝：
改简历 + 实操

尼恩老师，新年好！
能帮忙修改下简历吗？
金三银四准备啦
可以的
java开发-6年-简历-...
拜托了，尼恩，希望能拿25K回来给你报喜呀
好，我加一下
还有吗？
恩恩，决赛圈offer了
截图是我自认能写出来的评分了，麻烦帮我参考下
选择大于努力，尼恩老师上岸
这么多offer，我看看哈
都是尼恩指点有方，本来还有个新能源汽车的，35w给拒了，主要太远了
跟着尼恩的时间太短了，目前实力也只能到这了
这里边有个大数据的，感觉也不错
继续跟着尼恩，马上技术自由啦

卷王逆袭成功案例 6年小伙喜提4个Offer 最高涨9k，年薪35W

4月14日改简历 5月17日晒offer

涨薪法宝：
改简历 + 狠卷

Java开发工程师...
你看着我给你的
好的呀
好的呀
谢谢大佬
这个你自己刷
不对的，你自己刷
那我照着这个改一下库存系统呀
一个简历，...
这么漂亮的简历，涨50%，已经没啥问题
只要准备好，不出大错，基本没问题啦
保证押金收起来，你的offer最高涨了9k，多返现100
后面继续跟着尼恩哈，感觉卷的时间...
谢谢
我准备这一年的时间都看呢
加油卷哈
感觉自己学的不太透彻了
嗯哪
跟着大佬一起
我周围好几个年薪百万的，都是这

卷王逆袭成功案例

5年经验小伙收2个offer
最高涨薪8k，年薪42W

5月9日改简历 5月30日晒offer

秘诀：
简历指导+ 狠卷3高

以此为例
大家狠狼卷
打造最卷IT社群



卷王逆袭成功案例

非全日制 6年经验卷王
喜提3个Offer，年包30W

5月9日改简历 5月18日晒offer

面试法宝：
改简历+ 狠狼卷

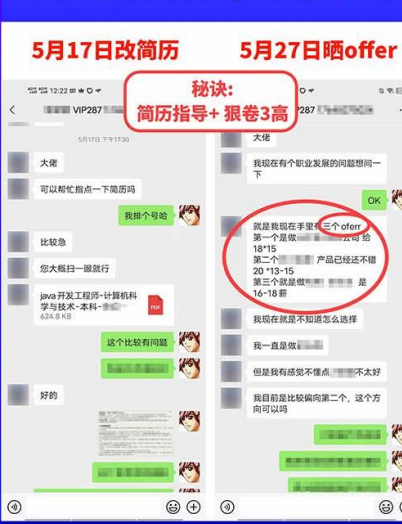


卷王逆袭成功案例

寒五冻六之际卷王大逆袭
收3大offer，涨30%

5月17日改简历 5月27日晒offer

秘诀：
简历指导+ 狠卷3高



卷王逆袭成功案例

4年卷王入职微软，涨50%

3月7日改简历 5月12日晒offer

涨薪法宝：
改简历+ 狠狼卷



4年小伙喜收百度、Boss直聘
等N个顶级Offer
最高涨幅100%

6月27日改简历

9月19日晒offer

秘诀:
改简历+狠狠卷

有个offer选择问题

boss直聘和度小满之间, boss那

还有好几个offer, 还有一个养

总体上是想着boss和度小满之间

了

boss比度小满年包多7w以上,

我这涨幅都快接近百分之百了

卷王逆袭成功案例

4年卷王入收2个offer, 涨50%

3月23日改简历

5月12日晒offer

offer决策

offer决策: n+1 电商erp, 部门成

又搞到个offer

涨薪法宝:
改简历+狠狠卷

涨50%的样子

小伙大三暑期很焦虑
跟着尼恩卷一年
校招斩获顶级央企Offer

去年5月19日加入VIP群

今年7月5日晒offer

秘诀:
狠狠卷书+视频

尼恩老师

我校招去华润电力控股有限公司了

跟着你卷了一年 大学顺便拿了几个

不错不错, 这是央企

这太牛啦

其实拿到手的也就一个s美国一

小伙高中学历
薪酬涨120%

5月6日改简历

7月22日晒offer

你这块估计要涨你668

模块的开发工作, 就这三个哈

哈哈, 不用了老师, 你帮我调了

就行 光今天晚上辅导就值几千了

老师很感谢

还有很多其他的模块

面试官要问

就是其他人做的

嘿嘿, 好

秘诀:
改简历+狠狠卷

5年卷王喜收2大Offer

最高涨5K

5月19日改简历

9月13日晒offer

秘訣:
改简历 + 狠涨卷

发我吧

辛苦了！

OK, 简历还需要好好改改哈

我主要介绍思路, 方法

嗯嗯, 我先消化一下。有问题请在请教

OK

12:25

崽哥, 我就搞成功了。目前手上有两家相对心仪的公司。崽哥, 有空给我参考下呗

恭喜恭喜！

一发是... 二发是... 三发是... 四发是... 五发是... 六发是... 七发是... 八发是... 九发是... 十发是... 十一发是... 十二发是... 十三发是... 十四发是... 十五发是... 十六发是... 十七发是... 十八发是... 十九发是... 二十发是... 二十一发是... 二十二发是... 二十三发是... 二十四发是... 二十五发是... 二十六发是... 二十七发是... 二十八发是... 二十九发是... 三十发是... 三十一发是... 三十二发是... 三十三发是... 三十四发是... 三十五发是... 三十六发是... 三十七发是... 三十八发是... 三十九发是... 四十发是... 四十一发是... 四十二发是... 四十三发是... 四十四发是... 四十五发是... 四十六发是... 四十七发是... 四十八发是... 四十九发是... 五十发是... 五十一发是... 五十二发是... 五十三发是... 五十四发是... 五十五发是... 五十六发是... 五十七发是... 五十八发是... 五十九发是... 六十发是... 六十一发是... 六十二发是... 六十三发是... 六十四发是... 六十五发是... 六十六发是... 六十七发是... 六十八发是... 六十九发是... 七十发是... 七十一发是... 七十二发是... 七十三发是... 七十四发是... 七十五发是... 七十六发是... 七十七发是... 七十八发是... 七十九发是... 八十发是... 八十一发是... 八十二发是... 八十三发是... 八十四发是... 八十五发是... 八十六发是... 八十七发是... 八十八发是... 八十九发是... 九十发是... 九十一发是... 九十二发是... 九十三发是... 九十四发是... 九十五发是... 九十六发是... 九十七发是... 九十八发是... 九十九发是... 一百发是...

OK, 我晚点看看回复哈

好的哈

5月21日 晚上20:24

崽哥, 今天有空看一看我的简历嘛? 帮稍微优化优化

后面跟着我在讯国一道

自研是啥公司呢

薪资涨幅都不差不多, 涨幅都不高, 都没达到你涨幅标准

感谢

卷王逆袭成功案例
双非二本小伙春招大翻身
喜提9大offer

2月22日改简历 **4月13日晒offer**

面试，能不能和我看看我的应聘被拒原因

java后端，薪资+年终奖
14.5k+12k
OK，稍晚一点发

好的谢谢哥哥

请问哥哥有时间的话，我也希望您了~

好的有

稍晚一点发，下午一起发下

好的~

面试法宝：
改简历+IM实操

您好，您面试下我公司的收获

还好像是哥哥帮我修改了一下简历
还有帮面试的的分布式em和app
应用

面试公司的面试官基本都是用这个程序

感谢您的指导

17:49

9大offer 最高年薪30万

公司	部门	岗位	薪资结构	年终奖
1. 字节跳动	能源数字化产品部	java后端开发	18.5k+14.5k+200k/月+500股票期权	<30w
2. 美团	交易研发部	16k+14k		22.4w
3. 快手	待定	java游戏开发	15k+15k+餐补+1000/月	22.5w
4. 腾讯		全栈	11k+13k	14.3w
5. 网易			14k	13.6w
6. 华为			9k	14.3w
7. 小米			9k+双休+项目津贴	13.6w
8. 京东			10k+双休+项目津贴	13.6w
9. 拼多多			9k+双休+项目津贴	13.6w

修改简历找尼恩（资深简历优化专家）

- 如果面试表达不好，尼恩会提供 简历优化指导
- 如果项目没有亮点，尼恩会提供 项目亮点指导
- 如果面试表达不好，尼恩会提供 面试表达指导

作为 40 岁老架构师，尼恩长期承担技术面试官的角色：

- 从业以来，“阅历”无数，对简历有着点石成金、改头换面、脱胎换骨的指导能力。
- 尼恩指导过刚刚就业的小白，也指导过 P8 级的老专家，都指导他们上岸。

如何联系尼恩。尼恩微信，请参考下面的地址：

语雀：<https://www.yuque.com/crazymakercircle/gkkw8s/khigna>

码云：<https://gitee.com/crazymaker/SimpleCrayIM/blob/master/疯狂创客圈总目录.md>